

中国银河证券格物金融服务平台
金融数据功能开发手册
(C++版)

目录

目录.....	1
1. 版本说明.....	1
1.1 文档管理信息表.....	1
1.2 文档变更记录表.....	1
2. 功能介绍.....	3
2.1 术语和缩略语.....	3
2.2 金融数据服务.....	3
2.3 两种使用场景.....	3
2.3.1 托管机房模式使用场景.....	3
2.3.2 互联网模式使用场景.....	3
2.4 数据内容.....	3
2.5 功能详情.....	4
2.5.1 功能详情.....	4
2.5.2 数据详情.....	5
3. C++ 开发指南.....	7
3.1 SDK.....	7
3.2 运行环境.....	8
3.2.1 操作系统及编译器版本.....	8
3.2.2 Linux 推荐运行环境配置.....	9
3.2.3 Windows 推荐运行环境配置.....	9
3.3 运行模式说明.....	9
3.3.1 托管机房模式.....	9
3.3.1.1 TCP 通道.....	10
3.3.1.2 QTCP 通道.....	10
3.3.1.3 RTCP 通道.....	10
3.3.1.4 TCP、QTCP、RTCP 通道同时使用的通道设置.....	10
3.3.2 互联网模式.....	11
3.4 API 开发步骤.....	11
3.4.1 初始化 TGW.....	12
3.4.2 异常释放资源处理.....	13
3.4.3 数据订阅.....	13
3.4.4 取消订阅.....	13
3.4.5 获取数据.....	14
3.5 API 接口详细.....	14
3.5.1 参数说明.....	14
3.5.2 基础接口.....	16
1) GetVersion 方法.....	16
2) Init 方法.....	16
3) Release 方法.....	16
4) FreeMemory 方法.....	17
5) GetTaskID 方法.....	17

	6) UpdatePassWord 方法	17
3.5.3	订阅数据方法.....	17
	1) Subscribe 方法	17
	2) UnSubscribe 方法.....	18
	3) SubFactor 方法	18
	4) UnSubFactor 方法.....	19
	5) SubscribeDerivedData 方法	19
3.5.4	订阅回调方法.....	20
	1) OnLog 方法	20
	2) OnEvent 方法	20
	3) OnIndicator 方法.....	20
	4) OnLogon 方法.....	21
	5) OnMDSnapshot 方法	21
	6) OnMDIndexSnapshot 方法.....	21
	7) OnMDOptionSnapshot 方法.....	21
	8) OnMDHKTSnapshot 方法	22
	9) OnMDAfterHourFixedPriceSnapshot 方法	22
	10) OnMDCSIIndexSnapshot 方法.....	22
	11) OnMDCnIndexSnapshot 方法	22
	12) OnMDHKTRealtimeLimit 方法	23
	13) OnMDHKTProductStatus 方法	23
	14) OnMDHKTVCM 方法	23
	15) OnMDFutureSnapshot 方法	23
	16) OnKLine 方法.....	24
	17) OnSnapshotDerive 方法	24
	18) OnFactor 方法.....	24
	19) OnMDOrderBook 方法.....	25
	20) OnMDOrderBookSnapshot 方法	25
3.5.5	查询数据方法.....	25
3.5.6	查询回调方法.....	31
	1) IGMDKlineSpi 接口	31
	2) IGMDSnapshotSpi 接口	31
	3) IGMDOrderQueueSpi 接口	33
	4) IGMDTickExecutionSpi 接口.....	34
	5) IGMDTickOrderSpi 接口	34
	6) IGMDCodeTableSpi 接口.....	34
	7) IGMDSecuritiesInfoSpi 接口.....	35
	8) IGMDExFactorSpi 接口	35
	9) IGMDFactorSpi 接口	35
	10) IGMDThirdInfoSpi 接口	36
	11) IGMDETFInfoSpi 接口	36
3.5.7	回放数据方法.....	36
	1) ReplayKline 方法	36
	2) ReplayRequest 方法	37

	3) CancelTask 方法	38
3.5.8	回放回调方法	38
	1) OnMDKline 方法	38
	2) OnMDSnapshot 方法	39
	3) OnMDTickExecution 方法	39
	4) OnRspTaskStatus 方法	39
3.6	开发 Demo 示例	40
3.7	编译运行步骤	46
3.8	开发注意事项	46
3.9	演示示例（获取和保存数据）	48
3.9.1	修改参数	48
3.9.2	运行启动脚本	49
3.9.3	开启订阅命令输入终端	49
3.9.4	查看订阅数据 csv 文件	50
3.9.5	tgw_test 程序订阅命令说明	50
4.	公共数据字典	53
4.1	事件级别(EventLevel)	54
4.2	错误码(ErrorCode)	54
4.3	数据定义长度(ConstField)	55
4.4	回放请求任务状态(HistoryTaskStatus)	56
4.5	数据类型(MDDatatype)	56
4.6	日志输出级别(LogLevel)	57
4.7	市场类型定义(MarketType)	57
4.8	API 通道模式(ApiMode)	57
4.9	托管机房模式通道(ColocatChannelMode)	58
4.10	订阅数据类型(SubscribeDataType)	58
4.11	品种类型定义(VarietyCategory)	59
4.12	衍生订阅类型(SubscribeType)	59
4.13	行情衍生数据订阅类型(SubscribeDerivedDataType)	59
5.	行情数据字典	60
5.1	K 线(MDKLine)	60
5.2	现货衍生(MDSnapshotDerive)	60
5.3	加工因子(Factor)	61
5.4	L1 现货快照(MDSnapshotL1)	61
5.5	指数快照(MDIndexSnapshot)	62
5.6	期权快照(MDOptionSnapshot)	62
5.7	港股通快照(MDHKTSnapshot)	63
5.8	期货快照(MDFutureSnapshot)	64
5.9	商业港股委托挂单(MDHKExOrderSnapshot)	65
5.10	商业港股执行动作(MDHKExOperation)	66
5.11	商业港股挂单买卖序列 (MDHKExListItem)	66
5.12	商业港股经纪商席位(MDHKExOrderBrokerSnapshot)	66
5.13	港股经纪席位详细(MDHKExDetailItem)	67
5.14	盘后定价交易快照(MDAfterHourFixedPriceSnapshot)	67

5.15	中证指数行情(MDCSIIIndexSnapshot)	67
5.16	国证指数快照(MDCnIndexSnapshot)	68
5.17	港股通实时额度(MDHKTRealtimeLimit)	68
5.18	港股通产品状态(MDHKTProductStatus)	69
5.19	港股 VCM(MDHKTVCN)	69
5.20	L2 现货快照(MDSnapshotL2)	70
5.21	现货逐笔成交(MDTickExecution)	71
5.22	逐笔委托(MDTickOrder)	71
5.23	委托队列(MDOrderQueue)	72
5.24	代码表(MDCCodeTable)	72
5.25	复权因子表(MDExFactorTable)	73
5.26	个股基础信息(MDStockInfo)	73
5.27	证券静态信息(MDCCodeTableRecord)	74
5.28	金融资讯数据(ThirdInfoData)	75
5.29	委托簿(MDOrderBook)	75
5.30	委托簿价格档位数据 (MDOrderBookItem)	76
5.31	委托簿快照(MDOrderBookSnapshot)	76
5.32	ETF 代码表(MDETFCCodeTableRecord)	77
5.33	ETF 成分股信息(ConstituentStockInfo)	78
6.	补充字段取值说明	80
6.1	交易所定义证券类型表 security_type	80
6.2	自定义证券子类别 security_type	81
6.3	证券代码表的 currency 取值	82
6.4	交易阶段代码取值	83
6.5	Security_status	83
6.6	K 线算法说明	84
6.7	商品期货查询规则	84
7.	附件 1: 资讯说明详细说明	86
7.1	股票基础信息	86
7.1.1	A 股基本资料 A010010001	86
7.1.1.1	请求参数说明	86
7.1.1.2	返回值说明	86
7.1.2	A 股行业分类 A010010002	87
7.1.2.1	请求参数说明	87
7.1.2.2	返回值说明	87
7.1.3	公司简介 A010010003	88
7.1.3.1	请求参数说明	88
7.1.3.2	返回值说明	88
7.1.4	股本结构 A010010004	91
7.1.4.1	请求参数说明	91
7.1.4.2	返回值说明	91
7.1.5	公司主营业务 A010010005	94
7.1.5.1	请求参数说明	94
7.1.5.2	返回值说明	95

7.1.6	历史股票列表(仅京沪深 A 股) A010010006.....	96
7.1.6.1	请求参数说明.....	96
7.1.6.2	返回值说明.....	96
7.1.7	历史股票列表(含期货期权、指数) A010010007.....	97
7.1.7.1	请求参数说明.....	97
7.1.7.2	返回值说明.....	97
7.2	发行与上市.....	98
7.2.1	A 股首次公开发行 A010020001	98
7.2.1.1	请求参数说明.....	98
7.2.1.2	返回值说明.....	98
7.2.2	A 股增发数据 A010020002	103
7.2.2.1	请求参数说明.....	103
7.2.2.2	返回值说明.....	103
7.3	收益分配.....	107
7.3.1	A 股股票分红 A010030001	107
7.3.1.1	请求参数说明.....	107
7.3.1.2	返回值说明.....	108
7.3.2	A 股股票配股 A010030002	109
7.3.2.1	请求参数说明.....	109
7.3.2.2	返回值说明.....	109
7.3.3	A 股股票除权除息 A010030003	112
7.3.3.1	请求参数说明.....	112
7.3.3.2	返回值说明.....	112
7.3.4	A 股复权因子表 A010030004	113
7.3.4.1	请求参数说明.....	113
7.3.4.2	返回值说明.....	113
7.4	股本与股东.....	115
7.4.1	A 股十大股东名单 A010040001	115
7.4.1.1	请求参数说明.....	115
7.4.1.2	返回值说明.....	115
7.4.2	限售股解禁明细 A010040002.....	116
7.4.2.1	请求参数说明.....	116
7.4.2.2	返回值说明.....	116
7.4.3	解禁数据 A010040003.....	117
7.4.3.1	请求参数说明.....	117
7.4.3.2	返回值说明.....	117
7.4.4	股权持股冻结/质押情况 A010040004.....	118
7.4.4.1	请求参数说明.....	118
7.4.4.2	返回值说明.....	118
7.5	财务数据.....	120
7.5.1	A 股一般企业资产负债表 A010050001	120
7.5.1.1	请求参数说明.....	120
7.5.1.2	返回值说明.....	120
7.5.2	A 股一般企业利润表 A010050002	129

7.5.2.1	请求参数说明.....	129
7.5.2.2	返回值说明.....	130
7.5.3	A 股一般企业现金流表 A010050003.....	136
7.5.3.1	请求参数说明.....	136
7.5.3.2	返回值说明.....	136
7.5.4	A 股财务指标 A010050004	144
7.5.4.1	请求参数说明.....	144
7.5.4.2	返回值说明.....	144
7.5.5	A 股业绩预告 A010050005	154
7.5.5.1	请求参数说明.....	154
7.5.5.2	返回值说明.....	155
7.5.6	A 股财务衍生指标 A010050006	156
7.5.6.1	请求参数说明.....	156
7.5.6.2	返回值说明.....	157
7.6	交易数据.....	164
7.6.1	交易日历(仅京沪深 A 股) A010060001.....	164
7.6.1.1	请求参数说明.....	164
7.6.1.2	返回值说明.....	164
7.6.2	交易日历(含期货期权) A010061001.....	165
7.6.2.1	请求参数说明.....	165
7.6.2.2	返回值说明.....	166
7.7	交易异动统计.....	167
7.7.1	大宗交易数据 A010070001.....	167
7.7.1.1	请求参数说明.....	167
7.7.1.2	返回值说明.....	167
7.7.2	交易异动营业部买卖信息 A010070002.....	168
7.7.2.1	请求参数说明.....	168
7.7.2.2	返回值说明.....	168
7.8	融资融券数据.....	169
7.8.1	融资融券交易明细 A010080001.....	169
7.8.1.1	请求参数说明.....	169
7.8.1.2	返回值说明.....	170
7.8.2	融资融券成交汇总 A010080002.....	171
7.8.2.1	请求参数说明.....	171
7.8.2.2	返回值说明.....	171
7.9	基金基本信息.....	172
7.9.1	基金最新指标 A010090001.....	172
7.9.1.1	请求参数说明.....	172
7.9.1.2	返回值说明.....	172
7.10	指数信息.....	172
7.10.1	A 股指数成分股 A010200001	172
7.10.1.1	请求参数说明.....	172
7.10.1.2	返回值说明.....	173
7.11	资讯数据信息.....	173

7. 11. 1	查询资讯数据更新日期 queryLastFuncDataTime	173
7. 11. 1. 1	请求参数说明.....	173
7. 11. 1. 2	返回值说明.....	174
7. 12	公共参数表.....	175
7. 12. 1	交易市场代码表.....	175
7. 12. 2	证券类别代码表.....	175
7. 12. 3	公司类型代码表.....	175
7. 12. 4	股票增发进度代码表.....	177
7. 12. 5	股票分红进度代码表.....	177
7. 12. 6	股票配股进度代码表.....	178
7. 12. 7	报表类型代码表.....	179

1. 版本说明

1.1 文档管理信息表

主题	中国银河证券格物金融服务平台金融数据功能开发手册 (C++版)
文档版本	V1.0.5
C++ SDK 版本	V1.0.4
创建时间	2023 年 1 月 28 日
创建人	中国银河证券信息技术部量化交易服务研发团队
最新发布日期	2023 年 10 月 10 日

1.2 文档变更记录表

修改人	修改时间	修改内容
张德高	2023 年 3 月 10 日	文档整体结构调整和细节表述优化 首次发布文档 V1.0.0 版本
郭顺祥	2023 年 5 月 25 日	SDK V1.0.3 功能变更： 1. 新增密码修改接口功能 2. 互联网模式支持查询复权因子表 3. 托管模式支持查询 SFS 代码表 4. 加工因子表查询及入库功能优化（新增字段参数） 5. QueryCodeTable 方法支持互联网模式 6. TGW_test 测试工具支持落地文件字段配置 7. 支持 ETF 基础信息及成份查询接口功能 8. Windows 版本集成 msvc 库 9. 支持订单簿及订单簿快照订阅 10. 支持账号多连接配置 11. 支持 CA 证书获取路径配置 更新发布文档 V1.0.1 版本
郭顺祥	2023 年 6 月 20 日	增加三方资讯数据字典信息 更新发布文档 V1.0.2 版本
郭顺祥	2023 年 8 月 01 日	增加三方资讯数据新接口： 1. A010050005 A 股业绩预告 2. A010010007 历史股票列表(含期货期权、指数) 3. A010061001 交易日历(含期货期权)

		更新发布文档 V1.0.3 版本
郭顺祥	2023 年 9 月 01 日	增加和完善三方资讯数据新接口： 1. A010050006 A 股财务衍生指标 2. 完善 A010010007 历史股票列表(含期货期权、指数)，区分证券类别 3. 新增 A010030004 复权因子表 4. 更改 A010040001A 股十大股东名单支持查看不同时间下的股东名单 更新发布文档 V1.0.4 版本
郭顺祥	2023 年 10 月 10 日	更新 SDK 版本至 1.0.4 功能变更： 1. 安装 python 版本不再需要安装证书 2. tgw-dataframe 输出格式中，error 返回错误码，互联网模式和托管模式查询返回空数据的方式进行统一 3. 修复多连接时查询带宽的分配缺陷 4. 优化 tgw-demo 程序输出接口中品种字段 5. tgw 的三方资讯和加工因子查询，新增分页查询功能 6. dgw 会定时将查询参数写到文件，写文件的时候会影响到查询性能 7. 修复权因子查询遗漏数据缺陷 8. 修复周末无法下载个股信息 9. 修复中金所期货数据查询时间段包含日盘和夜盘时间段时，未返回日盘时段数据问题 10. 修复期货合约快照在 security_code 字段过长无法查询的问题 11. 优化三方资讯查询、K 线查询性能 12. 新增港股历史经纪席位数据、挂单数据、成交数据查询功能 增加和完善三方资讯数据新接口： 1. 资产负债表、利润表、现金流表增加九号公司数据。 2. 新增 A010050005 A 股业绩预告 3. 新增 A010061001 期货期权和 A 股交易日历 增加系统功能详情说明 更新发布文档 V1.0.5 版本

2. 功能介绍

本文档是 tgw 的 SDK 开发指南，包含了对 API 接口的说明以及示例，用于指引开发人员通过 tgw 金融数据功能接口进行数据接收和查询的开发。

2.1 术语和缩略语

术语、缩写	解释
tgw	tgw，格物机构金融服务平台
SDK	Software Development Kit，金融数据功能软件开发工具包

2.2 金融数据服务

金融数据功能，是指用户使用 C++、Python 以及其他本功能可支持的程序设计语言或用户端页面，获取公司通过对证券交易所等渠道的公开信息加工而成的行情数据、金融资讯数据等金融数据的功能。

2.3 两种使用场景

tgw 包含了托管机房模式和互联网模式两种使用场景。

2.3.1 托管机房模式使用场景

在银河证券机房内部网络环境下，用户除 Level-1、K 线、资讯数据、因子数据等中低频数据外，通过 SDK 提供获取需要增强 Level-2 等高频数据。

2.3.2 互联网模式使用场景

在互联网网络环境下，用户通过 SDK 提供获取 Level-1、K 线、资讯数据、因子数据等中低频数据。

2.4 数据内容

托管机房模式和互联网模式都能提供的数据包括：

- 实时推送 K 线数据（1/3/5/10/15/30/60/120 分钟线）
- K 线历史数据（日/周/月/季年线、1/3/5/10/15/30/60/120 分钟线）
- 指数快照历史数据
- 期货快照历史数据

- 港股经纪席位历史数据、挂单历史数据、成交历史数据
- 实时推送快照衍生指标数据
- 实时推送加工因子数据
- 证券个股基本信息查询
- 证券代码表信息查询
- ETF 基本信息和成分股信息查询
- 复权因子表信息查询
- 加工因子历史数据
- 金融资讯数据查询

托管机房模式下的数据包括：

- Level2 现货快照历史数据
- Level2 历史委托队列数据
- Level2 历史逐笔委托数据
- Level2 历史逐笔成交数据
- 实时委托簿数据
- 委托簿生成实时快照数据

互联网模式下的数据包括：

- 实时推送 Level1 现货快照数据
- 实时推送指数快照数据
- 实时推送期权快照数据
- 实时推送期货快照数据
- 实时推送港股通快照数据
- 实时推送盘后定价交易快照数据
- 实时推送中证指数快照数据
- 实时推送深交所国证指数快照数据
- 实时推送港股通实时额度数据
- 实时推送港股通产品状态快照数据
- 港股市场波动调节机制(VCM)推送数据
- Level1 快照历史数据

2.5 功能详情

2.5.1 功能详情

两种模式下，tgw 提供的功能详情如下：

初始化模式	功能	提供数据内容
互联网模式/ 托管机房模式	订阅/取消订阅	K 线数据（1/3/5/10/15/30/60/120 分钟线）、Level1 现货快照数据、指数快照数据、期权快照数据、期货快照数据、港股通快照数据、盘后定价交易快照数据、中

		证指数快照数据、深交所国证指数快照数据、港股通实时额度数据、港股通产品状态快照数据、波动调节机制 (VCM) 推送数据
	订阅/取消订阅	因子数据
	订阅/取消订阅	订单簿数据、订单簿生成快照
	查询	K 线数据 (1/3/5/10/15/30/60/120 分钟线/周季月年线)、Level1 现货快照数据、指数快照数据、期权快照数据、期货快照数据、T 日最新代码表、T 日最新证券静态信息、T 日 ETF 静态信息和成分股信息、复权因子、因子历史数据、资讯数据
托管机房模式	查询	沪深 Level2 现货快照数据、沪深 Level2 逐笔委托数据、沪深 Level2 逐笔成交数据、沪深 Level2 委托队列数据
	回放	实现多代码快照数据回放、实现多代码逐笔成交数据回放、实现多代码 K 线数据回放

2.5.2 数据详情

两种模式下，tgw 目前支持数据类型以及数据时间跨度详细如下：

类型	品种	数据类型	数据起点	说明
行情数据	股票	Level-1、Level-2 快照	2013 年至今	上交所、深交所
	指数	逐笔成交、逐笔委托、委托队列数据		
	债券	K 线数据 (日/周/月/季年线、1/3/5/10/15/30/60/120 分钟线)		
	基金	行情快照衍生指标数据		
	期权	期权快照、K 线数据	2015 年至今	中金所期权、深交所 ETF 期权、上交所 ETF 期权
	期货	期货 Level-1、Level-2 快照数据	2010 年至今	上期所、大商所、郑商所、中金所、能源所
		K 线数据		
	港股	SF 级别股票指数快照、	2013 年至今	
	港股通	港股通行情快照	2022 年至今	
资讯数	基本信	公司信息、代码、上市板块等		

据	息			
	财务数据	财务报表、财务分析、经营状况等		
	权益数据	分红、派息等权益类事件		
因子数据	价量因子	集合竞价预撮合价量因子		
	高频因子	高频盘口因子数据、合成订单簿实时快照		
	日频因子	个股诊断因子、选股因子		
	衍生及指标	技术指标、涨跌模型数据、成交分价模型数据		

3. C++ 开发指南

金融数据功能提供 Windows 和 linux 两种操作系统下 C++和 Python 各两种 SDK 版本供投资者开发。

请参考以下指南进行 c++开发使用。

3.1 SDK

各操作系统版本 SDK 下载解压即可使用，目录结构如下：

Linux 系统下 C++版本 SDK 目录

```
. \c++
|——demo (样例程序目录)
|   |   make.sh           (编译样例程序脚本)
|   |   Makefile          (样例程序 tgw_demo 编译的 Makefile 文件)
|   |   tgw_demo.cpp      (样例程序)
|   |   tgw_push_spi.cpp  (实时推送 spi 及回调接口实现)
|   |   tgw_push_spi.h    (实时推送 spi 及回调接口实现)
|   |   tgw_query_spi.cpp (查询 spi 及回调接口实现)
|   |   tgw_query_spi.h   (查询 spi 及回调接口实现)
|   |   tgw_replay_spi.cpp(回放 spi 及回调接口实现)
|   |   tgw_replay_spi.h  (回放 spi 及回调接口实现)
|——include (头文件目录。使用时可直接 #include "tgw.h")
|   |   tgw.h              (API 推送和查询接口头文件)
|   |   tgw_datatype.h     (API 使用的类型定义)
|   |   tgw_export.h       (API 回放接口头文件)
|   |   tgw_history_spi.h  (API 使用的数据结构定义)
|   |   tgw_struct.h       (API 使用的数据结构定义)
|——test_tool
|   |   bin/tgw_test        (已编译好演示程序，无代码，使用时可直接运行 ./run.sh，运行前需更改配置文件)
|   |   etc/tgw.json        (tgw_test 演示程序配置文件目录)
|   |   run.sh              (tgw_test 演示程序运行脚本)
|——lib (依赖库文件目录)
```

Windows 系统下 C++版本 SDK 目录

```
. \c++
|——demo (样例程序目录)
|   |   tgw_demo.cpp      (样例程序)
|   |   tgw_push_spi.cpp  (实时推送 spi 及回调接口实现)
|   |   tgw_push_spi.h    (实时推送 spi 及回调接口实现)
|   |   tgw_query_spi.cpp (查询 spi 及回调接口实现)
```

```

|   tgw_query_spi.h   (查询 spi 及回调接口实现)
|   tgw_replay_spi.cpp(回放 spi 及回调接口实现)
|   tgw_replay_spi.h  (回放 spi 及回调接口实现)
|   └─project
|       tgw_demo.sln      (VC++2017 解决方案文件)
|       └─tgw_demo
|           tgw_demo.vcxproj (VC++2017 工程文件)
|           tgw_demo.vcxproj.filters (VC++2017 工程文件)
|           tgw_demo.vcxproj.user (VC++2017 工程文件)
|   └─include          (头文件目录。使用时可直接 #include "tgw.h")
|       tgw.h           (API 推送和查询接口头文件)
|       tgw_datatype.h  (API 使用的类型定义)
|       tgw_export.h
|       tgw_history_spi.h (API 回放接口头文件)
|       tgw_struct.h     (API 使用的数据结构定义)
|   └─test_tool
|       bin/tgw_test.exe  (已编译好演示程序，无代码，使用时可直接运行./run.sh，运行前需更改配置文件)
|       etc/tgw.json      (tgw_test 演示程序配置文件目录)
|       run.bat           (tgw_test 演示程序运行脚本)
|   └─lib                (依赖库文件目录)

```

使用前必须有中国银河证券股份有限公司授权的登录账户密码以及服务 IP 地址和端口。
建议使用 Telnet 命令验证网络连通性。

3.2 运行环境

3.2.1 操作系统及编译器版本

支持系统版本	编译器	备注
RedHat7.2 RedHat7.4 RedHat7.6	gcc4.8.5	支持多种 Level 1 或 Level 2 及自定义因子数据的订阅推送，支持快照，k 线，因子等数据查询，代码表查询，金融资讯数据查询
Windows10 (64 位)	VC++ 2017	支持多种 Level 1 或 Level 2 及自定义因子数据的订阅推送，支持快照，k 线，因子等数据查询，代码表查询，金融资讯数据查询

3.2.2 Linux 推荐运行环境配置

类型	最低配置	推荐配置
处理器	2.10GHz,4 核	2.10GHz,8 核
内存	DDR4 4GB	DDR4 4GB
硬盘	200G 机械硬盘/SSD	480G 机械硬盘/SSD
网卡	普通网卡	普通万兆网卡
操作系统	REDHAT 7.2/7.4/7.6	REDHAT 7.2/7.4/7.6

3.2.3 Windows 推荐运行环境配置

类型	最低配置	推荐配置
处理器	2.60GHz, 4 核	2.60GHz, 8 核
内存	DDR4 4GB	DDR4 4GB
硬盘	200G 机械硬盘/SSD	480G 机械硬盘/SSD
网卡	普通网卡	普通万兆网卡
操作系统	Windows 10(64 位)	Windows 10(64 位)

3.3 运行模式说明

tgw 包含托管机房模式和互联网模式。

通过在初始化启动时设置 IGMDApi::Init()接口中 api_mode 区分。

3.3.1 托管机房模式

三种通道 TCP、QTCP、RTCP，可单独使用，可两两结合，也可三个同时使用。

参数设置：

- (1) 通道模式 api_mode 设置：tgw.ApiMode.kColocationMode 为托管机房模式
- (2) 通道线程数 coloca_cfg.qtcp_channel_thread，默认 3。最小值为 1，最大值为 8。
- (3) 通道线程内数据条数 coloca_cfg.qtcp_max_req_cnt，默认 1000。最小值为 100，最大值为 3000。
- (4) 通道数据请求检查超时时间(单位为分钟) coloca_cfg.qtcp_req_time_out，默认为 10 分钟。最小值为 1000，最大值为 3000。
- (5) 是否启用订单簿订阅功能 enable_order_book，默认为 0 不启用委托簿功能，0：不启用委托簿，2：服务端主动推送委托簿数据
- (6) 委托簿价格档位数设置 entry_size，默认为 20 档。最小值为 1，最大值为 20。
- (7) 每个价位委托揭示委托笔数,order_queue_size，默认 50 笔。最小值为 1，最大值为

50。

- (8) 委托簿递交时间间隔(微秒级)`order_book_deliver_interval_microsecond`，默认 50ms，设置大于 50ms 按照指定值递送数据，小于 50ms 按照默认值 50ms 递送。

3.3.1.1 TCP 通道

- (1) 通道设置

`Cfg::ColocaCfg::channel_mode` 为 `ColocatChannelMode.kTCP`

- (2) 数据获取范围

只支持订阅接口的数据，不包含订阅范围外的数据。

- (3) 异常情况说明

- 1) 网络问题导致下游连接断开，接口会自动发起重连，无需调用特殊接口处理。
- 2) 网络堵塞或者下游处理速度太慢会导致消息堆积，如果一直没有改善，上游会将处理慢的下游连接断开，保证不同的连接之间不会互相影响。

3.3.1.2 QTCP 通道

- (1) 通道设置

`Cfg::ColocaCfg::channel_mode` 为 `ColocatChannelMode.kQTCP`

- (2) 数据获取范围

只支持查询接口的数据，不包含查询范围外的数据。

- (3) 异常情况说明

网络问题导致下游连接断开，接口会自动发起重连，无需调用特殊接口处理。

3.3.1.3 RTCP 通道

- (1) 通道设置

`Cfg::ColocaCfg::channel_mode` 为 `ColocatChannelMode.kRTCP`

- (2) 数据获取范围

只支持回放接口的数据，不包含回放范围外的数据。

- (3) 异常情况说明

网络问题导致下游连接断开，接口会自动发起重连，无需调用特殊接口处理。

3.3.1.4 TCP、QTCP、RTCP 通道同时使用的通道设置

- (1) 三个通道 TCP、QTCP、RTCP，可两两结合，也可三个同时使用。

- (2) 通过设置 `Cfg::ColocaCfg::channel_mod`，用 “|” 分开即可。

- (3) 例如：

```
galaxy::tgw::ColocatChannelMode::kQTCP|galaxy::tgw::ColocatChannelMode::kTCP
```

3.3.2 互联网模式

(1) 数据接入方式

通过 `websocket` 方式和上游保持连接，并进行行情数据的推送和查询。

(2) 通道模式 `api_mode` 设置

`tgw.ApiMode.kInternetMode` 为互联网模式

3.4 API 开发步骤

流程描述：

1. `tgw` 初始化配置

包含服务器地址、用户账号密码和运行模式配置

2. 基础接口包含初始化、订阅、退订、释放等方法，无需创建实例；

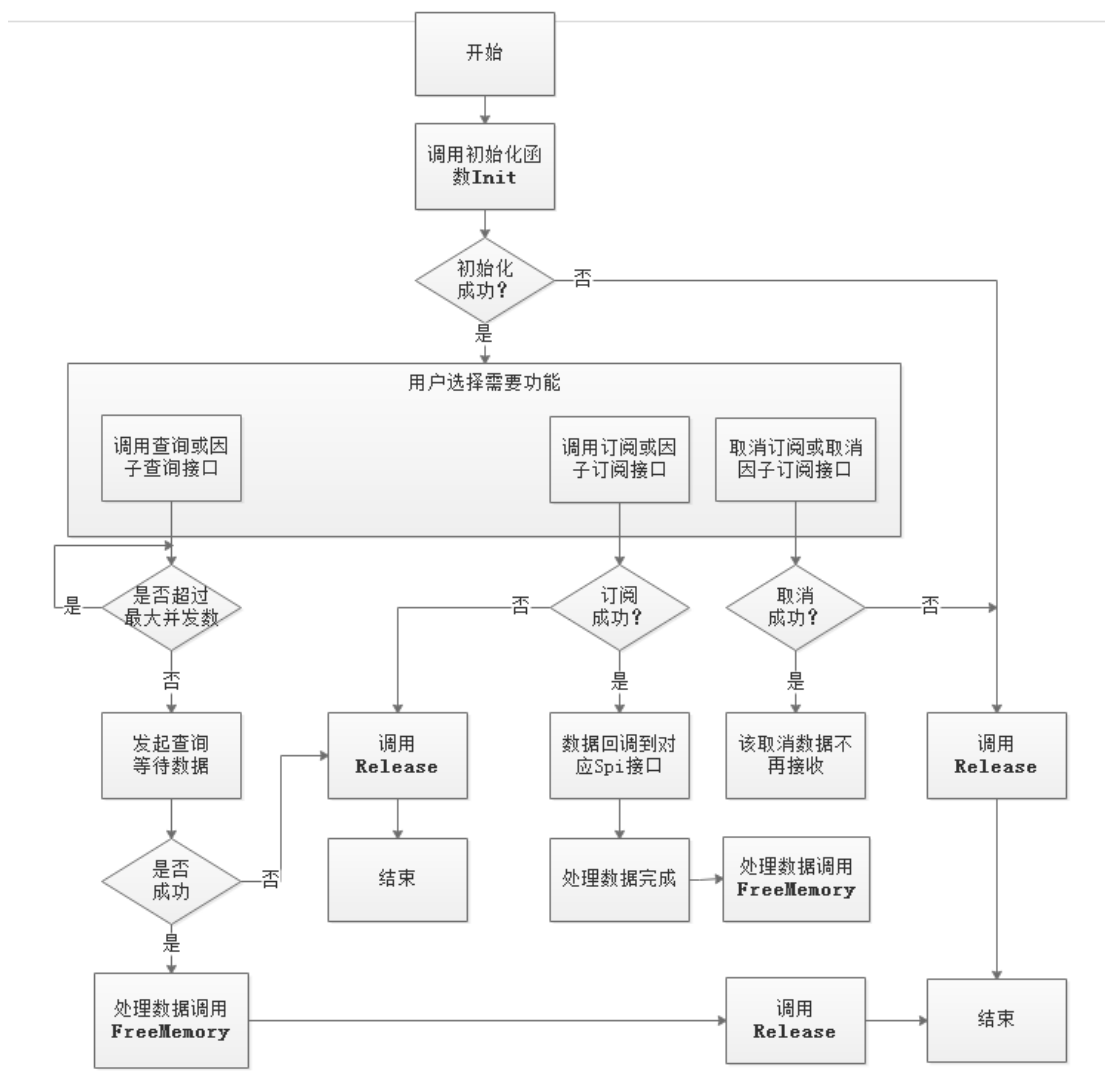
3. 数据接口包含订阅、查询和回放等三类方法；

(1) 继承所需的 `tgw` 中的 `IGMDSpi` 类，在派生类中重载回调函数；

(2) 派生类实例化；

(3) 派生类的实例传递给 `tgw.IGMDApi` 函数，供 `tgw` 回调使用；

(4) 派生类的实例生命周期长于 `tgw`，即派生类实例化必须在 `init` 初始化之前；



查询和订阅接口使用流程图

3.4.1 初始化 TGW

首先调用初始化函数 `IGMDApi::Init`，该函数传入用户申请后的账户密码、接入行情网关的 IP 地址和端口，以及注册回调函数类对象。

```

galaxy::tgw::Cfg cfg;
strcpy(cfg.server_vip, "127.0.0.1", sizeof(cfg.server_vip));
cfg.server_port = 9200;
strcpy(cfg.username, "*****", sizeof(cfg.username));
strcpy(cfg.password, "*****", sizeof(cfg.password));
uint16_t api_mode = galaxy::tgw::ApiMode::kColocationMode; // 设置 api 模式 托管机房模式/互联网模式
cfg.coloca_cfg.channel_mode = galaxy::tgw::ColocatChannelMode::kQTCPP |
    galaxy::tgw::ColocatChannelMode::kQTCPP |
    galaxy::tgw::ColocatChannelMode::kRTCP;;
if (api_mode == galaxy::tgw::ApiMode::kColocationMode)
{
    cfg.coloca_cfg.channel_mode = galaxy::tgw::ColocatChannelMode::kQTCPP
    cfg.coloca_cfg.qtcp_channel_thread = 8;
    cfg.coloca_cfg.qtcp_max_req_cnt = 1000;
    cfg.coloca_cfg.qtcp_req_time_out = 10;
    cfg.coloca_cfg.enable_order_book = galaxy::tgw::OrderBookType::kServerOrderBook;
    cfg.coloca_cfg.entry_size = 10;
    cfg.coloca_cfg.order_queue_size = 10;
    cfg.coloca_cfg.order_book_deliver_interval_microsecond = 10000;
}
  
```

```

}

galaxy::tgw::TgwPushSpi spi;
// 初始化 TGW
int32_t ec = galaxy::tgw::ErrorCode::kSuccess;
if ((ec = galaxy::tgw::IGMDApi::Init(&spi, cfg, api_mode)) != galaxy::tgw::ErrorCode::kSuccess)
{
    galaxy::tgw::IGMDApi::Release();
    return -1;
}

```

3.4.2 异常释放资源处理

初始化成功则可执行订阅或者取消订阅操作,初始化失败则调用 `IGMDApi::Release` 函数释放资源后退出。

3.4.3 数据订阅

订阅时调用接口 `IGMDApi::Subscribe`, 需要填写订阅参数 `SubscribeItem` 并传入该接口, 参数包括订阅的市场类型、数据类型、证券代码。接口可被多次调用多次订阅, 如果用户没有被限制订阅数量则可以填写订阅所有市场类型, 所有数据类型以及证券代码可提供所有选项, 从而可以订阅全市场行情。若有限制订阅数则只能单市场单证券代码多次订阅, 此时数据类型不受限制可填写所有类型。如果连续多次调用订阅接口, 可参照以下 **demo** 示例, 用数组或者其他方式存储参数进行多次订阅。

```

// 设置订阅参数, 以两次订阅为例
galaxy::tgw::SubscribeItem sub_cfg[2];

// 第一次订阅的市场类型设置
sub_cfg[0].market = galaxy::tgw::MarketType::kSZSE;
// 第一次订阅的数据类型设置
sub_cfg[0].flag = galaxy::tgw::SubscribeDataType::kNone;
// 第一次订阅的证券代码设置
strcpy(sub_cfg[0].security_code, "000001", sizeof(sub_cfg[0].security_code));
// 第一次订阅的品种类别设置
sub_cfg[0].category_type = galaxy::tgw::VarietyCategory::kNone;

// 第二次订阅的市场类型设置
sub_cfg[1].market = galaxy::tgw::MarketType::kNone;
// 第二次订阅的数据类型设置
sub_cfg[1].flag = galaxy::tgw::SubscribeDataType::kNone;
// 第二次订阅的证券代码设置, 为空代表全部证券代码
strcpy(sub_cfg[1].security_code, "", sizeof(sub_cfg[1].security_code));
// 第二次订阅的品种类别设置
sub_cfg[1].category_type = galaxy::tgw::VarietyCategory::kStock;

// 发送多个订阅请求
auto ec = galaxy::tgw::IGMDApi::Subscribe(sub_cfg, 2);
if (ec != galaxy::tgw::ErrorCode::kSuccess)
    std::cout << "GeneralSubscribe failed, ErrorCode: " << ec << std::endl;

```

3.4.4 取消订阅

取消订阅调用接口 `IGMDApi::UnSubscribe`, 参数设置与订阅接口相同, 取消订阅后将不再接收该类型的数据。

3.4.5 获取数据

订阅成功后，对应数据将回调到提供的接口，如 `OnMDSnapshot()` 方法将接收到现货快照数据，具体数据类型对应不同接口，用户可选择实现或者不处理，在完成对应处理后，必须调用 `FreeMemory()` 方法释放传入的指针内存，否则将造成内存泄漏。各种数据回调处理函数接口请参看下面的 `IGMDSpi` 接口详细介绍。

3.5 API 接口详细

3.5.1 参数说明

参数类型

- 入参 (in) 代表必须输入的参数。
- 出参 (out) 表示程序输出的参数。

精度说明

- 取值范围：为兼容多市场，API 为对各行情字段的取值范围规则如下：整形以 `int64` 表示，范围不限定，字符串以各行情字段设定为准，浮点型以 `int64` 表示，范围不限定。
- 价格精度：价格类型 (`pre_close_price`、`open_price`、`last_price`、`high_price` 等) 小数位数为 6，实际值需除以 1000000。
- 总金额精度：总金额 (`total_value_trade`) 小数位为 5，实际值需除以 100000。
- 总数量精度：总数量 (`total_volume_trade`) 小数位为 2，实际值需除以 100。

具体业务字段含义及精度说明如下：

业务字段	精度（不区分沪深）	转换规则
价格 (各种价格、均价)	整型，后 6 位表示小数位	价格、数量、金额字段统一全是 <code>int</code> 型。例如，收到价格后，除以 1000000 表示真正的价格。同样的，收到的量除以 100 得到真实量，收到的金额除以 100000 得到真实金额。
量（各种数量）	整型，后 2 位表示小数位	
金额（各种金额）	整型，后 5 位表示小数位	
比率（各种比率，百分率）	整型，后 5 位表示小数位	系统统一乘 10 的 5 次方。数据统一除以 100000 得到真实值。
期货价格	0 表示价格为空	
个股信息：昨收	整型，后 6 位表示小	收到昨收价格后，除以

	数位	1000000 表示真正的价格。
汇率	整型，后 8 位表示小数位	收到汇率数据后，除以 100000000 表示真正的汇率值
比例（涨跌幅、市盈率等）	整型，后 6 位表示小数位	涨跌幅一类比例数据需要除以 1000000
板块指数（含高开低收）	整型，后 6 位表示小数位	收到最新指数后，除以 1000000 表示最新指数。
换手率/涨跌幅/振幅/委比	整型，后 5 位表示小数位	收到换手率后，除以 100000 得到真实值。

单位说明

市场 证券品种	上海	深圳
股票	1 股	1 股
债券	10 张	1 张
基金	1 份	1 份
期权	1 张	1 张

常量定义

开发所需数据结构中对数据长度的常量定义请参考：

```
class ConstField
{
public:
    static const uint32_t kIPMaxLen           = 24;        // IP 地址的最大长度
    static const uint32_t kUMSItemLen         = 10;        // 服务项信息的最大个数
    static const uint32_t kUsernameLen        = 32;        // 用户名的最大长度
    static const uint32_t kPasswordLen        = 64;        // 用户名密码的最大长度
    static const uint32_t kSecurityCodeLen     = 16;        // 证券代码最大长度
    static const uint32_t kHistorySecurityCodeLen = 32;     // 历史回放和查询证券代码最大长度
    static const uint32_t kPushSecurityCodeLen = 32;        // 实时推送证券代码最大长度
    static const uint32_t kSecurityNameLen     = 32;        // 证券名称最大长度
    static const uint32_t kTradingPhaseCodeLen = 8;         // 交易状态标志
    static const uint32_t kExchangeInstIDLen  = 31;        // 合约在交易所代码
    static const uint32_t kPositionLevelLen   = 10;        // 行情档位
    static const uint32_t kMDStreamIDLen      = 6;         // 行情类别
    static const uint32_t kSecurityStatusLen   = 24;        // 证券状态最大长度
    static const uint32_t kSecuritySymbolLen   = 32;        // 证券简称最大长度
    static const uint32_t kSecurityEnglishNameLen = 128;    // 证券英文简称最大长度
    static const uint32_t kSecurityTypeLen     = 10;        // 证券类别长度
    static const uint32_t kCurrencyLen         = 4;         // 币种长度
    static const uint32_t kVolumeTradeLen      = 24;        // 成交量长度
    static const uint32_t kValueTradeLen       = 24;        // 交易额长度
    static const uint32_t kFutureSecurityCodeLen = 32;      // 期货代码最大长度
    static const uint32_t kMDStreamIDMaxLen    = 6;         // 行情类别字段最大长度
    static const uint32_t kDataPermission      = 128;       // 数据权限最大长度
    static const uint32_t kTokenMaxLen         = 128;       // token 的最大长度
    static const uint32_t kQuerySecurityCodeLen = 38;       // 查询状态回调代码最大长度
    static const uint32_t kTradingStatusLen    = 8;         // 证券交易状态档位
    static const uint32_t kSymbolLen           = 128;       // 证券简称最大长度（包括中文简称）
    static const uint32_t kSecurityAbbreviationLen = 64;    // 证券简称最大长度
    static const uint32_t kMaxTypesLen         = 16;        // 证券类型最大长度
    static const uint32_t kTypesLen            = 8;         //
    static const uint32_t kCodeTableSecurityStatusMaxLen = 16; // 代码表证券状态字符最大长度
}
```

```

static const uint32_t RegularShare          = 9;           // 对应回购标准券长度
static const uint32_t kConsSecurityCodeLen   = 32;         // 成份证券代码长度
static const uint32_t kSymbolETFLen        = 128;         // 基金名称长度
static const uint32_t KUnderlyingSecurityID  = 4;          // 成分股所属市场 ID 长度
static const uint32_t KReserved             = 30;          // 预留字段长度
static const uint32_t kManagmentETFLen      = 128;         // 基金公司名称长度
static const uint32_t KUnderlyingSecurityIDSource = 4;      // 拟合指数代码源长度
static const uint32_t AllCashAmount         = 12;          // 全现金替代的总金额长度
static const uint32_t AllCashAremiumRate    = 7;           // 全现金替代的申购溢价比例长度
static const uint32_t AllCashDiscountRate   = 7;           // 全现金替代的赎回折价比例长度
static const uint16_t DefaultMaxOBSSubNum   = 500;         // 委托簿默认最大订阅数量
static const uint16_t DefaultMaxOBSnapshotSubNum = 3000;    // 委托簿快照默认最大订阅数量
static const uint16_t DefaultMaxOBOOrderQueueSize = 50;     // 每一档委托队列数量上限
static const uint16_t DefaultMaxOBEntrySize = 20;          // 委托簿档位数量上限
static const uint16_t DefaultMaxOBDeliverIntervalMs = 50;   // 委托簿最小推送间隔
static const uint16_t DefaultMinOBSnapshotDeliverIntervalMs = 10; // 委托簿快照最小推送间隔
};

```

3.5.2 基础接口

TGW 接口操作类，主要用于推送接口。该类不需要创建实例，直接调用类方法即可。如 `IGMDApi::Release()`。

1) GetVersion 方法

获取 TGW 版本信息，托管机房和互联网模式适用。

函数原型：

```
static const char* GetVersion();
```

返回值：返回版本号信息。

2) Init 方法

初始化 TGW，托管机房和互联网模式适用。

函数原型：

```
static int32_t Init(const IGMDSpi* pSpi, const Cfg& cfg, uint16_t api_mode,
const std::string& path = "");
```

参数：

参数	解释
pSpi(in)	IGMDSpi 的派生类实例指针，必须在调用 Release 函数之后才能销毁该实例
cfg (in)	TGW 内部需要的配置参数
api_mode(in)	模式选择，取值请参照公共数据字典中的 ApiMode
path(in)	互联网模式指定证书及相关路径（无特殊用法可不填写，使用默认路径即可）

返回值：返回错误码，参照公共数据字典中 [ErrorCode](#) 定义。

3) Release 方法

释放 TGW，托管机房和互联网模式适用。

函数原型:

```
static int32_t Release();
```

返回值: 返回错误码, 参照公共数据字典中 [ErrorCode](#) 定义。

4) FreeMemory 方法

释放内存, 托管机房和互联网模式适用。

TGW 行情数据回调后, 其数据指针所有权归应用所有, 所以调用该接口释放内存。回调除 OnLog 外, 其余都需要显式地调用 FreeMemory 释放内存, 否则会造成内存泄漏。

函数原型:

```
static void FreeMemory(void* data);
```

参数:

参数	解释
data(in)	需要被释放的内存首地址

5) GetTaskID 方法

获取唯一的回放任务 id 值, 托管机房和互联网模式适用。

函数原型:

```
static int64_t GetTaskID();
```

说明:

返回值为回放任务 id, 格式为 MMDDHHmmSS+序列号 (1~1000000)。例如: 524153030000001 (5 月 24 日 15 时 30 分 30 秒, 序列号 1)

6) UpdatePassWord 方法

通过旧密码更新密码接口, 托管机房和互联网模式适用 (托管机房模式需开启 TCP 通道)。

函数原型:

```
static int32_t UpdatePassWord(const UpdatePassWordReq& req);
```

说明:

返回错误码, 参照 tgw_datatype.h 中的 ErrorCode。

错误码为超时 kTimeoutExit 则大概率是失败, 仍有小概率网络问题会修改成功, 需要注意该错误码。

3.5.3 订阅数据方法

1) Subscribe 方法

订阅行情数据, 托管机房和互联网模式适用。

函数原型:

```
static int32_t Subscribe(const SubscribeItem*item,uint32_t cnt);
```

参数:

参数	解释
*item(in)	订阅信息数据结构体(支持一次设置多个订阅信息)
cnt(in)	订阅信息数量

SubscribeItem 结构定义如下:

数据类型	数据名称	说明
uint8_t	market	市场类型, 参考 MarketType, 为 0 表示订阅所有支持的市场
uint64_t	flag	各数据类型的集合, 为 0 表示订阅所有支持的数据类型, 参考公共字典中的 SubscribeDataType
char	security_code[ConstField::kFutureSecurityCodeLen]	证券代码, 为空表示订阅所有代码
uint8_t	category_type	品种类别, 参考 VarietyCategory , 为 0 表示订阅所有支持的品种, 仅该参数仅互联网模式有效

2) UnSubscribe 方法

取消行情数据订阅, 托管机房和互联网模式适用。

函数原型:

static int32_t UnSubscribe(const SubscribeItem* item, uint32_t cnt);

参数:

参数	解释
*item(in)	取消订阅信息数据结构体(一次可取消多个订阅信息)
cnt(in)	取消订阅信息数量

其中, SubscribeItem 结构定义参照 Subscribe 方法:

3) SubFactor 方法

增加加工因子数据订阅, 托管机房和互联网模式适用。

函数原型:

static int32_t SubFactor(const SubFactorItem* item, uint32_t cnt);

参数:

参数	解释
*item(in)	订阅信息数据结构体(支持一次设置多个订阅信息)
cnt(in)	订阅信息数量

SubFactorItem 结构定义如下:

数据类型	数据名称	说明
char	factor_type[64]	因子父类型(英文)
char	factor_sub_type[64]	因子子类型(英文)
char	factor_name[64]	因子名称(英文)

全订阅仅支持以下三种方式：

- 1、全订阅支持 all all all 方式，表示订阅所有权限的组合；
- 2、支持 xxx all all 方式订阅（指定父类型），表示订阅某一父类型下子类型和因子名称的组合；
- 3、支持 xxx xxx all 方式订阅（指定父类型、子类型），表示订阅某一父类型和子类型下的全部因子名称；

4) UnSubFactor 方法

取消加工因子数据订阅，托管机房和互联网模式适用。

函数原型：

```
static int32_t UnSubFactor(const SubFactorItem * item, uint32_t cnt);
```

参数：

参数	解释
*item(in)	取消订阅信息数据结构体(一次可取消多个订阅信息)
cnt(in)	取消订阅信息数量

其中，SubFactorItem 结构定义参照 SubFactor 方法

取消全订阅仅支持以下三种方式：

- 1、取消全订阅支持 all all all 方式，表示取消订阅所有权限的组合；
- 2、支持 xxx all all 方式取消订阅（指定父类型），表示取消订阅某一父类型下子类型和因子名称的组合；
- 3、支持 xxx xxx all 方式取消订阅（指定父类型、子类型），表示取消订阅某一父类型和子类型下的全部因子名称；

5) SubscribeDerivedData 方法

衍生数据订阅，托管机房适用。

函数原型：

```
static int32_t SubscribeDerivedData(int32_t subscribe_type,
    uint32_t derived_data_type,
    const SubscribeDerivedDataItem* item, uint32_t cnt);
```

参数：

参数	解释
subscribe_type(in)	订阅操作类型，请参考 SubscribeType kSet:重新设置订阅(先取消所有旧订阅信息，然后增加新订阅信息) kAdd:增加订阅 kDel:删除订阅 kCancelAll:取消所有订阅
derived_data_type(in)	订阅衍生数据类型 参考 SubscribeDerivedDataType 1: 订阅委托簿数据 2: 订阅委托簿快照数据
*item(in)	衍生数据类型对应的订阅代码数据项，具体参数请参考 SubscribeDerivedDataItem 结构定义
cnt(in)	订阅信息数量

SubscribeDerivedDataItem 结构定义如下：

数据类型	数据名称	说明
int	market	市场类型：参考 MarketType, 行情衍生数据支持市场范围[kSSE/kSZSE], 其余市场暂时不支持
char	security_code[ConstField::kSecurityCodeLen]	证券代码 ConstField::kSecurityCodeLen 具体数值参看本文公共字典

3.5.4 订阅回调方法

TGW 中的接收数据的回调基类。使用 TGW 时需要继承该类，并将派生类的实例传递给 IGMDApi::Init 函数，供 TGW 回调使用。为保证程序正确运行，必须保证该实例生命周期长于 TGW。（接口方法默认支持托管机房和互联网模式，仅单一模式支持时将后缀说明）。

1) OnLog 方法

接收日志数据的回调函数，用户可选择如何处理。

函数原型：

```
void OnLog(const int32_t& level, const char* log, uint32_t len);
```

参数：

参数	解释
level(out)	日志数据级别，详见公共数据字典 LogLevel
log(out)	日志内容
len(out)	日志内容长度

2) OnEvent 方法

接收事件通知回调，使用者可根据该回调事件做相应的处理（仅托管机房模式适用）。

函数原型：

```
void OnEvent(uint32_t level, uint32_t code, const char* event_msg, uint32_t len);
```

参数：

参数	解释
level(out)	事件级别，参考公共字典 EventLevel
code(out)	事件代码
event_msg(out)	事件具体信息
len(out)	事件具体信息长度

3) OnIndicator 方法

接收监控数据回调（仅托管机房适用）。

函数原型:

```
void OnIndicator(const char* indicator, uint32_t len);
```

参数:

参数	解释
indicator(out)	监控数据内容, 格式为 JSON 字符串
len(out)	监控内容长度

4) OnLogon 方法

接收登录成功时的信息, 使用者可根据需求对该回调信息做相应的处理。

函数原型:

```
void OnLogon(LogonResponse* data);
```

参数:

参数	解释
data(out)	登录成功后的校验信息

LogonResponse 结构定义如下

数据类型	数据名称	说明
uint16_t	api_mode	1: 托管机房 2: 互联网
uint32_t	logon_msg_len	登录返回信息长度
char*	logon_json	登录返回信息, 格式为 json

5) OnMDSnapshot 方法

接收现货快照推送数据回调 (仅互联网模式适用)。

函数原型:

```
virtual void OnMDSnapshot(MDSnapshotL1* data, uint32_t cnt);
```

参数:

参数	解释
data(out)	现货快照数据首指针, 需要显式地调用 FreeMemory
cnt(out)	数据个数

6) OnMDIndexSnapshot 方法

接收指数快照推送数据回调 (仅互联网模式适用)。

函数原型:

```
virtual void OnMDIndexSnapshot(MDIndexSnapshot* data, uint32_t cnt);
```

参数:

参数	解释
data(out)	指数快照数据结构体
cnt(out)	数据个数

7) OnMDOptionSnapshot 方法

接收期权快照推送数据回调 (仅互联网模式适用)。

函数原型:

```
virtual void OnMDOptionSnapshot(MDOptionSnapshot* data, uint32_t cnt);
```

参数:

参数	解释
data(out)	期权快照数据结构体
cnt(out)	数据个数

8) OnMDHKTSnapshot 方法

接收港股通快照推送数据回调（仅互联网模式适用）。

函数原型:

```
virtual void OnMDHKTSnapshot(MDHKTSnapshot* data, uint32_t cnt);
```

参数:

参数	解释
data(out)	港股快照数据结构体
cnt(out)	数据个数

9) OnMDAfterHourFixedPriceSnapshot 方法

接收收盘后定价交易快照数据回调（仅互联网模式适用）。

函数原型:

```
Virtual void OnMDAfterHourFixedPriceSnapshot  
(MDAfterHourFixedPriceSnapshot* data, uint32_t cnt);
```

参数:

参数	解释
data(out)	接收收盘后定价交易快照数据结构体
cnt(out)	数据个数

10) OnMDCSIIndexSnapshot 方法

接收中证指数快照数据回调（仅互联网模式适用）。

函数原型:

```
virtual void OnMDCSIIndexSnapshot(MDCSIIndexSnapshot* data, uint32_t  
cnt);
```

参数:

参数	解释
data(out)	接收中证指数快照数据结构体
cnt(out)	数据个数

11) OnMDCnIndexSnapshot 方法

接收深交所国证指数快照数据回调（仅互联网模式适用）。

函数原型:

```
virtual void OnMDCnIndexSnapshot(MDCnIndexSnapshot* data, uint32_t cnt);
```

参数:

参数	解释
----	----

data(out)	接收深交所国证指数快照数据结构体
cnt(out)	数据个数

12) OnMDHKTRealtimeLimit 方法

接收港股通实时额度数据回调（仅互联网模式适用）。

函数原型：

```
virtual void OnMDHKTRealtimeLimit(MDHKTRealtimeLimit* data, uint32_t cnt);
```

参数：

参数	解释
data(out)	港股通实时额度推送数据结构体
cnt(out)	数据个数

13) OnMDHKTProductStatus 方法

接收港股通产品状态数据回调（仅互联网模式适用）。

函数原型：

```
virtual void OnMDHKTProductStatus(MDHKTProductStatus* data, uint32_t cnt);
```

参数：

参数	解释
data(out)	接收港股通产品状态快照数据结构体
cnt(out)	数据个数

14) OnMDHKTVCM 方法

接收港股 VCM 数据回调（仅互联网模式适用）。

函数原型：

```
virtual void OnMDHKTVCM(MDHKTVCVM* data, uint32_t cnt);
```

参数：

参数	解释
data(out)	接收港股 VCM 推送数据结构体
cnt(out)	数据个数

15) OnMDFutureSnapshot 方法

接收期货快照数据回调（仅互联网模式适用）。

函数原型：

```
virtual void OnMDFutureSnapshot(MDFutureSnapshot* data, uint32_t cnt);
```

参数：

参数	解释
data(out)	接收期货快照推送数据结构体
cnt(out)	期货快照数据条数

16) OnKLine 方法

接收历史 K 线数据回调。

函数原型：

```
virtual void OnKLine(MDKLine* data, uint32_t cnt, uint32_t kline_type);
```

参数：

参数	解释
data(out)	接收历史 K 线数据推送的数据结构体
cnt(out)	数据个数
kline_type(out)	K 线类型

17) OnSnapshotDerive 方法

接收现货衍生数据的推送回调。

函数原型：

```
virtual void OnSnapshotDerive(MDSnapshotDerive* data, uint32_t cnt);
```

参数：

参数	解释
data(out)	接收现货衍生数据结构体
cnt(out)	数据个数

18) OnFactor 方法

接收加工因子数据的推送回调。

函数原型：

```
virtual void OnFactor(Factor* data);
```

参数：

参数	解释
data(out)	加工因子数据首指针，需要显式地调用 FreeMemory

Factor 结构定义如下：

数据类型	数据名称	说明
uint32_t	data_size	json 数据的大小
char*	json_buf	json 结构

json_buf 结构如下：

```
{
  "headers":
  {
    "factor_name":,           // 因子名称
    "factor_type":,          // 因子父类型
    "factor_sub_type":,      // 因子子类型
    "origin_time":,          // 数据原始时间
    "security_code":,        // 证券代码
    "market":,               // 市场类型
    "category":,             // 品种类型
    "key1":,                  // 预留字段 1
    "key2":,                  // 预留字段 2
    "seq_num":,              // 因子序号
  }
  "body":                    // 透传的部分
```

```

{
    .....
}

```

19) OnMDOrderBook 方法

接收委托簿数据回调。

函数原型：

```
Virtual void OnMDOrderBook(std::vector<MDOrderBook>& order_book);
```

参数：

参数	解释
order_book (out)	委托簿数据结构，定义参看 MDOrderBook 结构

20) OnMDOrderBookSnapshot 方法

接收委托簿快照数据回调。

函数原型：

```
Virtual void OnMDOrderBookSnapshot(
    MDOrderBookSnapshot* order_book_snapshots, uint32_t cnt);
```

参数：

参数	解释
order_book_snapshots (out)	委托簿快照数据首指针，需要显示的调用 FreeMemory 快照数据定义参看 MDOrderBookSnapshot 结构
cnt(out)	快照数据个数

3.5.5 查询数据方法

1) QueryKline 方法

K 线查询接口，托管机房和互联网模式适用。

函数原型：

```
static int32_t QueryKline(const IGMDKlineSpi* kline_spi, const ReqKline&
    req_kline);
```

参数：

参数	解释
kline_spi(in)	IGMDKlineSpi 的派生类实例指针，必须在调用 Release 函数之后才能销毁该实例（对应数据回调为 IGMDKlineSpi 下的 OnMDKLine 方法）
req_kline(in)	K 线查询数据项首地址

ReqKline 结构定义如下：

数据类型	数据名称	说明
char	security_code[ConstField::kQuerySecurityCodeLen]	证券代码

	]	
uint8_t	market_type	市场类型
uint8_t	cq_flag	除权标志 (0:不复权 1:向前复权 2:向后复权默认 0)
uint32_t	cq_date	除权日期 (yyyMMdd) (暂不使用)
uint32_t	qj_flag	全价标志 (债券) (0: 净价, 1: 全价) (暂不使用)
uint16_t	cyc_type	数据周期 (参考 tgw_datatype.h 中的 MDDatatype 描述)
uint32_t	cyc_def	周期数量 (暂不使用)
uint8_t	auto_complete	自动补全 (0:不补齐, 1: 补齐默认 1)
uint32_t	begin_date	开始日期 (yyyyMMdd)
uint32_t	end_date	结束日期 (yyyyMMdd)
uint32_t	begin_time	开始时间 (默认: HHmm, 支持 HHmmssSSS)
uint32_t	end_time	结束时间 (默认: HHmm, 支持 HHmmssSSS)

2) QuerySnapshot 方法

快照 (现货、指数、港股、期权、期货) 查询接口, 托管机房和互联网模式适用。

函数原型:

```
static int32_t QuerySnapshot(const IGMDSnapshotSpi* snapshot_spi,
    const ReqDefault& req_snapshot);
```

参数:

参数	解释
snapshot_spi(in)	IGMDSnapshotSpi 的派生类实例指针, 必须在调用 Release 函数之后才能销毁该实例 (对应数据回调为 IGMDSnapshotSpi 下的现货、指数、期货、期权等方法)
req_snapshot(in)	快照查询信息数据项首地址

ReqDefault 结构定义如下:

数据类型	数据名称	说明
char	security_code[ConstField::kQuerySecurityCodeLen]	证券代码
uint8_t	market_type	市场类型
uint32_t	date	日期 (必须为 yyyyMMdd)
uint32_t	begin_time	开始时间 (HHmmssSSS)
uint32_t	end_time	结束时间 (HHmmssSSS)
uint16_t	data_type	默认 0 标识快照, 1 标识香港市场委托挂单, 2 标识经纪商席位数据, 其它数字无效 (该字段仅在 QuerySnapshot 接口调用生效)

3) QueryOrderQueue 方法

委托队列查询接口（仅托管机房适用）。

函数原型：

```
static int32_t QueryOrderQueue(const IGMDOrderQueueSpi* order_queue_spi,
    const ReqDefault& req_order_queue);
```

参数：

参数	解释
order_queue_spi(in)	IGMDOrderQueueSpi 的派生类实例指针，必须在调用 Release 函数之后才能销毁该实例（对应数据回调为 IGMDOrderQueueSpi 下的 OnMDOrderQueue 方法）
req_order_queue(in)	委托队列查询数据项首地址

其中，ReqDefault 结构定义参照 QuerySnapshot 方法

4) QueryTickExecution 方法

逐笔成交查询接口（仅托管机房适用）。

函数原型：

```
static int32_t QueryTickExecution(const IGMDTickExecutionSpi* tick_exec_spi,
    const ReqDefault& req_tick_execution);
```

参数：

参数	解释
tick_exec_spi(in)	IGMDTickExecutionSpi 的派生类实例指针，必须在调用 Release 函数之后才能销毁该实例（对应数据回调为 IGMDTickExecutionSpi 下的 OnMDTickExecution 方法）
req_order_queue(in)	委托队列查询数据项首地址

其中，ReqDefault 结构定义参照 QuerySnapshot 方法

5) QueryTickOrder 方法

逐笔委托查询接口（仅托管机房适用）。

函数原型：

```
static int32_t QueryTickOrder(const IGMDTickOrderSpi* tick_order_spi,
    const ReqDefault& req_tick_order);
```

参数：

参数	解释
tick_order_spi(in)	IGMDTickOrderSpi 的派生类实例指针，必须在调用 Release 函数之后才能销毁该实例（对应数据回调为 IGMDTickOrderSpi 下的 OnMDTickOrder 方法）
req_tick_order(in)	逐笔委托查询数据项首地址

其中，ReqDefault 结构定义参照 QuerySnapshot 方法

6) QueryCodeTable 方法

代码表查询接口（托管机房和互联网模式适用）。该方法查询当天交易所发布的最新代码表信息和系统自上线 2022-08-22 以来保存的已退市代码信息。

函数原型：

```
static int32_t QueryCodeTable(const IGMDCodeTableSpi* code_table_spi);
```

参数：

参数	解释
code_table_spi(in)	IGMDCodeTableSpi 的派生类实例指针，必须在调用 Release 函数之后才能销毁该实例（对应数据回调为 IGMDCodeTableSpi 下的 OnMDCodeTable 方法）

7) QuerySecuritiesInfo 方法

证券静态信息查询接口（托管机房和互联网模式适用）。该方法查询当天交易所发布的最新证券静态信息。

函数原型：

```
static int32_t QuerySecuritiesInfo (const IGMDSecuritiesInfoSpi* stock_info_spi,
const SubCodeTableItem* item, uint32_t cnt);
```

参数：

参数	解释
stock_info_spi(in)	IGMDSecuritiesInfoSpi* 的派生类实例指针，必须在调用 Release 函数之后才能销毁该实例（对应数据回调为 IGMDSecuritiesInfoSpi* 下的 OnMDSecuritiesInfo 方法）
item(in)	查询数组首地址
cnt(in)	查询请求个数

SubCodeTableItem 定义如下：

数据类型	数据名称	说明
int32_t	market	市场类型,参考 MarketType ,为 kNone 表示查询所有支持的市场(代码表目前只支持上交所、深交所与北交所)
char	security_code[ConstField::kFutureSecurityCodeLen]	证券代码,为空表示查询所有代码

8) QueryExFactorTable 方法

复权因子表查询接口（托管机房和互联网模式适用）。

函数原型：

```
static int32_t QueryExFactorTable(const IGMDExFactorSpi* ex_factor_spi, const
char* code);
```

参数:

参数	解释
ex_factor_spi(in)	IGMDExFactorSpi 的派生类实例指针, 必须在调用 Release 函数之后才能销毁该实例 (对应数据回调为 IGMDExFactorSpi 下的 OnMDExFactor 方法)
code(in)	输入待查询的证券代码 (例如: 000001)

9) QueryFactor 方法

加工因子查询接口 (托管机房和互联网模式适用)。

函数原型:

```
static int32_t QueryFactor(const IGMDFactorSpi* factor_spi, const ReqFactor& req_factor);
```

参数:

参数	解释
factor_spi(in)	IGMDFactorSpi 的派生类实例指针, 必须在调用 Release 函数之后才能销毁该实例 (对应数据回调为 IGMDFactorSpi 下的 OnFactor 方法)
req_factor(in)	加工因子数据项首地址

ReqFactor 结构中每个字符串的字段名称和类型参考下表:

数据类型	数据名称	说明
int64_t	task_id	任务 id, 对应 GetTaskID 接口产生的唯一 id
char	factor_type[64]	因子父类型(英文)
char	factor_sub_type[64]	因子子类型(英文)
char	factor_name[64]	因子名称(英文)
uint32_t	begin_date	开始日期 (yyyyMMdd)
uint32_t	end_date	结束日期 (yyyyMMdd)
uint32_t	begin_time	开始时间 (HHmmssSSS)
uint32_t	end_time	结束时间 (HHmmssSSS)
char	security_code[32]	证券代码 (填写*表示全代码查询)
uint16_t	market	市场类型
uint16_t	category	品种类别
uint32_t	offset	默认从 0 开始 (暂不支持, 预留分页查询)
uint32_t	count	默认 100 (暂不支持, 预留分页查询)
char	Key1[64]	预留字段 Key1
char	Key2[64]	预留字段 Key2
说明: 1、开始日期和结束日期需要相等, 目前仅支持某一天加工因子查询;		

10) SetThirdInfoParam 方法

设置金融资讯数据设置接口, 托管机房和互联网模式适用。

函数原型:

```
static int32_t SetThirdInfoParam(int64_t task_id, const std::string& key,
```

```
const std::string& value);
```

参数:

参数	解释
task_id(in)	金融资讯数据请求编号
key(in)	金融资讯数据请求 json 的 key
value(in)	金融资讯数据请求 json 的 value

说明: 在进行查询金融资讯数据前, 必须先设置 function_id(key)、功能号(value)、任务 id;

function_id: 此 key 为必须输入的值 (此字段为指定值, 不能为其他值), 在进行查询前必须先设置功能号和任务 id 才可以调用 QueryThirdInfo 方法。

具体功能号的定义以及支持资讯数据详细可以参看附件 1 “[资讯数据说明](#)”。

11) QueryThirdInfo 方法

金融资讯数据查询接口, 托管机房和互联网模式适用。

函数原型:

```
static int32_t QueryThirdInfo(const IGMDThirdInfoSpi* third_info_spi, int64_t task_id);
```

参数:

参数	解释
third_info_spi(in)	IGMDThirdInfoSpi 的派生类实例指针, 必须在调用 Release 函数之后才能销毁该实例 (对应数据回调为 IGMDThirdInfoSpi 下的 OnThirdInfo 方法)
task_id(in)	金融资讯数据请求编号

格物金融服务平台支持查询金融资讯数据, 查询接口 QueryThirdInfo 进行查询金融资讯数据前, 必须先通过 SetThirdInfoParam 方法设置 function_id、任务 id、以及具体某个功能号对应资讯数据请求参数后才可以调用 QueryThirdInfo 方法。

参数 Key	是否必填	参数说明	备注
taskid	是	每次查询资讯数据任务的唯一标识	格物提供方法生成 taskid 确保该 id 唯一性
function_id	是	功能号, 指定要查询金融数据的	比如 A 股基本资料查询功能号: A010010001
market_code	根据本文档说明填写	当前 taskid 查询任务需要输入的参数	本文档各类功能的请求参数

调用查询金融资讯数据方法后, 返回以下数据类型 ThirdInfoData, 查询结果作为 json 字符串保存:

数据类型	字段名称	说明
uint64_t	task_id	任务 id
uint64_t	data_size	数据大小
char*	json_data	数据 json 串

格物金融服务平台可查询的金融资讯数据的详细请求参数和返回数据内容说明请参考附件 1 “[资讯数据说明](#)”。

12) QueryETFInfo 方法

金融资讯数据查询接口，托管机房和互联网模式适用。

函数原型：

```
static int32_t QueryETFInfo(const IGMDETFInfoSpi* etf_info_spi,
                           const SubCodeTableItem* item, uint32_t cnt);
```

参数：

参数	解释
etf_info_spi(in)	IGMDETFInfoSpi 的派生类实例指针，必须在调用 Release 函数之后才能销毁该实例（对应数据回调为 IGMDETFInfoSpi 下的 OnMDETFInfo 方法）
item(in)	查询数组首地址
cnt(in)	查询请求个数

SubCodeTableItem 定义如下：

数据类型	数据名称	说明
int32_t	market	市场类型，参考 MarketType ，为 kNone 表示查询所有支持的市场（代码表目前只支持上交所、深交所与北交所）
char	security_code[ConstField::kFutureSecurityCodeLen]	证券代码，为空表示查询所有代码

3.5.6 查询回调方法

1) IGMDKlineSpi 接口

接收 K 线数据。

OnMDKLine 方法

接收 K 线数据回调接口（互联网和托管机房模式适用）。

函数原型：

```
void OnMDKLine(MDKLine* klines, uint32_t cnt, uint16_t kline_type);
```

参数：

参数	解释
klines(out)	K 线数据首指针，需要显式地调用 FreeMemory
cnt(out)	K 线数据个数

2) IGMDSnapshotSpi 接口

接收快照数据。

OnMDSnapshotL2 方法

L2 现货快照（股票、债券、基金）回调接口（仅托管机房适用）。

函数原型：

```
void OnMDSnapshotL2(MDSnapshotL2* snapshots, uint32_t cnt);
```

参数：

参数	解释
snapshots(out)	快照数据首指针，需要显式地调用 FreeMemory
cnt(out)	快照数据个数

OnMDSnapshotL1 方法

L1 现货快照（股票、债券、基金）回调接口（仅互联网使用）。

函数原型：

```
void OnMDSnapshotL1(MDSnapshotL1* snapshots, uint32_t cnt);
```

参数：

参数	解释
snapshots(out)	快照数据首指针，需要显式地调用 FreeMemory
cnt(out)	快照数据个数

OnMDIndexSnapshot 方法

指数快照回调接口（互联网和托管机房模式适用）。

函数原型：

```
void OnMDIndexSnapshot(MDIndexSnapshot* index_snapshots, uint32_t cnt);
```

参数：

参数	解释
index_snapshots(out)	指数快照数据首指针，需要显式地调用 FreeMemory
cnt(out)	指数快照数据个数

OnMDHKTSnapshot 方法

港股通快照回调接口（互联网和托管机房模式适用）。

函数原型：

```
void OnMDHKTSnapshot(MDHKTSnapshot* hkt_snapshots, uint32_t cnt);
```

参数：

参数	解释
hkt_snapshots(out)	港股通快照数据首指针，需要显式地调用 FreeMemory
cnt(out)	港股通快照数据个数

OnMDOptionSnapshot 方法

期权快照回调接口（互联网和托管机房模式适用）。

函数原型：

```
void OnMDOptionSnapshot(MDOptionSnapshot* opt_snapshots, uint32_t cnt);
```

参数：

参数	解释
opt_snapshots(out)	期权快照数据首指针，需要显式地调用 FreeMemory
cnt(out)	期权快照数据个数

OnMDFutureSnapshot 方法

期货快照回调接口（互联网和托管机房模式适用）。

函数原型：

```
void OnMDFutureSnapshot(MDFutureSnapshot* future_snapshots, uint32_t cnt);
```

参数：

参数	解释
future_snapshots(out)	期货快照数据首指针，需要显式地调用 FreeMemory
cnt(out)	期货快照数据个数

OnMDHKExOrderSnapshot 方法

港股挂单数据回调接口（互联网和托管机房模式适用）。

函数原型：

```
void OnMDHKExOrderSnapshot(MDHKExOrderSnapshot* hke_order_snapshot,
uint32_t cnt);
```

参数：

参数	解释
hke_order_snapshot (out)	港股挂单数据首指针 需要显式地调用 FreeMemory
cnt(out)	期货快照数据个数

OnMDHKExOrderBrokerSnapshot 方法

港股经纪席位数据回调接口（互联网和托管机房模式适用）。

函数原型：

```
void OnMDHKExOrderBrokerSnapshot(MDHKExOrderBrokerSnapshot*
hke_order_broker_snapshot, uint32_t cnt);
```

参数：

参数	解释
hke_order_broker_snapshot (out)	港股经纪席位数据首指针 需要显式地调用 FreeMemory
cnt(out)	期货快照数据个数

3) IGMDOrderQueueSpi 接口

接收委托队列数据（仅托管机房模式适用）

OnMDOrderQueue 方法

委托队列回调接口。

函数原型：

```
void OnMDOrderQueue(MDOrderQueue* order_queues, uint32_t cnt);
```

参数:

参数	解释
order_queues(out)	委托队列数据首指针, 需要显式地调用 FreeMemory
cnt(out)	委托队列数据个数

4) IGMDTickExecutionSpi 接口

接收逐笔成交数据。

OnMDTickExecution 方法

逐笔成交回调接口 (仅托管机房模式适用)。

函数原型:

```
void OnMDTickExecution(MDTickExecution* tick_execs, uint32_t cnt);
```

参数:

参数	解释
tick_execs(out)	逐笔成交数据首指针, 需要显式地调用 FreeMemory
cnt(out)	逐笔成交数据个数

5) IGMDTickOrderSpi 接口

逐笔委托回调接口 (仅托管机房模式适用)。

OnMDTickOrder 方法

逐笔委托回调接口 (仅托管机房模式适用)。

函数原型:

```
void OnMDTickOrder(MDTickOrder* tick_orders, uint32_t cnt);
```

参数:

参数	解释
tick_orders(out)	逐笔委托数据首指针, 需要显式地调用 FreeMemory
cnt(out)	逐笔委托数据个数

6) IGMDCodeTableSpi 接口

接收代码表数据。

OnMDCodeTable 方法

代码表信息回调接口 (仅托管机房模式适用)。

函数原型:

```
void OnMDCodeTable(MDCodeTable* code_tables, uint32_t cnt);
```

参数:

参数	解释
code_tables(out)	代码表数据首指针，需要显式地调用 FreeMemory
cnt(out)	代码表数据个数

7) IGMDSecuritiesInfoSpi 接口

接收证券静态信息数据

OnMDSecuritiesInfo 方法

证券静态信息回调接口（互联网和托管机房模式适用）。

函数原型：

```
void OnMDSecuritiesInfo(MDCodeTableRecord* code_tables, uint32_t cnt);
```

参数：

参数	解释
code_tables(out)	个股基本信息数据首指针，需要显式地调用 FreeMemory
cnt(out)	个股基本信息数据个数

8) IGMDExFactorSpi 接口

复权因子回调接口（互联网和托管机房模式适用）。

OnMDExFactor 方法

复权因子回调接口（互联网和托管机房模式适用）。

函数原型：

```
void OnMDExFactor(MDExFactorTable* ex_factor_tables, uint32_t cnt);
```

参数：

参数	解释
ex_factor_tables(out)	复权因子数据首指针，需要显式地调用 FreeMemory
cnt(out)	复权因子数据个数

9) IGMDFactorSpi 接口

接收加工因子数据。

OnFactor 方法

加工因子数据回调接口（互联网和托管机房模式适用）。

函数原型：

```
void OnFactor(Factor* factors, uint32_t cnt);
```

参数：

参数	解释
----	----

factors(out)	加工因子数据首指针，需要显式地调用 FreeMemory
cnt(out)	加工因子数据个数

10) IGMDThirdInfoSpi 接口

接收金融资讯数据（互联网和托管机房模式适用）。

OnThirdInfo 方法

金融资讯数据回调接口。

函数原型：

```
void OnThirdInfo(ThirdInfoData* data, uint32_t cnt);
```

参数：

参数	解释
data(out)	金融资讯数据首指针，需要显式地调用 FreeMemory
cnt(out)	金融资讯数据个数

11) IGMDETFInfoSpi 接口

接收 ETF 基础信息和成分股信息数据（互联网和托管机房模式适用）。

OnMDETFInfo 方法

ETF 代码表数据回调接口。

函数原型：

```
void OnMDETFInfo(MDETFCodeTableRecord* etf_info, uint32_t cnt);
```

参数：

参数	解释
etf_info (out)	ETF 信息数据首指针，需要显式地调用 FreeMemory 数据定义参看 MDETFCodeTableRecord
cnt(out)	ETF 信息数据个数

3.5.7 回放数据方法

1) ReplayKline 方法

K 线回放接口（支持分钟 K 和日 K 回放）（仅托管机房适用）。

函数原型：

```
static int32_t ReplayKline(const IGMDHistorySpi* pSpi, const  
ReqReplayKline& req_kline)
```

参数：

参数	解释
pSpi(in)	IGMDHistorySpi 的派生类实例指针，必须在调用

	Release 函数之后才能销毁该实例（对应数据回调为 IGMDHistorySpi 下的 OnMDKline 方法）
req_kline(in)	K 线回放请求

ReqReplayKline 结构定义如下：

数据类型	数据名称	说明
uint8_t	cq_flag	除权标志（0:不复权 1:向前复权 2:向后复权，默认值为 0）
uint32_t	cq_date	除权日期（yyyMMdd）（暂不使用）
uint32_t	qj_flag	全价标志（债券）（0: 净价，1: 全价）（暂不使用）
uint16_t	cyc_type	数据周期，参照 tgw_datatype.h 中的 MDDatatype（不支持周月季年 k）
uint32_t	cyc_def	周期数量（暂不使用）
uint8_t	auto_complete	自动补齐（0:不补齐，1: 补齐，默认值为 1）
uint32_t	begin_date	开始日期（yyyyMMdd）
uint32_t	end_date	结束日期（yyyyMMdd）
uint32_t	begin_time	开始时间（HHmm）
uint32_t	end_time	结束时间（HHmm）
uint16_t	replay_speed	返回倍速（暂不可用）
int64_t	task_id	任务 id(任务编号，例如:1)调用 GetTaskId 获取
ReqHistoryItem*	req_items	请求数组 item 头指针,不得为空
uint32_t	req_item_cnt	请求回放代码数量

ReqHistoryItem 结构定义如下：

数据类型	数据名称	说明
uint8_t	market	市场类型，参考 MarketType
char	security_code[ConstField::kHistorySecurityCodeLen]	证券代码，代码不能为空

2) ReplayRequest 方法

快照、逐笔成交回放接口（仅托管机房适用）。

函数原型：

static int32_t ReplayRequest(const IGMDHistorySpi* pSpi, const ReqReplay& req_replay)。

参数：

参数	解释
pSpi(in)	IGMDHistorySpi 的派生类实例指针，必须在调用 Release 函数之后才能销毁该实例（对应数据回调为 IGMDHistorySpi 下的快照和逐笔成交方法）
req_replay(in)	行情回放请求

ReqReplay 结构定义如下:

数据类型	数据名称	说明
uint16_t	md_data_type	回放数据类型,参照 tgw_datatype.h 中的 MDDatatype
uint32_t	begin_date	开始日期 (yyyyMMdd)
uint32_t	end_date	结束日期 (yyyyMMdd)
uint32_t	begin_time	开始时间 (HHmmssSSS)
uint32_t	end_time	结束时间 (HHmmssSSS)
uint16_t	replay_speed	返回倍速 (暂不可用)
int64_t	task_id	任务 id(任务编号,例如:1)调用 GetTaskId 获取
ReqHistoryItem*	req_items	请求数组 item 头指针,不得为空
uint32_t	req_item_cnt	请求回放代码数量

3) CancelTask 方法

取消指定的回放任务 (仅托管机房适用)。

函数原型:

```
static int32_t CancelTask(const int64_t& task_id)。
```

参数:

参数	解释
task_id(in)	需取消的回放任务 id

说明: 满足快速取消一个回放任务,重新定义另一个回放任务或想快速安全退出程序,重启程序快速开始新的回放任务;默认 0 表示取消所有的任务。

3.5.8 回放回调方法

1) OnMDKline 方法

接收 K 线回放数据回调 (仅托管机房适用)。

函数原型:

```
void OnMDKline(int64_t task_id, MDKLine\* klines, uint32_t cnt, uint16_t kline_type);
```

参数:

参数	解释
task_id(out)	回放任务 id
kline(out)	K 线数据首指针,需要显式地调用 FreeMemory
cnt(out)	K 线个数
kline_type(out)	取值参考公共字典 MDDatatype

2) OnMDSnapshot 方法

接收 K 线回放数据回调（仅托管机房适用）。

函数原型：

```
void OnMDSnapshot(int64_t task_id, MDSnapshotL2* snapshots, uint32_t cnt);
```

参数：

参数	解释
task_id(out)	回放任务 id
snapshots(out)	快照数据首指针，需要显式地调用 FreeMemory
cnt(out)	快照数据个数

3) OnMDTickExecution 方法

接收逐笔成交回放数据回调（仅托管机房适用）。

函数原型：

```
void OnMDTickExecution(int64_t task_id, MDTickExecution* ticks, uint32_t cnt);
```

参数：

参数	解释
task_id(out)	回放任务 id
ticks(out)	逐笔成交数据首指针，需要显式地调用 FreeMemory
cnt(out)	逐笔成交数据个数

4) OnRspTaskStatus 方法

接收回放任务状态（仅托管机房使用）。

函数原型：

```
void OnRspTaskStatus(int64_t task_id, RspTaskStatus* task_status);
```

参数：

参数	解释
task_id(out)	回放任务 id
task_status(out)	回放任务状态
task_id(out)	回放任务 id

RspTaskStatus 结构定义如下：

数据类型	数据名称	说明
int64_t	task_id	任务 id(任务编号，例如:1)
int16_t	status	状态值(详细参考 HistoryTaskStatus)
int16_t	process_rate	进度（暂不支持）
int16_t	error_code	状态为失败的错误码(详细参考 ErrorCode)
int16_t	error_msg_len	错误消息长度
char*	error_msg	错误消息首地址

3.6 开发 Demo 示例

SDK 安装包中提供 `tgw_demo` 开发示例可供参考：

```
#include <string.h>
#include <unistd.h>
#include "tgw_push_spi.h"
#include "tgw_query_spi.h"
#include "tgw_replay_spi.h"
#include <tgw_datatype.h>

// ----- 推送请求示例 -----
void GeneralSubscribeReq()
{
    // 设置订阅参数，以两次订阅为例
    galaxy::tgw::SubscribeItem sub_cfg[2];

    // 第一次订阅的市场类型设置
    sub_cfg[0].market = galaxy::tgw::MarketType::kSZSE;
    // 第一次订阅的数据类型设置
    sub_cfg[0].flag = galaxy::tgw::SubscribeDataType::kNone;
    // 第一次订阅的证券代码设置
    strncpy(sub_cfg[0].security_code, "000001", sizeof(sub_cfg[0].security_code));
    // 第一次订阅的品种类别设置
    sub_cfg[0].category_type = galaxy::tgw::VarietyCategory::kNone;

    // 第二次订阅的市场类型设置
    sub_cfg[1].market = galaxy::tgw::MarketType::kNone;
    // 第二次订阅的数据类型设置
    sub_cfg[1].flag = galaxy::tgw::SubscribeDataType::kNone;
    // 第二次订阅的证券代码设置，为空代表全部证券代码
    strncpy(sub_cfg[1].security_code, "", sizeof(sub_cfg[1].security_code));
    // 第二次订阅的品种类别设置
    sub_cfg[1].category_type = galaxy::tgw::VarietyCategory::kStock;

    // 发送多个订阅请求
    auto ec = galaxy::tgw::IGMDApi::Subscribe(sub_cfg, 2);
    if (ec != galaxy::tgw::ErrorCode::kSuccess)
        std::cout << "GeneralSubscribe failed, ErrorCode: " << ec << std::endl;

    // 取消订阅
    galaxy::tgw::IGMDApi::UnSubscribe(sub_cfg, 2);
}

void FactorSubscribeReq()
{
    /**
     * @note 父类型不支持单独全订阅及取消订阅，即 all xxx xxx
     *       因子子类型不支持单独全订阅及取消订阅，即 xxx all xxx
     *       不支持父类型和因子子类型全订阅及取消订阅，但是因子名称为指定因子，即 all all xxx
     *       不支持因子父类型和因子名称全订阅及取消订阅，但是因子子类型为指定因子，即 all xxx all
     */
    galaxy::tgw::SubFactorItem item[4];
    // 订阅全因子类型设置 all all all
    strncpy(item[0].factor_type, "all", sizeof(item[0].factor_type));
    strncpy(item[0].factor_sub_type, "all", sizeof(item[0].factor_sub_type));
    strncpy(item[0].factor_name, "all", sizeof(item[0].factor_name));

    galaxy::tgw::IGMDApi::SubFactor(&item[0], 1);

    // 订阅指定父类型下全因子类型设置 xxx all all
    strncpy(item[1].factor_type, "Financial_Factor", sizeof(item[1].factor_type));
    strncpy(item[1].factor_sub_type, "all", sizeof(item[1].factor_sub_type));
    strncpy(item[1].factor_name, "all", sizeof(item[1].factor_name));

    galaxy::tgw::IGMDApi::SubFactor(&item[1], 1);

    // 订阅指定父类型+子类型下全因子类型设置 xxx xxx all
    strncpy(item[2].factor_type, "Financial_Factor", sizeof(item[2].factor_type));
    strncpy(item[2].factor_sub_type, "PE", sizeof(item[2].factor_sub_type));
}
```

```

    strncpy(item[2].factor_name, "all", sizeof(item[2].factor_name));

    galaxy::tgw::IGMDApi::SubFactor(&item[2], 1);

    // 订阅指定父类型+子类型+因子名称类型设置 xxx xxx xxx
    strncpy(item[3].factor_type, "Financial_Factor", sizeof(item[3].factor_type));
    strncpy(item[3].factor_sub_type, "PE", sizeof(item[3].factor_sub_type));
    strncpy(item[3].factor_name, "PE_TTL", sizeof(item[3].factor_name));

    auto ec = galaxy::tgw::IGMDApi::SubFactor(&item[3], 1);
    if (ec != galaxy::tgw::ErrorCode::kSuccess)
        std::cout << "SubFactor failed, ErrorCode: " << ec << std::endl;

    // 取消因子订阅
    // galaxy::tgw::IGMDApi::UnSubFactor(&item[3], 1); 入参同上
}

void DerivedDataSubscribeReq()
{
    int32_t subscribe_type = galaxy::tgw::SubscribeType::kAdd; // 增加订阅
    uint32_t derived_data_type = galaxy::tgw::SubscribeDerivedDataType::kOrderBookSnapshot; // 订阅委托簿快照

    galaxy::tgw::SubscribeDerivedDataItem item[2];
    item[0].market = galaxy::tgw::MarketType::kSZSE;
    strncpy(item[0].security_code, "000001", sizeof(item[0].security_code));
    item[1].market = galaxy::tgw::MarketType::kSSE;
    strncpy(item[1].security_code, "600001", sizeof(item[1].security_code));

    auto ec = galaxy::tgw::IGMDApi::SubscribeDerivedData(subscribe_type, derived_data_type, item, 2);
    if (ec != galaxy::tgw::ErrorCode::kSuccess)
        std::cout << "SubscribeDerivedData failed, ErrorCode: " << ec << std::endl;
}

// ----- 查询请求示例 -----
void QueryKlineReq(galaxy::tgw::QueryKlineSpi* spi)
{
    galaxy::tgw::ReqKline req_kline;
    memset(&req_kline, 0, sizeof(req_kline));
    strncpy(req_kline.security_code, "000001", sizeof(req_kline.security_code));
    req_kline.market_type = galaxy::tgw::MarketType::kSZSE;
    req_kline.cq_flag = 0;
    req_kline.auto_complete = 1;
    req_kline.cyc_type = galaxy::tgw::MDDatatype::k1KLine; // 1 分钟 k
    req_kline.begin_date = 20210611;
    req_kline.end_date = 20210611;
    req_kline.begin_time = 930;
    req_kline.end_time = 1700;

    int32_t ec;
    if ((ec = galaxy::tgw::IGMDApi::QueryKline(spi, req_kline)) != galaxy::tgw::ErrorCode::kSuccess)
        std::cout << "QueryKline failed, ErrorCode: " << ec << std::endl;
}

void QuerySnapshotReq(galaxy::tgw::QuerySnapshotSpi* spi)
{
    galaxy::tgw::ReqDefault req_snapshot;
    memset(&req_snapshot, 0, sizeof(req_snapshot));
    strncpy(req_snapshot.security_code, "000001", sizeof(req_snapshot.security_code));
    req_snapshot.market_type = galaxy::tgw::MarketType::kSZSE;
    req_snapshot.date = 20210611;
    req_snapshot.begin_time = 930000000;
    req_snapshot.end_time = 1700000000;

    int32_t ec;
    if ((ec = galaxy::tgw::IGMDApi::QuerySnapshot(spi, req_snapshot)) != galaxy::tgw::ErrorCode::kSuccess)
        std::cout << "QuerySnapshot failed, ErrorCode: " << ec << std::endl;
}

void QueryTickExecReq(galaxy::tgw::QueryTickExecutionSpi* spi)
{
    galaxy::tgw::ReqDefault req;
    memset(&req, 0, sizeof(req));
    strncpy(req.security_code, "000001", sizeof(req.security_code));
    req.market_type = galaxy::tgw::MarketType::kSZSE;
    req.date = 20210611;
    req.begin_time = 930000000;
    req.end_time = 1700000000;

    int32_t ec;

```

```

    if ((ec = galaxy::tgw::IGMDApi::QueryTickExecution(spi, req)) != galaxy::tgw::ErrorCode::kSuccess)
        std::cout << "QueryTickExec failed, ErrorCode: " << ec << std::endl;
}

void QueryTickOrderReq(galaxy::tgw::QueryTickOrderSpi* spi)
{
    galaxy::tgw::ReqDefault req;
    memset(&req, 0, sizeof(req));
    strncpy(req.security_code, "000001", sizeof(req.security_code));
    req.market_type = galaxy::tgw::MarketType::kSZSE;
    req.date = 20210611;
    req.begin_time = 930000000;
    req.end_time = 1700000000;

    int32_t ec;
    if ((ec = galaxy::tgw::IGMDApi::QueryTickOrder(spi, req)) != galaxy::tgw::ErrorCode::kSuccess)
        std::cout << "QueryTickOrder failed, ErrorCode: " << ec << std::endl;
}

void QueryOrderQueueReq(galaxy::tgw::QueryOrderQueueSpi* spi)
{
    galaxy::tgw::ReqDefault req;
    memset(&req, 0, sizeof(req));
    strncpy(req.security_code, "000001", sizeof(req.security_code));
    req.market_type = galaxy::tgw::MarketType::kSZSE;
    req.date = 20210611;
    req.begin_time = 930000000;
    req.end_time = 1700000000;

    int32_t ec;
    if ((ec = galaxy::tgw::IGMDApi::QueryOrderQueue(spi, req)) != galaxy::tgw::ErrorCode::kSuccess)
        std::cout << "QueryOrderQueue failed, ErrorCode: " << ec << std::endl;
}

void QueryCodeTableReq(galaxy::tgw::QueryCodeTableSpi* spi)
{
    int32_t ec;
    if ((ec = galaxy::tgw::IGMDApi::QueryCodeTable(spi)) != galaxy::tgw::ErrorCode::kSuccess)
        std::cout << "QueryCodeTable failed, ErrorCode: " << ec << std::endl;
}

void QuerySecuritiesInfoReq(galaxy::tgw::QuerySecuritiesInfoSpi* spi)
{
    galaxy::tgw::SubCodeTableItem req_codelist;
    memset(&req_codelist, 0, sizeof(req_codelist));
    strncpy(req_codelist.security_code, "000001", sizeof(req_codelist.security_code));
    req_codelist.market = galaxy::tgw::MarketType::kSZSE;

    int32_t ec;
    if ((ec = galaxy::tgw::IGMDApi::QuerySecuritiesInfo(spi, &req_codelist, 1)) != galaxy::tgw::ErrorCode::kSuccess)
        std::cout << "QuerySecuritiesInfo failed, ErrorCode: " << ec << std::endl;
}

void QueryETFInfoReq(galaxy::tgw::QueryETFInfoSpi* spi)
{
    galaxy::tgw::SubCodeTableItem req_etf;
    memset(&req_etf, 0, sizeof(req_etf));
    strncpy(req_etf.security_code, "000001", sizeof(req_etf.security_code));
    req_etf.market = galaxy::tgw::MarketType::kSZSE;
    int32_t ec;
    if ((ec = galaxy::tgw::IGMDApi::QueryETFInfo(spi, &req_etf, 1)) != galaxy::tgw::ErrorCode::kSuccess)
        std::cout << "QueryETFInfo failed, ErrorCode: " << ec << std::endl;
}

void QueryExFactorReq(galaxy::tgw::QueryExFactorSpi* spi)
{
    int32_t ec;
    if ((ec = galaxy::tgw::IGMDApi::QueryExFactorTable(spi, "000001")) != galaxy::tgw::ErrorCode::kSuccess)
        std::cout << "QueryExFactorTable failed, ErrorCode: " << ec << std::endl;
}

void QueryFactorReq(galaxy::tgw::QueryFactorSpi* spi)
{
    galaxy::tgw::ReqFactor req_factor;
    memset(&req_factor, 0, sizeof(req_factor));
    req_factor.task_id = galaxy::tgw::IGMDApi::GetTaskID();
    strncpy(req_factor.factor_type, "factor_type", sizeof(req_factor.factor_type));
    strncpy(req_factor.factor_sub_type, "factor_sub_type", sizeof(req_factor.factor_sub_type));
    strncpy(req_factor.factor_name, "factor_name", sizeof(req_factor.factor_name));
    req_factor.begin_date = 20210611;
}

```

```

req_factor.end_date = 20210611;
req_factor.begin_time = 93000000;
req_factor.end_time = 170000000;
strcpy(req_factor.security_code, "000001", sizeof(req_factor.security_code));
req_factor.market = galaxy::tgw::MarketType::kSZSE;
req_factor.category = 2;
req_factor.offset = 0;
req_factor.count = 100;
int32_t ec;
if ((ec = galaxy::tgw::IGMDApi::QueryFactor(spi, req_factor)) != galaxy::tgw::ErrorCode::kSuccess)
    std::cout << "QueryFactor failed, ErrorCode: " << ec << std::endl;
}

void QueryThirdInfoReq(galaxy::tgw::QueryThirdInfoSpi* spi)
{
    auto task_id = galaxy::tgw::IGMDApi::GetTaskID();
    galaxy::tgw::IGMDApi::SetThirdInfoParam(task_id, "function_id", "01");
    galaxy::tgw::IGMDApi::SetThirdInfoParam(task_id, "WD_CODE", "0000001");
    galaxy::tgw::IGMDApi::SetThirdInfoParam(task_id, "trade_data", "20210101");

    int32_t ec;
    if ((ec = galaxy::tgw::IGMDApi::QueryThirdInfo(spi, task_id)) != galaxy::tgw::ErrorCode::kSuccess)
        std::cout << "QueryThirdInfo failed, ErrorCode: " << ec << std::endl;
}

void ReplaySnapshotReq(galaxy::tgw::ReplayHistorySpi* spi)
{
    sleep(3); // 延迟 3s 至登录成功
    galaxy::tgw::ReqHistoryItem history_item[2];
    history_item[0].market = galaxy::tgw::MarketType::kSZSE;
    strcpy(history_item[0].security_code, "000001");
    history_item[1].market = galaxy::tgw::MarketType::kSZSE;
    strcpy(history_item[1].security_code, "000002");

    galaxy::tgw::ReqReplay req_replay;
    req_replay.begin_date = 20211228;
    req_replay.end_date = 20211228;
    req_replay.begin_time = 91500000;
    req_replay.end_time = 103100000;
    req_replay.req_item_cnt = 2;
    req_replay.req_items = history_item;
    req_replay.md_data_type = galaxy::tgw::MDDatatype::kSnapshot;

    int32_t ec;
    if ((ec = galaxy::tgw::IGMDApi::ReplayRequest(spi, req_replay)) != galaxy::tgw::ErrorCode::kSuccess)
        std::cout << "ReplaySnapshot failed, ErrorCode: " << ec << std::endl;
}

void ReplayTickExecReq(galaxy::tgw::ReplayHistorySpi* spi)
{
    sleep(3); // 延迟 3s 至登录成功
    galaxy::tgw::ReqHistoryItem history_item[2];
    history_item[0].market = galaxy::tgw::MarketType::kSZSE;
    strcpy(history_item[0].security_code, "000001");
    history_item[1].market = galaxy::tgw::MarketType::kSZSE;
    strcpy(history_item[1].security_code, "000002");

    galaxy::tgw::ReqReplay req_replay;
    req_replay.begin_date = 20211228;
    req_replay.end_date = 20211228;
    req_replay.begin_time = 91500000;
    req_replay.end_time = 103100000;
    req_replay.req_item_cnt = 2;
    req_replay.req_items = history_item;

    req_replay.md_data_type = galaxy::tgw::MDDatatype::kTickExecution;

    int32_t ec;
    if ((ec = galaxy::tgw::IGMDApi::ReplayRequest(spi, req_replay)) != galaxy::tgw::ErrorCode::kSuccess)
        std::cout << "ReplayTickExec failed, ErrorCode: " << ec << std::endl;
}

void ReplayKlineReq(galaxy::tgw::ReplayHistorySpi* spi)
{
    sleep(3); // 延迟 3s 至登录成功
    galaxy::tgw::ReqHistoryItem history_item[2];
    history_item[0].market = galaxy::tgw::MarketType::kSZSE;
    strcpy(history_item[0].security_code, "000001");
    history_item[1].market = galaxy::tgw::MarketType::kSZSE;
    strcpy(history_item[1].security_code, "000002");

```

```

galaxy::tgw::ReqReplayKline req_k;
memset(&req_k, 0, sizeof(req_k));
req_k.begin_date = 20211228;
req_k.begin_time = 930;
req_k.cq_flag = 0;
req_k.cyc_type = galaxy::tgw::MDDatatype::k1KLine; // 1 分钟 k
req_k.auto_complete = 1;
req_k.end_date = 20211228;
req_k.end_time = 1032;
req_k.task_id = galaxy::tgw::IGMDApi::GetTaskID();
req_k.req_item_cnt = 2;
req_k.req_items = history_item;

int32_t ec;
if ((ec = galaxy::tgw::IGMDApi::ReplayKline(spi, req_k)) != galaxy::tgw::ErrorCode::kSuccess)
    std::cout << "ReplayKline failed, ErrorCode: " << ec << std::endl;
}

int main()
{
    // 准备配置
    galaxy::tgw::Cfg cfg;
    strcpy(cfg.server_vip, "127.0.0.1", sizeof(cfg.server_vip));
    cfg.server_port = 9000;
    strcpy(cfg.username, "*****", sizeof(cfg.username));
    strcpy(cfg.password, "*****", sizeof(cfg.password));

    uint16_t api_mode = galaxy::tgw::ApiMode::kColocationMode; // 设置 api 模式 托管机房模式/互联网模式
    if (api_mode == galaxy::tgw::ApiMode::kColocationMode)
    {
        cfg.coloca_cfg.channel_mode = galaxy::tgw::ColocatChannelMode::kQTCPP;
        cfg.coloca_cfg.qtcp_channel_thread = 8;
        cfg.coloca_cfg.qtcp_max_req_cnt = 1000;
        cfg.coloca_cfg.qtcp_req_time_out = 10;
        cfg.coloca_cfg.enable_order_book = galaxy::tgw::OrderBookType::kServerOrderBook;
        cfg.coloca_cfg.entry_size = 10;
        cfg.coloca_cfg.order_queue_size = 10;
        cfg.coloca_cfg.order_book_deliver_interval_microsecond = 10000;
    }

    galaxy::tgw::TgwPushSpi spi;
    bool is_running = true;
    // 初始化 TGW
    int32_t ec = galaxy::tgw::ErrorCode::kSuccess;
    // if ((ec = galaxy::tgw::IGMDApi::Init(&spi, cfg, api_mode, "/")) != galaxy::tgw::ErrorCode::kSuccess) // 第四个参数
    // 可不填写, 直接使用默认参数
    if ((ec = galaxy::tgw::IGMDApi::Init(&spi, cfg, api_mode))
        != galaxy::tgw::ErrorCode::kSuccess)
    {
        std::cout << "Init TGW failed, ErrorCode: " << ec << std::endl;
        galaxy::tgw::IGMDApi::Release();
        return -1;
    }

    // 修改密码
    // galaxy::tgw::UpdatePassWordReq req;
    // memset(&req, 0, sizeof(req));
    // strcpy(req.username, "*****", sizeof(req.username));
    // strcpy(req.old_password, "*****", sizeof(req.old_password));
    // strcpy(req.new_password, "*****", sizeof(req.new_password));
    // ec = galaxy::tgw::IGMDApi::UpdatePassWord(req);
    // if (ec != galaxy::tgw::ErrorCode::kSuccess)
    //     std::cout << "UpdatePassWord failed, ErrorCode: " << ec << std::endl;
    // else
    //     std::cout << "UpdatePassWord success" << std::endl;
    /*=====推送示例=====*/
    // // 1、常规订阅示例
    // GeneralSubscribeReq();
    // // 2、加工因子订阅示例
    // FactorSubscribeReq();
    // // 3、衍生数据订阅示例
    // DerivedDataSubscribeReq();

    /*=====查询示例=====*/
    // // 1、查询 K 线
    // galaxy::tgw::QueryKlineSpi* kline_spi = new galaxy::tgw::QueryKlineSpi();
    // QueryKlineReq(kline_spi);

```

```

// 2、查询快照
// galaxy::tgw::QuerySnapshotSpi* snapshot_spi = new galaxy::tgw::QuerySnapshotSpi();
// QuerySnapshotReq(snapshot_spi);

// 3、查询逐笔成交
// galaxy::tgw::QueryTickExecutionSpi* tick_exec_spi = new galaxy::tgw::QueryTickExecutionSpi();
// QueryTickExecReq(tick_exec_spi);

// 4、查询逐笔委托
// galaxy::tgw::QueryTickOrderSpi* tick_order_spi = new galaxy::tgw::QueryTickOrderSpi();
// QueryTickOrderReq(tick_order_spi);

// 5、查询委托队列
// galaxy::tgw::QueryOrderQueueSpi* order_queue_spi = new galaxy::tgw::QueryOrderQueueSpi();
// QueryOrderQueueReq(order_queue_spi);

// 6、查询代码表
// galaxy::tgw::QueryCodeTableSpi* code_table_spi = new galaxy::tgw::QueryCodeTableSpi();
// QueryCodeTableReq(code_table_spi);

// 7、查询证券代码基本信息
// galaxy::tgw::QuerySecuritiesInfoSpi* secur_info_spi = new galaxy::tgw::QuerySecuritiesInfoSpi();
// QuerySecuritiesInfoReq(secur_info_spi);

// 8、查询复权因子
// galaxy::tgw::QueryExFactorSpi* ex_fator_spi = new galaxy::tgw::QueryExFactorSpi();
// QueryExFactorReq(ex_fator_spi);

// 9、查询加工因子
// galaxy::tgw::QueryFactorSpi* factor_spi = new galaxy::tgw::QueryFactorSpi();
// QueryFactorReq(factor_spi);

// 10、查询三方资讯
// galaxy::tgw::QueryThirdInfoSpi* thirdinfo_spi = new galaxy::tgw::QueryThirdInfoSpi();
// QueryThirdInfoReq(thirdinfo_spi);

// 11、查询 ETF 代码信息
// galaxy::tgw::QueryETFInfoSpi* etf_info_spi = new galaxy::tgw::QueryETFInfoSpi();
// QueryETFInfoReq(etf_info_spi);

/*=====回放示例=====*/
// 1、回放快照
// galaxy::tgw::ReplayHistorySpi* hSpi = new galaxy::tgw::ReplayHistorySpi();
// ReplaySnapshotReq(hSpi);

// 2、回放逐笔成交
// ReplayTickExecReq(hSpi);

// 3、回放 k 线
// ReplayKlineReq(hSpi);

// 这里必须保证用户的 spi 实例的生命周期长于 TGW 的 API
while (is_running)
{
    sleep(3);
    std::cout << "TGW is running ..." << std::endl;
}

// 释放申请的对象（需保证在数据全部查询/回放完毕后，才可释放）
{
    // delete kline_spi;
    // delete snapshot_spi;
    // delete tick_exec_spi;
    // delete tick_order_spi;
    // delete order_queue_spi;
    // delete code_table_spi;
    // delete internet_stock_info_spi;
    // delete coloc_stock_info_spi;
    // delete ex_fator_spi;
    // delete factor_spi;
    // delete thirdinfo_spi;
    // delete etf_info_spi;
    // delete hSpi;
}

// 释放资源
galaxy::tgw::IGMDApi::Release();
return 0;
}

```

3.7 编译运行步骤

- Linux 系统下 TGW 使用前需要运行根目录下 `tgw_install.sh` 脚本设置环境变量
- Linux 系统下 TGW 已经提供了 `Makefile` 编译文件, 用户可选择如下方案编译运行

方案 1: (linux)

- a) 进入 `c++/demo` 所在目录, 直接运行 `make` 命令。
- b) 再次运行 `make install` 命令, 将 `tgw_demo` 可执行程序安装到 `bin` 目录下。
- c) 再次执行 `make clean` 命令, 可直接清除编译过程中产生的 `.o` 文件和其他不需要的文件。
- d) 接下来可以运行, 直接执行如下命令:

```
LD_LIBRARY_PATH=$LD_LIBRARY_PATH:./lib ./bin/tgw_demo
```

方案 2: (linux)

- a) 已经提供了 `make.sh` 脚本, 其中包含了方案 1 的所有操作步骤。
 - b) 直接运行 `./make.sh` 命令即可完成编译运行所有步骤。
- Windows 系统下 TGW 使用前需要运行根目录下 `tgw_install.bat` 脚本设置环境变量
 - Windows 系统下 TGW 运行报错链接不到系统 `dll` 时, 请尝试安装包中的 `VC_redist` 运行时库。
 - Windows 系统下 TGW 已经提供了 `VC++2017` 工程文件, 用户可在 `visual studio` 开发环境中打开对应解决方案文件并编译运行。

编译步骤: (windows)

- a) 进入到 `c++` 目录: `cd AMD_tgw_msvc-14.0_Windows_*/c++/`。
- b) 进入 `project` 目录(visual studio 工程)。
- c) 打开工程即可, `demo` 程序源文件需要根据实际情况填写自行修改。
- d) 修改完成后即可按键盘 `F5` 直接编译运行。

3.8 开发注意事项

- 1) 初始化成功则可执行订阅或者取消订阅操作或进行查询, 初始化失败则调用 `Release` 函数释放资源后退出。
- 2) 订阅时调用接口 `Subscribe` 或 `SubFactor`, 若账号不限制订阅数则可以填写成订阅所有市场、类型、证券代码。若有限制则只能单市场, 单证券代码, 数据类型不受限制), 需要连续订阅多只, 则可参照 `demo` 示例, 使用数组入参的方式。
- 3) 订阅成功后, 对应数据将回调到提供的接口, 如 `OnMDSnapshot()` 方法将接收到现货快照数据, 具体数据类型对应不同接口, 用户可选择实现或者不处理, 在完成对应处理后, 必须调用 `FreeMemory()` 方法释放传入的指针内存, 否则将造成内存泄漏。
- 4) 取消订阅的流程、规则, 与订阅流程、规则类似, 取消后将不再接收该类型的数据, 取消即时生效但数据为异步回调, 取消前已收到的数据仍然会接收。
- 5) 查询接口使用时请先判断返回错误码, 若不成功则出参无数据, 若成功则根据 `QueryData` 中数据类型进行指定处理 (即将指针转成对应的结构体进行解析处理,

查询和推送结构相同), 使用后需要对 QueryData 中的 msg 指针调用 FreeMemory() 方法释放内存, 以避免内存泄漏

- 6) 调用查询接口时有最大并发量限制, 超过最大并发量则等待部分任务完成, 未超过则发起查询请求等待结果返回。
- 7) 登录和初始化不冲突, 初始化成功并不一定登陆成功, 登录失败将有对应错误日志提示, 用户可参照 tgw_datatype.h 文件中的错误码, 排查对应原因。
- 8) 编译运行当前 demo 没有问题, 但是如果用户移动了默认的目录层级或者自行引入了开发所需的其他三方库, 可能造成编译运行的问题, 用户需自行解决。
- 9) 如果运行报证书认证失败, 请运行根目录下设置环境变量脚本 tgw_install。

3.9 演示示例（获取和保存数据）

SDK 安装包分为 Windows 和 Linux 两种版本，请分别下载。SDK 提供 `tgw_test` 标准演示程序，该程序支持动态订阅和取消订阅操作：在终端与 `tgw_test` 监听地址连接后，终端按照指定格式完成输入订阅或退订命令，从而获取实时行情数据，并将数据通过所配置的路径，落地成 csv 文件供查看，以下是程序的命令参数说明和示例。

`tgw_test` 程序用于指引用户了解 API 工作流程和使用步骤，也供开发人员以及测试人员进行功能验证和测试分析。

3.9.1 修改参数

etc/tgw.json 关键参数更改

```
"ApiMode":1,
"CsvFileDir": "./",
"CommandMode": 1,
"DynamicSubscribeIp": "127.0.0.1",
"DynamicSubscribePort":8017,
"ReqFile": "./req_file.txt",
"ServerVip": "127.0.0.1",
"ServerPort": 9200,
"UserName": "amd",
"Password": "amd@2022",
"ColocChannelMode": ["TCP"],
"ColocQTcpReqTimeOut": 1,
"ColocQTcpChannelThread": 2,
"ColocQTcpMaxReqCnt": 100,
"EnableOrderBook":0,
"EntrySize":10,
"OrderQueueSize": 0,
"OrderBookOutputInternal": 10000,
"MinLogLevel": 2
```

- `ApiMode` 功能模式，1 托管机房模式，2 互联网模式
- `CsvFileDir` 为落地 csv 文件路径。
- `CommandMode` 为命令输入方式，1 动态输入命令，2 文件读取命令
- `DynamicSubscribeIp` 动态订阅时的 ip 地址，仅可为本机 ip
- `DynamicSubscribePort` 动态订阅的端口，默认 8019，可改动
- `ReqFile` 为查询命令文件路径及文件，格式遵循 nc 动态查询的格式
- `ServerVip` 连接的服务地址
- `ServerPort` 连接的服务端口，默认 8600。
- `UserName` 用户名
- `Password` 密码
- `ColocChannelMode` 托管机房模式下的通道模式

- ColocQTcpReqTimeOut 托管机房模式查询通道请求超时时间
- ColocQTcpChannelThread 托管机房模式下通道线程数
- ColocQTcpMaxReqCnt 托管机房模式查询通道查询任务最大请求数
- EnableOrderBook 是否启用委托簿功能：默认为 0 不启用委托簿功能，0：不启用委托簿，2：服务端主动推送委托簿数据
- EntrySize 委托簿价格档位设置，默认为 20 档。最小值为 1，最大值为 20
- OrderQueueSize 每个价位委托揭示委托笔数，默认 50 笔。最小值为 1，最大值为 50
- OrderBookOutputInterval 委托簿递交时间间隔(微秒级)，默认 50ms，设置大于 50ms 按照指定值递送数据，小于 50ms 按照默认值 50ms 递送
- MinLogLevel 输出日志等级，默认 info=2

3.9.2 运行启动脚本

- Linux 版本 SDK 运行前需要先运行根目录下 `tgw_install.sh` 脚本设置环境变量；之后对应启动位于 SDK 根目录 `c++/test_tool` 下的 `run.sh` 脚本。终端中输入 `./run.sh` 启动运行。
- Windows 版本 SDK 运行前需要先运行根目录下 `tgw_install.bat` 脚本设置环境变量；之后对应启动位于 SDK 根目录 `c++/test_tool` 下的 `run.bat` 脚本。cmd 命令框中输入 `run.bat` 启动运行。

3.9.3 开启订阅命令输入终端

Linux 版本下，另开一个终端，通过 `nc` 连接，然后，输入对应的测试示例，逐条输入，回车进入下一条命令即可完成对应操作。

```
(base) [root@localhost bin]# nc 127.0.0.1 8039
Sub All All All
```

Windows 版本下，另开一个 cmd 命令输入框，输入 `ipconfig`，找到 IPv4 地址。



```
PS C:\Users\s0659> ipconfig

Windows IP 配置

以太网适配器 本地连接:

    连接特定的 DNS 后缀 . . . . . : 
    本地链接 IPv6 地址. . . . . : fe80::48fc:17fb:1af8:9d58%11
    IPv4 地址 . . . . . : 10.240.0.54
    子网掩码 . . . . . : 255.255.255.0
    默认网关. . . . . : 10.240.0.254

隧道适配器 isatap.{F5D2885F-1082-4B42-93D2-1960FA00108B}:

    媒体状态 . . . . . : 媒体已断开
    连接特定的 DNS 后缀 . . . . . : 
PS C:\Users\s0659>
```

通过 `telnet` 连接，回车即可进行连接。然后，输入对应的测试示例，逐条输入，回车进入下一条命令即可完成对应操作（此时命令输入不可见，最好由其他地方提前准备，之后

右键复制进去),



如果提示 telnet 无法识别,



则进入计算机找到该选项进入,

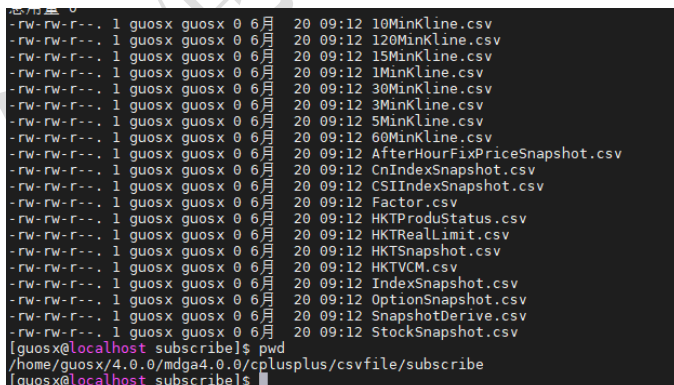


选择下方两个选项, 点击确定即可启用。



3.9.4 查看订阅数据 csv 文件

在设置的 csv 路径下自动生成“subscribe”、“query/查询日期”两个子目录用于存放订阅和查询接收到的行情数据文件, tgw_test 按照数据类型生成单独的 csv 文件。



3.9.5 tgw_test 程序订阅命令说明

1) 查询命令示例:

- QuerySnapshot \${市场} \${证券代码} \${日期} \${开始时间} \${结束时间} (查询快照数据)
 示例: QuerySnapshot SSE 000002 20200924 90000000 93500000
 说明:
 20200924: 表示 2020 年 9 月 24 日
 93500000: 表示 9 点 35 分 00 秒 000 毫秒
- QueryTickOrder \${市场} \${证券代码} \${日期} \${开始时间} \${结束时间} (查询逐笔委托数据)
 示例: QueryTickOrder SZSE 000002 20211229 90000000 95000000
- QueryTickExecution \${市场} \${证券代码} \${日期} \${开始时间} \${结束时间} (查询逐笔成交数据)
 示例: QueryTickExecution SZSE 000002 20211229 90000000 95000000
- QueryKline \${市场} \${k 线类型} \${证券代码} \${复权类型} \${自动补全} \${开始日期} \${结束日期} \${开始时间} \${结束时间} (查询 K 线数据)
 k 线类型: MD1MinKline、MD3MinKline、MD5MinKline、MD10MinKline、MD15MinKline、MD30MinKline、MD60MinKline、MD120MinKline、MDDayKline、MDWeekKline、MDMonthKline、MDSeasonKline、MDYearKline
 复权: 0 1 2 (0:不复权;1:向前复权;2:向后复权)
 自动补全: 0:不补齐, 1: 补齐 默 1
 示例 1: QueryKline SSE MD1MinKline 600004 0 1 20211229 20211229 900 1500
- QueryOrderQueue \${市场} \${证券代码} \${日期} \${开始时间} \${结束时间} (查询委托队列数据)
 示例: QueryOrderQueue SZSE 000002 20211229 90000000 95000000
- QuerySecurInfo \${市场类型} \${证券代码} (查询证券个股基本信息数据)
 示例: QuerySecurInfo SZSE 000002
- QueryCodeTable codeTable (查询证券代码表数据)
 示例: QueryCodeTable codeTable
- QueryExFactor \${证券代码} (查询复权因子表信息数据)
 示例: QueryExFactor 000002
- QueryFactor \${任务 id} \${因子类型} \${因子子类型} \${因子名称} \${证券代码} \${开始日期} \${结束日期} \${开始时间} \${结束时间} \${市场} \${品种} \${页偏移量} \${页数} (查询因子数据)
 示例: QueryFactor 20230301 A B C 20211229 20211229 90000000 95000000 000001 102 0 0 0 备注: 开始日期和结束日期需要相等, 目前不支持多天查询。页偏移量和页数暂不支持 (暂填 0)
- 三方资讯获取命令:
 设置参数: SetThirdInfoParam \${任务 id} \${Key} \${Value}
 示例: SetThirdInfoParam 2022112101 function_id A010060002 SetThirdInfoParam 2022112101 market_code 000001.SZ
 说明: 在进行查询三方资讯前, 必须首先设置 function_id+功能号及任务 id;
 function_id: 此 key 为必须输入的值 (此字段为指定值, 不能为其他值)。
 查询命令: QueryThirdInfo \${任务 id}
 示例: QueryThirdInfo 2022112101

- 更新密码获取命令:
UpdatePassWord \${用户名} \${旧密码} \${新密码}
示例: UpdatePassWord 1123 4561

- ETF 代码信息获取命令:
QueryETFInfo \${市场类型} \${证券代码}
示例: QueryETFInfo SSE 510050

2) 订阅命令类型: Sub(订阅)

UnSub(取消订阅)

SubFactor(订阅因子类型)

UnSubFactor(取消因子类型订阅)

格式: Sub \${市场类型} \${数据类型} \${品种类别} \${证券代码}

UnSub \${市场类型} \${数据类型} \${品种类别} \${证券代码}

- 市场类型: All (全市场), NEEQ (北京所), SHFE (上期所), CFFEX (中金所), DCE (大商所), CZCE (郑商所), INE (上海国际能源交易中心), SSE (上交所), SZSE (深交所), HKEx (港交所)。
- 数据类型: (多个数据类型之间用逗号“,”分隔) All 包含所有数据类型, 类型有 MDSnapshot, MD1MinKline, MDCnIndexSnapshot 等。
- 品种类别: All 包含所有类别, 类别有 Stock, Fund, Bond, Option, Index, HKT, FutureOption 等。
- 证券代码: 支持已有的任意代码
- 订阅命令示例: (多个数据类型之间用逗号“,”分隔)

示例: Sub SZSE MDSnapshot Stock 000002

Sub SZSE MD1MinKline All 000002

Sub All All All All

- 取消订阅命令示例: (多个数据类型之间用逗号“,”分隔)

示例: UnSub SZSE MDSnapshot Bond 000002

UnSub All All All All

- 因子类型订阅示例:

SubFactor all all all

SubFactor Quantitative_Factor MA MA_S

说明:

Quantitative_Factor: 因子父类型 MA: 因子子类型 MA_S: 因子名称

支持的全订阅方式为以下三种:

- ✧ 全订阅支持 all all all 方式;
- ✧ 支持 xxx all all 方式订阅 (指定父类型)
- ✧ 支持 xxx xxx all 方式订阅 (指定父类型、子类型);

不支持以下方式:

- ✧ 父类型不支持单独全订阅, 即 all xxx xxx;
- ✧ 因子子类型不支持单独全订阅, 即 xxx all xxx
- ✧ 不支持父类型和子类型全订阅, 因子名称为指定因子, 即 all lall xxx;
- ✧ 不支持父类型和因子名称全订阅, 子类型为指定类型, 即 all xxx all;

- 取消因子类型订阅示例:

示例: UnSubFactor all all all

UnSubFactor Quantitative_Factor MA MA_S

说明:

Quantitative_Factor: 因子父类型 MA: 因子子类型 MA_S: 因子名称

支持的取消因子全订阅方式为以下三种:

- ✧ 取消全订阅支持 allall all 方式;
- ✧ 支持 xxx all all 方式取消订阅 (指定父类型);
- ✧ 支持 xxx.xxx all 方式取消订阅 (指定父类型、子类型);

不支持以下方式:

- ✧ 父类型不支持单独取消全订阅, 即 all xxxx xx;
- ✧ 因子子类型不支持单独取消全订阅, 即 xxx all xxx;
- ✧ 不支持父类型和子类型取消全订阅, 因子名称为指定因子, 即 all all xxx;
- ✧ 不支持父类型和因子名称取消全订阅, 子类型为指定类型, 即 all xxx all;

■ 衍生数据订阅示例:

SubDerivedData 1 1 000002.SZSE,600381.SSE 增加订阅深市 000002 和沪市 600381 代码, 委托簿数据

SubDerivedData 2 2 000002.SZSE,600381.SSE 删除订阅沪市 600381 和深市 000002 代码, 委托簿快照数据

说明:

SubDerivedData \${订阅操作类型}\${衍生数据类型}\${证券代码.市场,证券代码.市场..}

订阅操作类型 (0: 重置订阅-取消旧订阅, 并设置新订阅 1: 增加订阅 2: 删除订阅 3: 取消所有订阅)

衍生数据类型 (1: 订阅委托簿数据 2: 订阅委托簿快照数据)

市场类别 (仅支持 SZSE、SSE)

- 3) 回放命令类型: \${命令类型} \${证券代码.市场,证券代码.市场...} \${开始日期} \${结束日期} \${开始日期} \${结束日期}

■ ReplayTickExecution\${证券代码.市场}\${订阅日期}\${开始日期}\${结束日期}\${开始时间}\${结束时间} (逐笔成交回放获取命令)

示例: ReplayTickExecution 000001.SZSE,600381.SSE 20200924 20200924 093000000 093500000

说明:

20200924: 表示 2020 年 9 月 24 日

093500000: 表示 9 点 35 分 00 秒 000 毫秒

备注: 支持多天回放

■ ReplaySnapshot\${证券代码.市场,证券代码.市场..}\${开始日期}\${结束日期}\${开始时间}\${结束时间} (历史快照回放获取命令)

示例: ReplaySnapshot000001.SZSE,600381.SSE 20211228 20211228 0930000000103600000

备注: 支持多天回放。

■ ReplayKLine \${k 线类型} \${证券代码.市场,证券代码.市场...} \${复权类型} \${自动补全} \${开始日期} \${结束日期} \${开始时间} \${结束时间} (历史 K 线回放获取命令)

示例 1: 分钟 k: ReplayKLine MD1MinKline 000001.SZSE,000002.SZSE 0 1 20201118 20201118 0930 11300

k 线类型: 可参考查询命令

复权: 0 1 2 (0: 不复权; 1: 向前复权; 2: 向后复权)

自动补全: 0: 不补齐, 1: 补齐 默认 1

4. 公共数据字典

所有公共数据字典的定义都在 C++/include/tgw_datatype.h 下面定义。

4.1 事件级别(EventLevel)

事件级别	数值	说明
kInfo	枚举(=1)	普通事件
kWarn	枚举(=2)	告警事件
kError	枚举(=3)	错误事件, 比较严重,需要介入处理

4.2 错误码(ErrorCode)

错误码	数值	说明
kFailure	枚举(-100)	失败
kUnInited	枚举(-99)	未初始化
kNullSpi	枚举(-98)	空指针
kParamIllegal	枚举(-97)	参数非法
kNetError	枚举(-96)	网络异常
kPermissionError	枚举(-95)	数据无权限
kLogonFailed	枚举(-94)	未登录
kAllocateMemoryFailed	枚举(-93)	分配内存失败
kChannelError	枚举(-92)	通道错误
kOverLoad	枚举(-91)	hqs 任务队列溢出
kLogoned	枚举(-90)	账号已登录
kHqsError	枚举(-89)	HQS 系统错误
kNonQueryTimePeriod	枚举(-88)	非查询时间段(非查询时间段不支持查询)
kDbAndCodeTableNoCode	枚举(-87)	数据库和代码表中没有指定的代码
kIllegalMode	枚举(-86)	api 模式非法
kThreadBusy	枚举(-85)	超过最大可用线程资源
kParseDataError	枚举(-84)	数据解析出错
kTimeout	枚举(-83)	获取数据超时
kFlowOverLimit	枚举(-82)	周流量耗尽
kCodeTableCacheNotAvailable	枚举(-81)	代码表缓存不可用
kOverMaxSubLimit	枚举(-80)	超过最大订阅限制
kLostConnection	枚举(-79)	丢失连接
kOverMaxQueryLimit	枚举(-78)	超过最大查询数(含代码表)
kFunctionIdNull	枚举(-77)	金融资讯数据查询未设置功能号
kDataEmpty	枚举(-76)	数据为空
kUserNotExist	枚举(-75)	用户不存在
kVerifyFailure	枚举(-74)	账号/密码错误
kApiInterfaceUsing	枚举(-73)	api 接口不能同时多次调用
kSuccess	枚举(0)	成功

4.3 数据定义长度(ConstField)

数据类型	数据名称	数值	说明
static const uint32_t	kIPMaxLen	24	IP 地址的最大长度
static const uint32_t	kUMSItemLen	10	服务项信息的最大个数
static const uint32_t	kUsernameLen	32	用户名的最大长度
static const uint32_t	kPasswordLen	64	用户名密码的最大长度
static const uint32_t	kSecurityCodeLen	16	证券代码最大长度
static const uint32_t	kHistorySecurityCodeLen	32	历史回放和查询证券代码最大长度
static const uint32_t	kPushSecurityCodeLen	32	实时推送证券代码最大长度
static const uint32_t	kSecurityNameLen	32	证券名称最大长度
static const uint32_t	kTradingPhaseCodeLen	8	交易状态标志
static const uint32_t	kExchangeInstIDLen	31	合约在交易所代码
static const uint32_t	kPositionLevelLen	10	行情档位
static const uint32_t	kMDStreamIDLen	6	行情类别
static const uint32_t	kSecurityStatusLen	24	证券状态最大长度
static const uint32_t	kSecuritySymbolLen	32	证券简称最大长度
static const uint32_t	kSecurityEnglishNameLen	128	证券英文简称最大长度
static const uint32_t	kSecurityTypeLen	10	证券类别长度
static const uint32_t	kCurrencyLen	4	币种长度
static const uint32_t	kVolumeTradeLen	24	成交量长度
static const uint32_t	kValueTradeLen	24	交易额长度
static const uint32_t	kFutureSecurityCodeLen	32	期货代码最大长度
static const uint32_t	kMDStreamIDMaxLen	6	行情类别字段最大长度
static const uint32_t	kDataPermission	128	数据权限最大长度
static const uint32_t	kTokenMaxLen	128	token 的最大长度
static const uint32_t	kQuerySecurityCodeLen	38	查询状态回调代码最大长度
static const uint32_t	kTradingStatusLen	8	证券交易状态档位
static const uint32_t	kSymbolLen	128	证券简称最大长度（包括中文简称）
static const uint32_t	kSecurityAbbreviationLen	64	证券简称最大长度
static const uint32_t	kMaxTypesLen	16	证券类型最大长度
static const uint32_t	kTypesLen	8	
static const uint32_t	kCodeTableSecurityStatusMaxLen	16	代码表证券状态字符最大长度
static const uint32_t	RegularShare	9	对应回购标准券长度
static const uint32_t	kConsSecurityCodeLen	32	成份证券代码长度
static const uint32_t	kSymbolETFLen	128	基金名称长度
static const uint32_t	KUnderlyingSecurityID	4	成分股所属市场 ID 长度
static const uint32_t	KReserved	30	预留字段长度
static const uint32_t	kManagmentETFLen	128	基金公司名称长度

static const uint32_t	KUnderlyingSecurityIDSource	4	拟合指数代码源长度
static const uint32_t	AllCashAmount	12	全现金替代的总金额长度
static const uint32_t	AllCashAremiumRate	7	全现金替代的申购溢价比例长度
static const uint32_t	AllCashDiscountRate	7	全现金替代的赎回折价比例长度
static const uint32_t	DefaultMaxOBSSubNum	500	委托簿默认最大订阅数量
static const uint32_t	DefaultMaxOBSnapshotSubNum	3000	委托簿快照默认最大订阅数量
static const uint32_t	DefaultMaxOBOOrderQueueSize	50	每一档委托队列数量上限
static const uint32_t	DefaultMaxOBEntrySize	20	委托簿档位数量上限
static const uint32_t	DefaultMaxOBDeliverIntervalMs	50	委托簿最小推送间隔
static const uint32_t	DefaultMinOBSnapshotDeliverIntervalMs	10	委托簿快照最小推送间隔

4.4 回放请求任务状态(HistoryTaskStatus)

数据类型	数据名称	数值	说明
static const uint8_t	kSuccess	0	任务成功完成
static const uint8_t	kFailed	1	任务执行失败
static const uint8_t	kTaskCancel	2	任务取消
static const uint8_t	kTaskWaiting	3	任务执行中
static const uint8_t	kTaskTimeOut	4	回放任务超时

4.5 数据类型(MDDatatype)

类型名	数值	说明
k1KLine	枚举(=10000)	1 分钟 K 线
k3KLine	枚举(=10001)	3 分钟 K 线
k5KLine	枚举(=10002)	5 分钟 K 线
k10KLine	枚举(=10003)	10 分钟 K 线
k15KLine	枚举(=10004)	15 分钟 K 线
k30KLine	枚举(=10005)	30 分钟 K 线
k60KLine	枚举(=10006)	60 分钟 K 线
k120KLine	枚举(=10007)	120 分钟 K 线
kDayKline	枚举(=10008)	日 K 线
kWeekKline	枚举(=10009)	周 K 线
kMonthKline	枚举(=10010)	月 K 线

kSeasonKline	枚举(=10011)	季 K 线
kYearKline	枚举(=10012)	年 K 线
kTickExecution	枚举(=10013)	逐笔成交
kSnapshot	枚举(=10014)	快照

4.6 日志输出级别(LogLevel)

日志输出级别	数值	说明
kTrace	枚举(=0)	跟踪级别日志
kDebug	枚举(=1)	调试级别日志
kInfo	枚举(=2)	普通级别日志
kWarn	枚举(=3)	警告级别日志
kError	枚举(=4)	错误级别日志
kFatal	枚举(=5)	致命级别日志

4.7 市场类型定义(MarketType)

市场类型	数值	说明
kNone	枚举(=0)	表示全市场
kNEEQ	枚举(=2)	北交所
kSHFE	枚举(=3)	上期所
kCFFEX	枚举(=4)	中金所
kDCE	枚举(=5)	大商所
kCZCE	枚举(=6)	郑商所
kINE	枚举(=7)	上海国际能源交易中心
kSSE	枚举(=101)	上交所
kSZSE	枚举(=102)	深交所
kHKEx	枚举(=103)	港交所
kBK	枚举(=201)	板块
kMax	枚举(=255)	市场类型最大值

4.8 API 通道模式(ApiMode)

数据类型	数据名称	数值	说明
static const uint16_t	kColocationMode	1	托管机房模式
static const uint16_t	kInternetMode	2	互联网模式

4.9 托管机房模式通道(ColocatChannelMode)

数据类型	数据名称	数值	说明
static const uint64_t	kTCP	0x0000000200000000	TCP 推送获取数据
static const uint64_t	kQTCP	0x0000000800000000	TCP 查询获取数据
static const uint64_t	kRTCP	0x0000001000000000	TCP 回放获取数据

4.10 订阅数据类型(SubscribeDataType)

数据类型	数据名称	数值	说明	适用模式
static const uint32_t	kNone	0	订阅全部数据	托管机房模式 互联网模式
static const uint32_t	k1MinKline	1	订阅 1 分钟 k 线数据	托管机房模式 互联网模式
static const uint32_t	k3MinKline	2	订阅 3 分钟 k 线数据	托管机房模式 互联网模式
static const uint32_t	k5MinKline	3	订阅 5 分钟 k 线数据	托管机房模式 互联网模式
static const uint32_t	k10MinKline	4	订阅 10 分钟 k 线数据	托管机房模式 互联网模式
static const uint32_t	k15MinKline	5	订阅 15 分钟 k 线数据	托管机房模式 互联网模式
static const uint32_t	k30MinKline	6	订阅 30 分钟 k 线数据	托管机房模式 互联网模式
static const uint32_t	k60MinKline	7	订阅 60 分钟 k 线数据	托管机房模式 互联网模式
static const uint32_t	k120MinKline	8	订阅 120 分钟 k 线数据	托管机房模式 互联网模式
static const uint32_t	kSnapshotDerive	9	订阅快照衍生数据	托管机房模式 互联网模式
static const uint32_t	kSnapshot	10	订阅现货快照数据	互联网模式
static const uint32_t	kOptionSnapshot	11	订阅期权快照数据	互联网模式
static const uint32_t	kHKTSnapshot	12	订阅港股快照数据	互联网模式
static const uint32_t	kIndexSnapshot	13	订阅指数快照数据	互联网模式
static const uint32_t	kAfterHourFixedPriceSnapshot	14	订阅盘后快照数据	互联网模式
static const uint32_t	kCSIIndexSnapshot	15	订阅中证指数数据	互联网模式
static const uint32_t	kCnIndexSnapshot	16	订阅国证指数快照数据	互联网模式
static const uint32_t	kHKTRealtimeLimit	17	订阅港股通实时额度数据	互联网模式
static const uint32_t	kHKTProductStatus	18	订阅港股通产品状态数据	互联网模式
static const uint32_t	kHKTVCM	19	订阅港股 VCM 数据	互联网模式

static const uint32_t	kFutureSnapshot	20	订阅期货数据	互联网模式
-----------------------	-----------------	----	--------	-------

4.11 品种类型定义(VarietyCategory)

数据类型	数据名称	数值	说明
static const uint8_t	kNone	0	None
static const uint8_t	kStock	1	股票
static const uint8_t	kFund	2	基金
static const uint8_t	kBond	3	债券
static const uint8_t	kOption	4	期权
static const uint8_t	kIndex	5	指数
static const uint8_t	kHKT	6	港股通
static const uint8_t	kFutureOption	7	期货期权
static const uint8_t	kCFETSRMB	8	银行间本币交易产品
static const uint8_t	kHKEx	9	港股
static const uint8_t	kOthers	255	其他

4.12 衍生订阅类型(SubscribeType)

数据类型	数据名称	数值	说明
static const uint8_t	kSet	0	重新设置订阅(先取消所有旧订阅信息, 然后增加新订阅信息)
static const uint8_t	kAdd	1	添加订阅
static const uint8_t	kDel	2	删除订阅
static const uint8_t	kCancelAll	3	取消所有订阅

4.13 行情衍生数据订阅类型(SubscribeDerivedDataType)

数据类型	数据名称	数值	说明
static const uint64_t	kOrderBook	1	订阅委托簿数据
static const uint64_t	kOrderBookSnapshot	2	订阅委托簿快照数据

5. 行情数据字典

所有行情数据字典的定义都在 C++/include/tgw_struct.h 下面定义。

5.1 K 线(MDKLine)

数据类型	字段名称	说明
uint8_t	market_type	市场类型
char	security_code[ConstField::kPushSecurityCodeLen]	证券代码
int64_t	orig_time	时间 (YYYYMMDDHHMMSSsss)
int64_t	kline_time	k 线时间
int64_t	open_price	开盘价, 实际值需除以 1000000
int64_t	high_price	最高价, 实际值需除以 1000000
int64_t	low_price	最低价, 实际值需除以 1000000
int64_t	close_price	最新价, 实际值需除以 1000000
int64_t	volume_trade	成交量, 无倍数放大
int64_t	value_trade	成交金额, 无倍数放大

5.2 现货衍生(MDSnapshotDerive)

数据类型	字段名称	说明
uint8_t	market_type	市场类型
char	security_code[ConstField::kSecurityCodeLen]	证券代码
int64_t	orig_time	交易所行情数据时间 (格式为 YYYYMMDDHHMMSSsss)
uint64_t	average_price	均价, 实际值需除以 1000000
uint64_t	circulation_value	流通市值, 实际值需除以 1000000
uint64_t	total_value	总市值, 实际值需除以 1000000
uint64_t	initiative_buy_volume	外盘, 实际值需除以 100
uint64_t	initiative_sell_volume	内盘, 实际值需除以 100
uint32_t	turnover_rate	换手率, 实际值需除以 100000
int32_t	volume_ratio	量比, 实际值需除以 100000
int32_t	ask_bid_ratio	委比, 实际值需除以 100000
uint32_t	amplitude	振幅, 实际值需除以 100000
int32_t	PE_static	静态市盈率, 实际值需除以 100
int32_t	PE_dynamic	动态市盈率, 实际值需除以 100

int32_t	PE_TTM	最近 4 季度滚动市盈率，实际值需除以 100
int32_t	PB	市净率，实际值需除以 100
int64_t	entrustment_diff	委差
char	initiative_flag	内外盘标记(B/S)

5.3 加工因子(Factor)

数据类型	字段名称	说明
uint32_t	data_size	json 数据的大小
char*	json_buf	json 结构

5.4 L1 现货快照(MDSnapshotL1)

数据类型	字段名称	说明
uint8_t	market_type	市场类型
char	security_code[ConstField::kSecurityCodeLen]	证券代码
uint8_t	variety_category	品种类别（参照描述）
int64_t	orig_time	交易所行情数据时间 (格式为 YYYYMMDDHHMMSSsss)
char	trading_phase_code[ConstField::kTradingPhaseCodeLen]	交易阶段代码
int64_t	pre_close_price	昨收价，实际值需除以 1000000
int64_t	open_price	开盘价，实际值需除以 1000000
int64_t	high_price	最高价，实际值需除以 1000000
int64_t	low_price	最低价，实际值需除以 1000000
int64_t	last_price	最新价，实际值需除以 1000000
int64_t	close_price	收盘价，实际值需除以 1000000（北交所为 0）
int64_t	bid_price[ConstField::kPositionLevelLen]	申买价，实际值需除以 1000000
int64_t	bid_volume[ConstField::kPositionLevelLen]	申买量，实际值需除以 100
int64_t	offer_price[ConstField::kPositionLevelLen]	申卖价，实际值需除以 1000000
int64_t	offer_volume[ConstField::kPositionLevelLen]	申卖量，实际值需除以 100
int64_t	num_trades	成交笔数
int64_t	total_volume_trade	成交总量，实际值需除以 100

int64_t	total_value_trade	成交总金额，实际值需除以 100000
int64_t	IOPV	IOPV 净值估产（仅基金品种有效），实际值需除以 1000000，北交所为 0
int64_t	high_limited	涨停价，实际值需除以 1000000
int64_t	low_limited	跌停价，实际值需除以 1000000

5.5 指数快照(MDIndexSnapshot)

数据类型	字段名称	说明
uint8_t	market_type	市场类型
char	security_code[ConstField::kSecurityCodeLen]	证券代码
int64_t	orig_time	交易所行情数据时间 (格式为 YYYYMMDDHHMMSSsss)
char	trading_phase_code[ConstField::kTradingPhaseCodeLen]	产品实时阶段及标志（仅深圳有效）
int64_t	pre_close_index	前收盘指数，实际值需除以 1000000
int64_t	open_index	今开盘指数，实际值需除以 1000000
int64_t	high_index	最高指数，实际值需除以 1000000
int64_t	low_index	最低指数，实际值需除以 1000000
int64_t	last_index	最新指数，实际值需除以 1000000
int64_t	close_index	收盘指数（仅上海有效），实际值需除以 1000000
int64_t	total_volume_trade	成交总量，实际值需除以 100（上交所:手，深交所:张）
int64_t	total_value_trade	成交总金额，实际值需除以 100000

说明：商业港股指数快照也使用 MDIndexSnapshot 结构

5.6 期权快照(MDOptionSnapshot)

数据类型	字段名称	说明
uint8_t	market_type	市场类型
char	security_code[ConstField::kSecurityCodeLen]	期权代码
int64_t	orig_time	交易所行情数据时间 (格式为 YYYYMMDDHHMMSSsss)
char	trading_phase_code[ConstField::kTradingPhaseCodeLen]	产品实时阶段及标志
int64_t	total_long_position	总持仓量，实际值需除以 100
int64_t	total_volume_trade	成交总量，实际值需除以 100

int64_t	total_value_trade	成交总金额，实际值需除以 100000
int64_t	pre_settle_price	昨结算价（仅上海有效），实际值需除以 1000000
int64_t	pre_close_price	昨收盘价，实际值需除以 1000000
int64_t	open_price	今开盘价，实际值需除以 1000000
int64_t	auction_price	动态参考价（波动性中断参考价，仅上海有效），实际值需除以 1000000
int64_t	auction_volume	虚拟匹配数量（仅上海有效），实际值需除以 100
int64_t	high_price	最高价，实际值需除以 1000000
int64_t	low_price	最低价，实际值需除以 1000000
int64_t	last_price	最新价，实际值需除以 1000000
int64_t	close_price	今收盘价，实际值需除以 1000000
int64_t	high_limited	涨停价，实际值需除以 1000000
int64_t	low_limited	跌停价，实际值需除以 1000000
int64_t	bid_price[5]	申买价，实际值需除以 1000000
int64_t	bid_volume[5]	申买量，实际值需除以 100
int64_t	offer_price[5]	申卖价，实际值需除以 1000000
int64_t	offer_volume[5]	申卖量，实际值需除以 100
int64_t	settle_price	今结算价（仅上海有效），实际值需除以 1000000
int64_t	ref_price	参考价（仅深圳有效），实际值需除以 1000000（此字段仅互联网模式有效）
char	contract_type	合约类别（此字段仅互联网模式有效）
int32_t	expire_date	到期日（此字段仅互联网模式有效）
char	underlying_security_code[ConstField::kSecurityCodeLen]	标的代码（此字段仅互联网模式有效）
int64_t	exercise_price	行权价，实际值需除以 1000000（此字段仅互联网模式有效）

5.7 港股通快照(MDHKTSnapshot)

数据类型	字段名称	说明
uint8_t	market_type	市场类型
char	security_code[ConstField::kSecurityCodeLen]	港股代码
int64_t	orig_time	交易所行情数据时间 (格式为 YYYYMMDDHHMMSSsss)
char	trading_phase_code[ConstField::kTradingPhaseCodeLen]	产品实时阶段及标志
int64_t	total_volume_trade	成交总量，实际值需除以 100

int64_t	total_value_trade	成交总金额, 实际值需除以 100000
int64_t	pre_close_price	昨收价, 实际值需除以 1000000
int64_t	nominal_price	按盘价, 实际值需除以 1000000
int64_t	high_price	最高价, 实际值需除以 1000000
int64_t	low_price	最低价, 实际值需除以 1000000
int64_t	last_price	最新价, 实际值需除以 1000000
int64_t	bid_price[5]	申买价,(正常情况下为一档, 集中竞价阶段可能有两档)实际值需除以 1000000
int64_t	bid_volume[5]	申买量,(正常情况下为一档, 集中竞价阶段可能有两档)实际值需除以 100
int64_t	offer_price[5]	申卖价,(正常情况下为一档, 集中竞价阶段可能有两档)实际值需除以 1000000
int64_t	offer_volume[5]	申卖量,(正常情况下为一档, 集中竞价阶段可能有两档)实际值需除以 100
int64_t	ref_price	参考价(此字段仅互联网模式有效)
int64_t	high_limited	冷静期价格上限(此字段仅互联网模式有效)
int64_t	low_limited	冷静期价格下限(此字段仅互联网模式有效)
int64_t	bid_price_limit_up	买盘上限价, 实际值需除以 1000000(仅深圳有效)(此字段仅互联网模式有效)
int64_t	bid_price_limit_down	买盘下限价, 实际值需除以 1000000(仅深圳有效)(此字段仅互联网模式有效)
int64_t	offer_price_limit_up	卖盘上限价, 实际值需除以 1000000(仅深圳有效)(此字段仅互联网模式有效)
int64_t	offer_price_limit_down	卖盘下限价, 实际值需除以 1000000(仅深圳有效)(此字段仅互联网模式有效)

5.8 期货快照(MDFutureSnapshot)

数据类型	字段名称	说明
uint8_t	market_type	市场类型
char	security_code[ConstField::kFutureSecurityCodeLen]	合约代码
int64_t	orig_time	交易所行情数据时间 格式为 YYYYMMDDHHMMSSsss (ActionDay+UpdateTime+UpdateMillisec)
int32_t	action_day	业务日期
int64_t	last_price	最新价, 实际值需除以 1000000
int64_t	pre_settle_price	上次结算价, 实际值需除以 1000000
int64_t	pre_close_price	昨收价, 实际值需除以 1000000

int64_t	pre_open_interest	昨持仓量，实际值需除以 100
int64_t	open_price	开盘价，实际值需除以 1000000
int64_t	high_price	最高价，实际值需除以 1000000
int64_t	low_price	最低价，实际值需除以 1000000
int64_t	total_volume_trade	数量，实际值需除以 100
int64_t	total_value_trade	总成交金额，实际值需除以 100000
int64_t	open_interest	持仓量，实际值需除以 100
int64_t	close_price	今收盘，实际值需除以 1000000
int64_t	settle_price	本次结算价，实际值需除以 1000000
int64_t	high_limited	涨停板价，实际值需除以 1000000
int64_t	low_limited	跌停板价，实际值需除以 1000000
int64_t	pre_delta	昨虚实度，实际值需除以 1000000
int64_t	curr_delta	今虚实度，实际值需除以 1000000
int64_t	bid_price[5]	申买价，实际值需除以 1000000
int64_t	bid_volume[5]	申买量，实际值需除以 100
int64_t	offer_price[5]	申卖价，实际值需除以 1000000
int64_t	offer_volume[5]	申卖量，实际值需除以 100
int64_t	average_price	当日均价，实际值需除以 1000000
int32_t	trading_day	交易日期
uint8_t	variety_category	品种类别（暂不可用）
int64_t	latest_volume_trade	最新成交量，实际值需除以 100（此字段仅托管机房模式有效）
int64_t	init_volume_trade	初始持仓量，实际值需除以 100（此字段仅托管机房模式有效）
int64_t	change_volume_trade	持仓量变化，实际值需除以 100（此字段仅托管机房模式有效）
int64_t	bid_imply_volume	申买推导量，实际值需除以 100（此字段仅托管机房模式有效）
int64_t	offer_imply_volume	申卖推导量，实际值需除以 100（此字段仅托管机房模式有效）
int64_t	total_bid_volume_trade	总买入量，实际值需除以 100（此字段仅托管机房模式有效）
int64_t	total_ask_volume_trade	总卖出量，实际值需除以 100（此字段仅托管机房模式有效）
char	exchange_inst_id[ConstField::kExchangeInstIDLen]	合约在交易所的代码（此字段仅互联网模式有效）

5.9 商业港股委托挂单(MDHKExOrderSnapshot)

数据类型	字段名称	说明
uint8_t	market_type	市场类型（港交所：103）
char	security_code[ConstField::	证券代码

	kSecurityCodeLen]	
int64_t	orig_time	交易所行情数据时间 格式为 YYYYMMDDHHMMSSsss
MDHKExOperation	operation	执行动作 参考 MDHKExOperation 结构
MDHKExListItem	bid_list[ConstField::kHKE xOrderSnapshotLevelLen]	买单序列 (20 档) 参考 MDHKExListItem 结构
MDHKExListItem	ask_list[ConstField::kHKE xOrderSnapshotLevelLen]	卖单序列 (20 档) 参考 MDHKExListItem 结构

5.10 商业港股执行动作(MDHKExOperation)

数据类型	字段名称	说明
char	operation_type[10]	操作类型, 值为 add/mod/del 其中一个, 约定长度为 10
int64_t	order_id	委托 id
char	side	买卖方向, B 买, S 卖
uint8_t	order_type	仅 add 操作类型有效, '1'市价, '2'限价
int64_t	order_price	仅 add 操作类型有效, 委托价格, 数值=真实值*1000
int64_t	order_volume	add,mod 操作类型有效, 单位为股

5.11 商业港股挂单买卖序列 (MDHKExListItem)

数据类型	字段名称	说明
int64_t	order_price	挂单价格, 实际值需除以 1000000
int64_t	order_volume	挂单数量, 实际值需除以 100
int64_t	num_of_orders	挂单经纪席位数目

5.12 商业港股经纪商席位(MDHKExOrderBrokerSnapshot)

数据类型	字段名称	说明
uint8_t	market_type	市场类型 (港交所: 103)
char	security_code[ConstField::kSecurityCodeLen]	证券代码
int64_t	orig_time	交易所行情数据时间 格式为 YYYYMMDDHHMMSSsss
uint8_t	side	买卖方向 'B' 买, 'S' 卖
char	broker_flag	是否还有未展示完的席位排位 Y-是 N-否
MDHKExD	detail[ConstField::kDetailL	买卖经纪席位明细 所有档位的席位号之和最

etailItem	en]	多有 40 个, 参看 MDHKExDetailItem 结构
-----------	-----	---

5.13 港股经纪席位详细(MDHKExDetailItem)

数据类型	字段名称	说明
uint8_t	level	档位信息
uint16_t	broker_num	席位号

5.14 盘后定价交易快照(MDAfterHourFixedPriceSnapshot)

数据类型	字段名称	说明
uint8_t	market_type	市场类型
char	security_code[ConstField::kSecurityCodeLen]	证券代码
uint8_t	variety_category	品种类别
int64_t	orig_time	交易所行情数据时间 (格式为 YYYYMMDDHHMMSSsss)
char	trading_phase_code[ConstField::kTradingPhaseCodeLen]	交易阶段代码
int64_t	close_price	今日收盘价 (仅上海有效), 实际值需除以 1000000
int64_t	bid_price	申买价, 实际值需除以 1000000
int64_t	bid_volume	申买量, 实际值需除以 100
int64_t	offer_price	申卖价, 实际值需除以 1000000
int64_t	offer_volume	申卖量, 实际值需除以 100
int64_t	pre_close_price	昨收价, 实际值需除以 1000000
int64_t	num_trades	成交笔数
int64_t	total_volume_trade	成交总量, 实际值需除以 100
int64_t	total_value_trade	成交总金额, 实际值需除以 100000

5.15 中证指数行情(MDCSIIndexSnapshot)

数据类型	字段名称	说明
uint8_t	market_type	市场类型
uint8_t	index_market	指数市场
char	security_code[ConstField::kSecurityCodeLen]	证券代码
int64_t	orig_time	交易所行情数据时间 (格式为 YYYYMMDDHHMMSSsss)

int64_t	last_index	最新指数，实际值需除以 1000000
int64_t	open_index	今开盘指数，实际值需除以 1000000
int64_t	high_index	最高指数，实际值需除以 1000000
int64_t	low_index	最低指数，实际值需除以 1000000
int64_t	close_index	收盘指数，实际值需除以 1000000
int64_t	pre_close_index	前收盘指数，实际值需除以 1000000
int64_t	change	涨跌，实际值需除以 1000000
int64_t	ratio_of_change	涨跌幅，实际值需除以 1000000
int64_t	total_volume_trade	成交总量，实际值需除以 100
int64_t	total_value_trade	成交总金额，实际值需除以 100000(单位为万元)
int64_t	exchange_rate	汇率，实际值需除以 100000000
char	currency_symbol	币种标志(0-人民币 1-港币 2-美元 3-台币 4-日元)

5.16 国证指数快照(MDCnIndexSnapshot)

数据类型	字段名称	说明
uint8_t	market_type	市场类型
char	security_code[ConstField::kSecurityCodeLen]	证券代码
int64_t	orig_time	交易所行情数据时间 (格式为 YYYYMMDDHHMMSSsss)
char	trading_phase_code[ConstField::kTradingPhaseCodeLen]	产品实时阶段及标志
int64_t	pre_close_index	前收盘指数，实际值需除以 1000000
int64_t	open_index	今开盘指数，实际值需除以 1000000
int64_t	high_index	最高指数，实际值需除以 1000000
int64_t	low_index	最低指数，实际值需除以 1000000
int64_t	last_index	最新指数，实际值需除以 1000000
int64_t	close_index	收盘指数，实际值需除以 1000000
int64_t	total_volume_trade	成交总量，实际值需除以 100
int64_t	total_value_trade	成交总金额，实际值需除以 100000

5.17 港股通实时额度(MDHKTRealtimeLimit)

数据类型	字段名称	说明
uint8_t	market_type	市场类型
int64_t	orig_time	交易所行情数据时间

		(格式为 YYYYMMDDHHMMSSsss)
int64_t	threshold_amount	每日初始额度, 单位人民币元, 实际值需除以 100000
int64_t	pos_amt	日中剩余额度, 单位人民币元, 实际值需除以 100000
char	amount_status	额度状态(1-额度用完或其他原因全市场禁止买入 2-额度可用)
char	mkt_status[ConstField::kTradingStatusLen]	上交所港股通市场状态(上交所独有, 来源于上交所文件行情)

5.18 港股通产品状态(MDHKTProductStatus)

数据类型	字段名称	说明
uint8_t	market_type	市场类型
char	security_code[ConstField::kSecurityCodeLen]	证券代码
int64_t	orig_time	交易所行情数据时间 (格式为 YYYYMMDDHHMMSSsss)
char	trading_status1[ConstField::kTradingStatusLen]	证券交易状态 (整手订单)
char	trading_status2[ConstField::kTradingStatusLen]	证券交易状态 (零股订单)

5.19 港股 VCM(MDHKTVCVM)

数据类型	字段名称	说明
uint8_t	market_type	市场类型
char	security_code[ConstField::kSecurityCodeLen]	港股代码
int64_t	orig_time	交易所行情数据时间 (格式为 YYYYMMDDHHMMSSsss)
int64_t	start_time	市调机制开始时间
int64_t	end_time	市调机制结束时间
int64_t	ref_price	市调机制参考价格, 实际值需除以 1000000
int64_t	low_price	市调机制最低价格, 实际值需除以 1000000
int64_t	high_price	市调机制最高价格, 实际值需除以 1000000

5.20L2 现货快照(MDSnapshotL2)

数据类型	字段名称	说明
uint8_t	market_type	市场类型
char	security_code[ConstField::kSecurityCodeLen]	证券代码
int64_t	orig_time	交易所行情数据时间 (格式为 YYYYMMDDHHMMSSsss)
char	trading_phase_code[ConstField::kTradingPhaseCodeLen]	交易阶段代码
int64_t	pre_close_price	昨收价, 实际值需除以 1000000
int64_t	open_price	开盘价, 实际值需除以 1000000
int64_t	high_price	最高价, 实际值需除以 1000000
int64_t	low_price	最低价, 实际值需除以 1000000
int64_t	last_price	最新价, 实际值需除以 1000000
int64_t	close_price	收盘价 (仅上海有效), 实际值需除以 1000000
int64_t	bid_price[ConstField::kPositionLevelLen]	申买价, 实际值需除以 1000000
int64_t	bid_volume[ConstField::kPositionLevelLen]	申买量, 实际值需除以 100
int64_t	offer_price[ConstField::kPositionLevelLen]	申卖价, 实际值需除以 1000000
int64_t	offer_volume[ConstField::kPositionLevelLen]	申卖量, 实际值需除以 100
int64_t	num_trades	成交笔数
int64_t	total_volume_trade	成交总量, 实际值需除以 100
int64_t	total_value_trade	成交总金额, 实际值需除以 100000
int64_t	total_bid_volume	委托买入总量, 实际值需除以 100
int64_t	total_offer_volume	委托卖出总量, 实际值需除以 100
int64_t	weighted_avg_bid_price	加权平均为委买价格, 实际值需除以 1000000
int64_t	weighted_avg_offer_price	加权平均为委卖价格, 实际值需除以 1000000
int64_t	IOPV	IOPV 净值估产, 实际值需除以 1000000
int64_t	yield_to_maturity	到期收益率, 实际值需除以 1000
int64_t	high_limited	涨停价, 实际值需除以 1000000
int64_t	low_limited	跌停价, 实际值需除以 1000000
int64_t	price_earning_ratio1	市盈率 1, 实际值需除以 1000000
int64_t	price_earning_ratio2	市盈率 2, 实际值需除以 1000000
int64_t	change1	涨跌 1(对比昨收价), 实际值需除以 1000000
int64_t	change2	涨跌 2(对比上一笔), 实际值需除以 1000000

说明: 商业港股股票行情快照也使用 MDSnapshotL2 结构

5.21 现货逐笔成交(MDTickExecution)

数据类型	字段名称	说明
int32_t	market_type	市场类型
char	security_code[ConstField::kSecurityCodeLen]	证券代码
int64_t	exec_time	成交时间 (格式为 YYYYMMDDHHMMSSsss)
int32_t	channel_no	频道号
int64_t	appl_seq_num	频道编号
int64_t	exec_price	成交价格(类型:价格)
int64_t	exec_volume	成交数量(类型:数量)
int64_t	value_trade	成交金额(类型:金额)
int64_t	bid_appl_seq_num	买方委托索引
int64_t	offer_appl_seq_num	卖方委托索引
uint8_t	side	买卖方向(仅上海有效 B-外盘,主动买 S-内盘,主动卖 N-未知)
uint8_t	exec_type	成交类型(深圳:4-撤销 F-成交,上海:F-成交)
char	md_stream_id[ConstField::kMDStreamIDMaxLen]	行情类别(仅深圳有效)
int64_t	biz_index	业务序号
uint8_t	variety_category	品种类别(此字段暂不可用)

5.22 逐笔委托(MDTickOrder)

数据类型	字段名称	说明
uint8_t	market_type	市场类型
char	security_code[ConstField::kSecurityCodeLen]	证券代码
int64_t	appl_seq_num	消息记录号
int32_t	channel_no	频道编号
int64_t	order_time	委托时间 (格式为 YYYYMMDDHHMMSSsss)
int64_t	order_price	委托价格
int64_t	order_volume	委托数量
uint8_t	side	买卖方向 深圳市场:(1-买 2-卖 G-借入 F-出借) 上海市场:(B:买单,S:卖单)
uint8_t	order_type	订单类别
char	md_stream_id[ConstField::kMDStreamIDLen]	行情类别
char	product_status[ConstField::kTr	产品状态(仅上海有效)

	adingPhaseCodeLen]	
int64_t	orig_order_no	原始订单号
int64_t	biz_index	业务序号

5.23 委托队列(MDOrderQueue)

数据类型	字段名称	说明
uint8_t	market_type	市场类型
char	security_code[ConstField::kSecurityCodeLen]	证券代码
int64_t	order_time	委托时间 格式为 YYYYMMDDHHMMSSsss
uint8_t	side	买卖方向(B-买 S-卖)
int64_t	order_price	委托价格
int64_t	order_volume	委托数量
int32_t	num_of_orders	总委托笔数
int32_t	items	明细个数
int64_t	volume[50]	订单明细
int32_t	channel_no	频道代码
char	md_stream_id[ConstField::kMDStreamIDLen]	行情类别

5.24 代码表(MDCodeTable)

数据类型	字段名称	说明
char	security_code[ConstField::kSecurityCodeLen]	交易所证券代码
char	symbol[ConstField::kSecuritySymbolLen]	证券简称
char	english_name[ConstField::kSecurityEnglishNameLen]	英文简称
uint8_t	market_type	市场类型
char	security_type[ConstField::kSecurityTypeLen]	交易所定义证券类别(参看补充字段取值说明)
char	currency[ConstField::kCurrencyLen]	币种

说明：此数据结构为 IGMDCodeTableSpi 中 OnMDCodeTable 数据回调接口所用。

5.25 复权因子表(MDExFactorTable)

数据类型	字段名称	说明
char	inner_code[ConstField::kSecurityCodeLen]	证券内部代码
char	security_code[ConstField::kSecurityCodeLen]	证券代码
uint32_t	ex_date	除权除息日（格式为 yyyyMMdd）
double	ex_factor	复权因子
double	cum_factor	累计复权因子

5.26 个股基础信息(MDStockInfo)

数据类型	字段名称	说明
char	security_code[ConstField::kSecurityCodeLen]	交易所证券代码
char	symbol[ConstField::kSecuritySymbolLen]	证券简称
uint8_t	market_type	市场类型
char	security_type[ConstField::kSecurityTypeLen]	证券类别
char	currency[ConstField::kCurrencyLen]	币种
char	security_status[ConstField::kSecurityStatusLen]	证券状态
uint64_t	pre_close_price	昨收价
uint64_t	total_shares	总股本
uint64_t	flow_shares	流通股本
uint8_t	noprofit	是否盈利 Y: 是, 未盈利 N: 否, 已盈利
uint8_t	weighted_voting_rights	是否存在投票权差异 Y.是 N.否
uint8_t	registration_flag	是否注册制 Y.是 N.否
double	eps	每股收益
double	eps_cell	预计每股收益
double	net_profit_ttm	净利润（TTM）
double	net_profit	净利润
double	net_asset	净资产
double	net_profit_recent_annual	净利润
double	net_profit_first_quarter	净资产

5.27 证券静态信息(MDCodeTableRecord)

数据类型	字段名称	说明
char	security_code[ConstField::kFutureSecurityCodeLen]	证券代码
uint8_t	market_type	证券市场
char	symbol[ConstField::kSymbolLen]	证券简称
char	english_name[ConstField::kSecurityAbbreviationLen]	证券英文名
char	security_type[ConstField::kMaxTypesLen]	自定义证券子类别（参看补充字段取值说明）
char	currency[ConstField::kTypesLen]	币种 (CNY: 人民币, HKD: 港币, USD: 美元, AUD: 澳币, CAD: 加币, JPY: 日圆, SGD: 新加坡币, GBP: 英镑, EUR: 欧元, TWD: 新台币, Other: 其他)
uint8_t	variety_category	品种类别
int64_t	pre_close_price	昨收价(类型:价格)
char	underlying_security_id[ConstField::kSecurityCodeLen]	标的代码(仅期权/权证/期货期权有效)
char	contract_type[ConstField::kMaxTypesLen]	合约类别(仅期权/期货期权有效)
int64_t	exercise_price	行权价(仅期权/期货期权有效,类型:价格)
uint32_t	expire_date	到期日(仅期权/期货期权有效)
int64_t	high_limited	涨停价(类型:价格)
int64_t	low_limited	跌停价(类型:价格)
char	security_status[ConstField::kCodeTableSecurityStatusMaxLen]	产品状态标志
int64_t	price_tick	最小价格变动单位(类型:价格)
int64_t	buy_qty_unit	限价买数量单位(类型:数量)
int64_t	sell_qty_unit	限价卖数量单位(类型:数量)
int64_t	market_buy_qty_unit	市价买数量单位(类型:数量)
int64_t	market_sell_qty_unit	市价卖数量单位(类型:数量)
int64_t	buy_qty_lower_limit	限价买数量下限(类型:数量)
int64_t	buy_qty_upper_limit	限价买数量上限(类型:数量)
int64_t	sell_qty_lower_limit	限价卖数量下限(类型:数量)
int64_t	sell_qty_upper_limit	限价卖数量上限(类型:数量)
int64_t	market_buy_qty_lower_limit	市价买数量下限(类型:数量)
int64_t	market_buy_qty_upper_limit	市价买数量上限(类型:数量)
int64_t	market_sell_qty_lower_limit	市价卖数量下限(类型:数量)
int64_t	market_sell_qty_upper_limit	市价卖数量上限(类型:数量)

uint32_t	list_day	上市日期
int64_t	par_value	面值(类型:价格)
int64_t	outstanding_share	总发行量(上交所不支持,类型:数量)
int64_t	public_float_share_quantity	流通股数(上交所不支持,类型:数量)
int64_t	contract_multiplier	对回购标准券折算率(类型:比例)
char	regular_share[ConstField::RegularShare]	对应回购标准券(仅深交所)
int64_t	interest	应计利息(类型:汇率)
int64_t	coupon_rate	票面年利率(类型:比例)
char	product_code[ConstField::kFutureSecurityCodeLen]	期货品种产品代码(仅期货期权有效)
uint32_t	delivery_year	交割年份(仅期货期权有效)
uint32_t	delivery_month	交割月份(仅期货期权有效)
uint32_t	create_date	创建日期(仅期货期权有效)
uint32_t	start_deliv_date	开始交割日(仅期货期权有效)
uint32_t	end_deliv_date	结束交割日(仅期货期权有效)
uint32_t	position_type	持仓类型(仅期货期权有效)

说明：此数据结构为 IGMDStockInfoSpi 中 OnMDStockInfo 数据回调接口所用。

5.28 金融资讯数据(ThirdInfoData)

数据类型	字段名称	说明
uint64_t	task_id	任务 id
uint64_t	data_size	数据大小
char*	json_data	数据 json 串

5.29 委托簿(MDOrderBook)

数据类型	字段名称	说明
int32_t	channel_no	频道号
int32_t	market_type	市场类型
char	security_code[ConstField::kSecurityCodeLen]	证券代码
int64_t	last_tick_time	最新逐笔生成时间
int64_t	last_snapshot_time	最新快照生成时间(固定为 0,无实际意义)
int64_t	last_tick_seq	最新逐笔序列号
std::vector<MDOrderBookItem>	bid_order_book	买委托簿 (MDOrderBookItem 参考委托簿 Item)
std::vector<MDOrderBookItem>	offer_order_book	卖委托簿 (MDOrderBookItem 参考委托簿 Item)

int64_t	total_num_trades	基于委托簿演算的成交总笔数
int64_t	total_volume_trade	基于委托簿演算的成交总量, 实际值需除以 100
int64_t	total_value_trade	基于委托簿演算的成交总金额, 实际值需除以 100000
int64_t	last_price	基于委托簿演算的最新价, 实际值需除以 1000000

5.30 委托簿价格档位数据 (MDOOrderBookItem)

数据类型	字段名称	说明
int64_t	price	价格, 实际值需除以 1000000
int64_t	volume	总数量, 实际值需除以 100
int64_t	order_queue_size	委托队列大小
int64_t	order_queue[50]	委托队列数量, 最多揭示 50 笔

5.31 委托簿快照(MDOOrderBookSnapshot)

数据类型	字段名称	说明
uint8_t	market_type	市场类型, 参考公共数据字典市场类型 (MarketType)
uint8_t	variety_category	品种类别, 参照公共数据字典品种类型 (VarietyCategory)
char	security_code[ConstField::kSecurity CodeLen]	证券代码
int64_t	last_tick_seq	构建快照的最新逐笔记录号
int32_t	channel_no	构建快照的逐笔原始频道编号
int64_t	orig_time	基于委托簿演算的行情快照时间(最新逐笔记录号对应的逐笔时间)(格式为 YYYYMMDDHHMMSSs)
int64_t	last_price	基于委托簿演算的最新价, 实际值需除以 1000000
int64_t	total_num_trades	基于委托簿演算的成交总笔数
int64_t	total_volume_trade	基于委托簿演算的成交总量, 实际值需除以 100
int64_t	total_value_trade	基于委托簿演算的成交总金额, 实际值需除以 100000
int64_t	total_bid_volume	基于委托簿演算的委托买入总量, 实际值需除以 100
int64_t	total_offer_volume	基于委托簿演算的委托卖出总量, 实际值需除以 100
int64_t	num_bid_orders	基于委托簿演算的买方委托价位数

int64_t	num_offer_orders	基于委托簿演算的卖方委托价位数
int64_t	bid_price[ConstField::kPositionLevelLen]	申买价，实际值需除以 10000
int64_t	bid_volume[ConstField::kPositionLevelLen]	申买量，实际值需除以 100
int64_t	offer_price[ConstField::kPositionLevelLen]	申卖价，实际值需除以 100
int64_t	offer_volume[ConstField::kPositionLevelLen]	申卖量，实际值需除以 100

5.32ETF 代码表(MDETFCodeTableRecord)

数据类型	字段名称	说明
char	security_code[ConstField::kSecurityCodeLen]	证券代码
int64_t	creation_redemption_unit	每个篮子对应的 ETF 份数
int64_t	max_cash_ratio	最大现金替代比例
char	publish	是否发布 IOPV,Y=是,N=否
char	creation	是否允许申购,Y=是,N=否(仅深圳有效)
char	redemption	是否允许赎回,Y=是,N=否(仅深圳有效)
char	creation_redemption_switch	申购赎回切换(仅上海有效,0-不允许申购/赎回,1-申购和赎回皆允许,2-仅允许申购,3-仅允许赎回)
int64_t	record_num	深市成份证券数目
int64_t	total_record_num	所有成份证券数量
int64_t	estimate_cash_component	预估现金差额
int64_t	trading_day	当前交易日(格式:YYYYMMDD)
int64_t	pre_trading_day	前一交易日(格式:YYYYMMDD)
int64_t	cash_component	前一日现金差额
int64_t	nav_per_cu	前一日最小申赎单位净值
int64_t	nav	前一日基金份额净值
uint8_t	market_type	证券所属市场(参考 MarketType)
char	symbol[ConstField::kSymbolETFLen]	基金名称(仅深圳有效)
char	fund_management_company[ConstField::kManagementETFLen]	基金公司名称(仅深圳有效)
char	underlying_security_id[ConstField::kSecurityCodeLen]	拟合指数代码(仅深圳有效)
char	underlying_security_id_source[ConstField::KUnderlyingSecurityIDSo]	拟合指数代码源(仅深圳有效)

	urce]	
int64_t	dividend_per_cu	红利金额
int64_t	creation_limit	累计申购总额限制,为 0 表示没有限制(仅深圳有效)
int64_t	redemption_limit	累计赎回总额限制,为 0 表示没有限制(仅深圳有效)
int64_t	creation_limit_per_user	单个账户累计申购总额限制,为 0 表示没有限制(仅深圳有效)
int64_t	redemption_limit_per_user	单个账户累计赎回总额限制,为 0 表示没有限制(仅深圳有效)
int64_t	net_creation_limit	净申购总额限制,为 0 表示没有限制(仅深圳有效)
int64_t	net_redemption_limit	净赎回总额限制,为 0 表示没有限制(仅深圳有效)
int64_t	net_creation_limit_per_user	单个账户净申购总额限制,为 0 表示没有限制(仅深圳有效)
int64_t	net_redemption_limit_per_user	单个账户净赎回总额限制,为 0 表示没有限制(仅深圳有效)
char	all_cash_flag	是否支持全现金申赎(暂时未启用,取值为空)
char	all_cash_amount[ConstField::AllCashAmount]	全现金替代的总金额(暂时未启用,取值为空)
char	all_cash_premium_rate[ConstField::AllCashAremiumRate]	全现金替代的申购溢价比例(暂时未启用,取值为空)
char	all_cash_discount_rate[ConstField::AllCashDiscountRate]	全现金替代的赎回折价比例(暂时未启用,取值为空)
char	rtgs_flag	是否支持 RTGS(暂时未启用,取值为空)
char	reserved[ConstField::KReserved]	预留字段(暂时未启用,取值为空)
std::vector<ConstituentStockInfo>	constituent_stock_infos	成分股信息

5.33ETF 成分股信息(ConstituentStockInfo)

数据类型	字段名称	说明
char	security_code[ConstField::kConsSecurityCodeLen]	成份证券代码
uint8_t	market_type	成份证券所属市场(仅深圳有效,参考MarketType)
char	underlying_symbol[ConstField::kSymbolETFLen]	成份证券简称
int64_t	component_share	成份证券数量

char	substitute_flag	现金替代标志 /**深圳现金替代标志* //0=禁止现金替代(必须有证券),1=可以进行现金替代(先用证券,证券不足时差额部分用现金替代),2=必须用现金替代 /**上海现金替代标志* //ETF 公告文件 1.0 版格式 //0 - 沪市不可被替代, 1 - 沪市可以被替代, 2 - 沪市必须被替代, 3 - 深市退补现金替代, 4 - 深市必须现金替代 //5 - 非沪深市场成份证券退补现金替代(不适用于跨沪深港 ETF 产品), 6 - 非沪深市场成份证券必须现金替代(不适用于跨沪深港 ETF 产品) //ETF 公告文件 2.1 版格式 //0 - 沪市不可被替代, 1 - 沪市可以被替代, 2 - 沪市必须被替代, 3 - 深市退补现金替代, 4 - 深市必须现金替代 //5 - 非沪深市场成份证券退补现金替代(不适用于跨沪深港 ETF 产品), 6 - 非沪深市场成份证券必须现金替代(不适用于跨沪深港 ETF 产品) //7 - 港市退补现金替代(仅适用于跨沪深港 ETF 产品), //8 - 港市必须现金替代(仅适用于跨沪深港 ETF 产品)
int64_t	premium_ratio	溢价比例
int64_t	discount_ratio	折价比例
int64_t	creation_cash_substitute	申购替代金额(仅深圳有效)
int64_t	redemption_cash_substitute	赎回替代金额(仅深圳有效)
int64_t	substitution_cash_amount	替代总金额(仅上海有效)
char	underlying_security_id[ConstField::KUnderlyingSecurityID]	成份证券所属市场 ID(仅对跨市场债券(银行间)ETF 启用)
char	buy_or_sell_to_open	期权期货买入开仓或卖出开仓(暂时未启用,取值为空)
char	reserved[ConstField::KReserved]	预留字段(暂时未启用,取值为空)

6. 补充字段取值说明

6.1 交易所定义证券类型表 security_type

深交所:

- 1 主板 A 股
- 2:中小板股票
- 3:创业板股票
- 4:主板 B 股
- 5:国债（含地方债）
- 6:企业债
- 7:公司债
- 8:可转债
- 9:私募债
- 10:可交换私募债
- 11:证券公司次级债
- 12:质押式回购
- 13:资产支持证券
- 14:本市场股票 ETF
- 15:跨市场股票 ETF
- 16:跨境 ETF
- 17:本市场实物债券 ETF
- 18:现金债券 ETF
- 19:黄金 ETF
- 20:货币 ETF
- 21:杠杆 ETF（预留）
- 22:商品期货 ETF
- 23:标准 LOF
- 24:分级子基金
- 25:封闭式基金
- 26:仅申赎基金
- 28:权证
- 29:个股期权
- 30:ETF 期权
- 33:优先股
- 34:证券公司短期债
- 35:可交换公司债
- 36:主板、中小板存托凭证
- 37:创业板存托凭证

港股:

- EQTY:股本
- TRST:信托
- WRNT:权证
- BOND:债券
- BWRT:一篮子权证

上交所：

GBF:国债

GBZ:无息国债

DST:国债分销（仅用于分销阶段）

DVP:公司债（地方债）分销

CBF:企业债券

CCF:可转换企业债券

CPF:公司债券（或地方债券）

FBF:金融机构发行债券

CRP:质押式国债回购

BRP:质押式企债回购

ORP:买断式债券回购

CBD:分离式可转债

OBD:其它债券

CEF:封闭式基金

OEF:开放式基金

EBS:交易所交易基金（买卖）

OFN:其它基金

ASH:以人民币交易的股票

BSH:以美元交易的股票

KSH:以人民币交易的股票（科创板）

CSH:国际版股票

OEQ:其它股票

CIW:企业发行权证

COV:备兑权证

FEQ:个股期货

FBD:债券期货

OFT:其它期货

AMP:集合资产管理计划

WIT:国债预发行

LOF:LOF 基金

OPS:公开发行优先股

PPS:非公开发行优先股

QRP:报价回购

CMD:控制指令（中登身份认证密码服务产品复用 CMD 证券子类别）

TCB 定向可转债

RET 公募 REITs

6.2 自定义证券子类别 security_type

品种证券子类别：security_type 取值

指数 1000

主板 A 股：2001

主板 B 股：2002

创业板股：2003

科创板股：2004

股转系统挂牌股：2005

优先股：2006

主板存托凭证：2007

创业板存托凭证：2008
 科创板存托凭证：2009
 其他股票：2999
 交易型开放式指数基金（ETF）：3001
 上市型开放式基金（LOF）：3002
 分级子基金：3003
 未上市开放基金（仅申赎）：3004
 封闭式基金：3005
 基础设施基金（REITs）：3006
 其他基金：3999
 可转债：4001
 国债（含地方债）：4002
 公司债：4003
 企业债：4004
 私募债：4005
 可交换私募债：4006
 证券公司次级债：4007
 证券公司短期债：4008
 其他债券：4999
 质押式回购：5001
 买断式债券回购：5002
 报价回购：5003
 个股认购期权：6001
 个股认沽期权：6002
 指数认购期权：6003
 指数认沽期权：6004
 商品认购期权：6005
 商品认沽期权：6006
 商品期货：6007
 指数期货：6008
 股票期货：6009
 其他期货（规划中）：6010
 权证：7001
 信托：7002
 港股股本：7003
 集合资产管理计划：99001
 资产支持证券：99002
 控制指令：99003

6.3 证券代码表的 currency 取值

CNY: 人民币
 HKD: 港币
 USD: 美元
 AUD: 澳元
 CAD: 加币
 JPY: 日圆
 SGD: 新加坡币
 GBP: 英镑

EUR: 欧元

6.4 交易阶段代码取值

*****上海现货快照交易状态*****

该字段为 8 位字符数组,左起每位表示特定的含义,无定义则填充格。

第 0 位: 'S'表示启动(开市前)时段,'C'表示开盘集合竞价时段,'T'表示连续交易时段,'E'表示闭市时段,'P'表示产品停牌。

第 1 位: '0'表示此产品不可正常交易,'1'表示此产品可正常交易。

第 2 位: '0'表示未上市,'1'表示已上市。

第 3 位: '0'表示此产品在当前时段不接受进行新订单申报,'1'表示此产品在当前时段可接受进行新订单申报。

*****深圳现货快照交易状态*****

第 0 位: 'S'= 启动(开市前)'O'= 开盘集合竞价'T'= 连续竞价'B'= 休市'C'= 收盘集合竞价'E'= 已闭市'H'= 临时停牌'A'= 盘后交易'V'=波动性中断。

第 1 位: '0'= 正常状态 '1'= 全天停牌。交易阶段代码

*****港股股票行情交易状态*****

'1'表示正常交易, '2'表示停牌, '3'表示复牌

*****港股指数快照交易状态*****

'C'=收市价 'T'=Indicative 指示 'O'=指数开盘 'P'=昨收价格 'R'=预备开盘 'S'=止损指数 'T'=指数现价

6.5 Security_status

1:停牌
2:除权
3:除息
4:ST
5:*ST
6:上市首日
7:公司再融资
8:恢复上市首日
9:网络投票
10:退市整理期
12:增发股份上市
13:合约调整
14:暂停上市后协议转让
15:实施双转单调整
16:特定债券转让
17:上市初期

上交所, 参考参考 IS101_上海证券交易所竞价撮合平台市场参与者接口规格说明书 1.47 版_20200703 .docx

该字段为 20 位字符串, 每位表示含义如下, 无定义则填充格。

该字段为 20 位字符串, 每位表示含义如下, 无定义则填充格。

第 1 位对应: 'N'表示首日上市。

第 2 位对应: 'D'表示除权。

第 3 位对应: 'R'表示除息。

第 4 位对应：‘D’表示国内主板正常交易产品，‘S’表示股票风险警示产品，‘P’表示退市整理产品，‘T’表示退市转让产品，‘U’表示优先股产品。

第 5 位不启用。

第 6 位对应：‘L’表示债券投资者适当性要求类，‘M’表示债券机构投资者适当性要求类。

第 7 位对应：‘F’表示 15:00 闭市的产品，对应 mkt00.txt 行情文件中市场行情状态第 4 位闭市标志，‘S’表示 15:30 闭市的产品，对应 mkt00.txt 行情文件中市场行情状态第 5 位闭市标志，为空表示非竞价撮合平台挂牌产品，无意义。

6.6 K 线算法说明

前推算法举例：9:31 的 1 分钟 K 线，计算的是 9:30:00.000~9:30:59.999 期间的 K 线。

9:40 的 5 分钟 K 线，计算的是 9:35:00.000~9:39:59.999 期间的 K 线。

6.7 商品期货查询规则

当查询非中金所（大商所、郑商所、上期所、上期能源）的商品期货快照时，因涉及夜盘快照，需根据查询时间参数做相应区分，查询上以 20:00 作为夜盘的分割时间点，处理逻辑见下表。

归属 T-1 日范围 20:00:00.000~23:59:59.999			归属 T 日范围：00:00:00.000~19:59:59.999
TGW 上送日期	开始时间	结束时间	系统响应逻辑
20220407	093000000	150000000	开始、结束时间均归属 T 日，且开始时间 < 结束时间，为有效查询，返回[4 月 7 日 9:30, 4 月 7 日 15:00]的数据
20220407	200000000	235900000	开始、结束时间均归属 T-1 日，且开始时间 < 结束时间，为有效查询，返回[4 月 6 日 20:00, 4 月 6 日 23:59]的数据
20220407	200000000	010000000	开始时间归属 T-1 日，结束时间归属 T 日，为有效查询，返回[4 月 6 日 20:00, 4 月 7 日 01:00]的数据
正常周一（未跨法定假节日）	235959999	030000000	开始时间归属 T-1 日，结束时间归属 T 日，为有效查询，返回[周五 23:59:59.999, 周一 03:00]的数据，需包括周末的数据（部分品种周六 0 点~02:30 会有行情）
特殊日（跨法定假节日）	200000000	010000000	开始时间归属 T-1 日，结束时间归属 T 日，为有效查询，返回[T-1 日 20:00, T 日 01:00]的数据
20220407	230000000	200000000	开始、结束时间均归属 T-1 日，但开始时间 > 结束时间，为无效查询，无数据返回，并需弹出相应告警
20220407	030000000	010000000	开始、结束时间均归属 T 日，但开始时间 > 结束时间，为无效查询，无数据返回，并需弹出相应告警

20220407	030000000	230000000	开始时间归属 T 日, 结束时间归属 T-1 日, 为无效查询, 无数据返回, 并需弹出相应告警
----------	-----------	-----------	--

7. 附件 1：资讯说明详细说明

7.1 股票基础信息

7.1.1A 股基本资料 A010010001

7.1.1.1 请求参数说明

参数	是否必填	参数说明	备注
function_id	是	功能号	功能号为 A010010001
market_code	是	市场代码	代码.市场, 例如: 688105.SH

7.1.1.2 返回值说明

字段名称	类型	字段说明	备注
MARKET_CODE	string	市场代码	
SECURITY_NAME	string	证券简称	
COMP_NAME	string	证券中文名称	
PINYIN	string	中文拼音简称	
COMP_NAME_ENG	string	证券英文名称	
LISTDATE	int32	上市日期	
DELISTDATE	int32	退市日期	
LISTPLATE_NAME	string	上市板块名称	
COMP_ID	string	公司代码	
COMP_SNAME_ENG	string	英文名称缩写	
IS_LISTED	int32	上市状态	1: 上市交易 3: 终止上市
TYPE_CODE	string	证券类别	参考证券类别代码表
TYPE_CODE_INT	int32	证券类别代码	参考证券类别代码表
EXCHANGE_STR	string	交易市场	参考交易市场代码表
EXCHANGE_INT	int32	交易市场代码	参考交易市场代码表

示例数据:

```
{
  "code": "0",
  "msg": "",
  "body": {
    "data": [
      {
        "MARKET_CODE": "601881.SH",
        "SECURITY_NAME": "中国银河",
        "COMP_NAME": "中国银河证券股份有限公司",
        "PINYIN": "zgyh",
        "COMP_NAME_ENG": "China Galaxy Securities Co., Ltd.",
        "LISTDATE": 20170123,

```

```

"DELISTDATE":null,
"LISTPLATE_NAME":"主板",
"COMP_ID":"604498",
"COMP_SNAME_ENG":null,
"IS_LISTED":null,
"TYPE_CODE":"A",
"TYPE_CODE_INT":1001,
"EXCHANGE_STR":"SSE",
"EXCHANGE_INT":101}}}}

```

7.1.2A 股行业分类 A010010002

7.1.2.1 请求参数说明

参数	是否必填	参数说明	备注
function_id	是	功能号	功能号为 A010010002
market_code	是	市场代码	代码.市场, 例如: 688105.SH

7.1.2.2 返回值说明

字段名称	类型	字段说明	备注
MARKET_CODE	string	市场代码	
INDUSTRIES_TYPE	int32	行业标准	10:万得行业分类 12:证监会行业分类 13:申万行业分类 14:中信行业分类
IN_DATE	int32	启用日期	
OUT_DATE	int32	剔除日期	
CURRENT_SIGN	int32	最新标志	1: 最新 0: 非最新
IND_CODE	int32	行业代码	
IND_NAME	string	行业名称	
IND_LEVEL	int32	所属行业层级	
IND_CODE_L1	string	对应一级行业代码	
IND_NAME_L1	string	对应一级行业名称	
IND_CODE_L2	string	对应二级行业代码	
IND_NAME_L2	string	对应二级行业名称	
IND_CODE_L3	string	对应三级行业代码	
IND_NAME_L3	string	对应三级行业名称	
SECURITY_NAME	string	证券简称	
COMP_ID	string	公司编码	
TYPE_CODE	string	证券类别	参考证券类别代码表

TYPE_CODE_INT	int32	证券类别代码	参考证券类别代码表
EXCHANGE_STR	string	交易市场	参考交易市场代码表
EXCHANGE_INT	int32	交易市场代码	参考交易市场代码表

示例数据：

```
{
  "code": "0",
  "msg": "",
  "body": {
    "data": [
      {
        "MARKET_CODE": "601881.SH",
        "INDUSTRIES_TYPE": 10,
        "IN_DATE": "20130522",
        "OUT_DATE": null,
        "CURRENT_SIGN": "1",
        "IND_CODE": "6240203020000000",
        "IND_NAME": "投资银行业与经纪业",
        "IND_LEVEL": 5,
        "IND_CODE_L1": "6240",
        "IND_NAME_L1": "金融",
        "IND_CODE_L2": "624020",
        "IND_NAME_L2": "多元金融",
        "IND_CODE_L3": "62402030",
        "IND_NAME_L3": "资本市场",
        "SECURITY_NAME": "中国银河",
        "COMP_ID": "604498",
        "TYPE_CODE": "A",
        "TYPE_CODE_INT": 1001,
        "EXCHANGE_STR": "SSE",
        "EXCHANGE_INT": 101
      }
    ]
  }
}
```

7.1.3 公司简介 A010010003

7.1.3.1 请求参数说明

参数	是否必填	参数说明	备注
function_id	是	功能号	功能号为 A010010003
market_code	是	市场代码	代码.市场，例如: 688105.SH

7.1.3.2 返回值说明

字段名称	类型	字段说明	备注
LISTDATE	int32	上市日期	
SECURITY_NAME	string	证券简称	
MARKET_CODE	string	市场代码	

COMP_WEBSITE	string	公司网址	
COMP_TOTAEMPLOYEES	int32	员工总数(人)	
COMP_REGCAPITAL	double	注册资本	
COMP_PROVINCE	string	省份	
COMP_PRESIDENT	string	总经理	
COMP_OFFICE	string	办公地址	
COMP_MAIN_BUSINESS	string	主要产品及业务	
COMP_FOUNDDATE	string	成立日期	
COMP_EMAIL	string	电子邮件	
COMP_COUNTRY	string	国籍	
COMP_COMPTYPE	string	公司类型名称	
COMP_COMP_TYPE	string	公司类别	1:一般公司;2:金融公司;3:基金公司
COMP_CITY	string	城市	
COMP_CHINESEINTRODUCTION	string	公司简介	
COMP_CHAIRMAN	string	法定代表人	
COMP_BUSINESSSCOPE	string	经营范围	
COMP_BDSECRETARY	string	董事会秘书	
ANN_DT	string	公告日期	
IS_VIE	string	是否 VIE(协议控制)	
IS_PROFIT	string	是否盈利(交易所)	
COMP_ID	string	公司 ID	
COMP_NAME	string	公司中文简称	
COMP_SNAME	string	公司名称	
COMP_NAME_ENG	string	英文名称缩写	
COMP_SNAME_ENG	string	英文名称	
ADDRESS	string	注册地址	
ZIPCODE	string	邮编	
PHONE	string	电话	
FAX	string	传真	
REGISTER_NUM	string	统一社会信用代码	
DISCLOSER	string	信息披露人	
CURRENCY_CODE	string	货币代码	
END_DATE	string	公司终止日期	
COMP_PROPERTY	string	企业性质	
ORG_CODE	string	组织机构代码	
IS_LISTED	int32	是否上市公司	1: 是 0: 否
COMPANY_TYPE	string	公司类型	参考公司类型代码表
TYPE_CODE	string	证券类别	参考证券类别代码表
TYPE_CODE_INT	int32	证券类别代码	参考证券类别代码表
EXCHANGE_STR	string	交易市场	参考交易市场代码表
EXCHANGE_INT	int32	交易市场代码	参考交易市场代码表

示例数据：

```
{
  "code": "0",
  "msg": "",
  "body": {
    "data": [
      {
        "LISTDATE": "20170123",
        "SECURITY_NAME": "中国银河",
        "MARKET_CODE": "601881.SH",
        "COMP_WEBSITE": "www.chinastock.com.cn",
        "COMP_TOTALEMPOYEEES": 12861,
        "COMP_REGCAPITAL": 1013728.3377,
        "COMP_PROVINCE": "北京",
        "COMP_PRESIDENT": "王晟",
        "COMP_OFFICE": "北京市丰台区西营街 8 号院 1 号楼青海金融大厦",
        "COMP_MAIN_BUSINESS": "本公司是我国证券行业领先的综合性金融服务提供商...",
        "COMP_FOUNDDATE": "20070126",
        "COMP_EMAIL": "Shengxing_bgs@chinastock.com.cn;yhzd@chinastock.com.cn;zgyh@china  
stock.com.cn",
        "COMP_COUNTRY": "中国",
        "COMP_COMPTYPE": "证券公司",
        "COMP_COMP_TYPE": "2",
        "COMP_CITY": "北京市",
        "COMP_CHINESEINTRODUCTION": "公司是中国证券行业领先的综合性金融服务提供商...",
        "COMP_CHAIRMAN": "陈亮",
        "COMP_BUSINESSSCOPE": "(一) 证券经纪; (二) 证券投资咨询; (三) ...",
        "COMP_BDSECRETARY": "杜鹏飞",
        "ANN_DT": "20230304",
        "IS_VIE": null,
        "IS_PROFIT": null,
        "COMP_ID": "604498",
        "COMP_NAME": "中国银河证券股份有限公司",
        "COMP_SNAME": "中国银河",
        "COMP_NAME_ENG": "China Galaxy Securities Co., Ltd.",
        "COMP_SNAME_ENG": "CGS",
        "ADDRESS": "北京市丰台区西营街 8 号院 1 号楼 7 至 18 层 101",
        "ZIPCODE": "100073",
        "PHONE": "86-10-80926716, 86-10-80929800, 86-10-80926608",
        "FAX": "86-10-80926725",
        "REGISTER_NUM": "91110000710934537G",
        "DISCLOSER": "杜鹏飞",
        "CURRENCY_CODE": "CNY",
        "END_DATE": null,
        "COMP_PROPERTY": "中央国有企业",
        "ORG_CODE": "710934537",
        "IS_LISTED": 1,
        "COMPANY_TYPE": "2",
      }
    ]
  }
}
```

```
"TYPE_CODE": "A",
"TYPE_CODE_INT": 1001,
"EXCHANGE_STR": "SSE",
"EXCHANGE_INT": 101}}}]}
```

7.1.4 股本结构 A010010004

7.1.4.1 请求参数说明

参数	是否必填	参数说明	备注
function_id	是	功能号	功能号为 A010010004
market_code	是	市场代码	代码.市场, 例如: 688105.SH
start_date	是	股本变动查询起始日期	填写 0 表示查询最新日期的数据
end_date	是	股本变动查询结束日期	填写 0 表示查询最新日期的数据

7.1.4.2 返回值说明

字段名称	类型	字段说明	备注
RTD_A_SHARE_INST	double	限售 A 股(其他内资持股:机构配售股)	
RTD_A_SHARE_DOMESNP	double	限售 A 股(其他内资持股:境内自然人持股)	
RTD_SHARE_SENIOR	double	限售股份(高管持股)(万股)	
RTD_A_SHARE_FOREIGN	double	限售 A 股(外资持股)	
RTD_A_SHARE_FORJUR	double	限售 A 股(境外法人持股)	
RTD_A_SHARE_FORNP	double	限售 A 股(境外自然人持股)	
RESTRICTED_B_SHARE	double	限售 B 股(万股)	
OTHER_RTD_SHARE	double	其他限售股	
NON_TRADABLE_SHARE	double	非流通股	
NTRD_SHARE_STATE_PCT	double	非流通股(国有股)	
NTRD_SHARE_STATE	double	非流通股(国家股)	
NTRD_SHARE_STATEJUR	double	非流通股(国有法人股)	
NTRD_SHARE_DOMESJUR	double	非流通股(境内法人股)	
NTRD_SHARE_DOMES_INITIATOR	double	非流通股(境内法人股:境内发起人股)	
NTRD_SHARE_IPOJURIS	double	非流通股(境内法人股:募集法人股)	
NTRD_SHARE_GENJURIS	double	非流通股(境内法人股:	

		一般法人股)	
NTRD_SHARE_STRA_INVESTOR	double	非流通股(境内法人股:战略投资者持股)	
NTRD_SHARE_FUND	double	非流通股(境内法人股:基金持股)	
NTRD_SHARE_NAT	double	非流通股(自然人股)	
TRAN_SHARE	double	转配股(万股)	
FLOAT_SHARE_SENIOR	double	流通股(高管持股)	
SHARE_INEMP	double	内部职工股(万股)	
PREFERRED_SHARE	double	优先股(万股)	
NTRD_SHARE_NLIST_FRGN	double	非流通股(非上市外资股)	
STAQ_SHARE	double	STAQ 股(万股)	
NET_SHARE	double	NET 股(万股)	
SHARE_CHANGE_REASON	string	股本变动原因	
TOT_A_SHARE	double	A 股合计	
TOT_B_SHARE	double	B 股合计	
OTCA_SHARE	double	三板 A 股	
OTCB_SHARE	double	三板 B 股	
TOT_OTC_SHARE	double	三板合计	
SHARE_HK	double	香港上市股	
TOT_TRADABLE_SHARE	double	流通股合计	
TOT_RESTRICTED_SHARE	double	限售股合计	
PRE_NON_TRADABLE_SHARE	double	股改前非流通股	
ANN_DATE	string	公告日期	
EX_CHANGE_DATE	string	除权日期	注: 股票分红送转股时的除权日;股票增发时的登记日
CURRENT_SIGN	int32	最新标志	
IS_VALID	int32	是否有效	
MARKET_CODE	string	市场代码	
CHANGE_DATE	string	变动日期	注: 股票分红送转股时的红股上市日;股票增发时的新股上市日
TOT_SHARE	double	总股本(万股)	
FLOAT_SHARE	double	流通股(万股)	
FLOAT_A_SHARE	double	流通 A 股(万股)	
FLOAT_B_SHARE	double	流通 B 股(万股)	
FLOAT_HK_SHARE	double	香港流通股(万股)	
FLOAT_OS_SHARE	double	海外流通股(万股)	
RESTRICTED_A_SHARE	double	限售 A 股(万股)	
RTD_A_SHARE_STATE	double	限售 A 股(国家持股)	
RTD_A_SHARE_STATEJUR	double	限售 A 股(国有法人持	

		股)	
RTD_A_SHARE_OTHER_DOMES	double	限售 A 股(其他内资持股)	
RTD_A_SHARE_OTHER_DOMES_JUR	double	限售 A 股(其他内资持股:境内法人持股)	
SECURITY_NAME	string	证券简称	
COMP_ID	string	公司编号	
SHARE_CHANGE_REASON_STR	string	股本变动原因描述	
TYPE_CODE	string	证券类别	参考证券类别代码表
TYPE_CODE_INT	int32	证券类别代码	参考证券类别代码表
EXCHANGE_STR	string	交易市场	参考交易市场代码表
EXCHANGE_INT	int32	交易市场代码	参考交易市场代码表

示例数据:

```
{
  "code": "0",
  "msg": "",
  "body": {
    "data": [
      {
        "RTD_A_SHARE_INST": 0,
        "RTD_A_SHARE_DOMESNP": 0,
        "RTD_SHARE_SENIOR": 0,
        "RTD_A_SHARE_FOREIGN": 0,
        "RTD_A_SHARE_FORJUR": 0,
        "RTD_A_SHARE_FORNP": 0,
        "RESTRICTED_B_SHARE": 0,
        "OTHER_RTD_SHARE": 0,
        "NON_TRADABLE_SHARE": 0,
        "NTRD_SHARE_STATE_PCT": 0,
        "NTRD_SHARE_STATE": 0,
        "NTRD_SHARE_STATEJUR": 0,
        "NTRD_SHARE_DOMESJUR": 0,
        "NTRD_SHARE_DOMES_INITIATOR": 0,
        "NTRD_SHARE_IPOJURIS": 0,
        "NTRD_SHARE_GENJURIS": 0,
        "NTRD_SHARE_STRA_INVESTOR": 0,
        "NTRD_SHARE_FUND": 0,
        "NTRD_SHARE_NAT": 0,
        "TRAN_SHARE": 0,
        "FLOAT_SHARE_SENIOR": 0,
        "SHARE_INEMP": 0,
        "PREFERRED_SHARE": 0,
        "NTRD_SHARE_NLIST_FRGN": 0,
        "STAQ_SHARE": 0,
        "NET_SHARE": 0,
        "SHARE_CHANGE_REASON": "KK",
        "TOT_A_SHARE": 644629.874399999971501529216766357421875,
        "TOT_B_SHARE": 0,
        "OTCA_SHARE": 0,
      }
    ]
  }
}
```

```

"OTCB_SHARE":0,
"TOT_OTC_SHARE":0,
"SHARE_HK":369098.4633000000030733644962310791015625,
"TOT_TRADABLE_SHARE":1013728.337700000003278255462646484375,
"TOT_RESTRICTED_SHARE":0,
"PRE_NON_TRADABLE_SHARE":0,
"ANN_DATE":"20230304",
"EX_CHANGE_DATE":"20230228",
"CURRENT_SIGN":1,
"IS_VALID":1,
"MARKET_CODE":"601881.SH",
"CHANGE_DATE":"20230228",
"TOT_SHARE":1013728.337700000003278255462646484375,
"FLOAT_SHARE":1013728.337700000003278255462646484375,
"FLOAT_A_SHARE":644629.874399999971501529216766357421875,
"FLOAT_B_SHARE":0,
"FLOAT_HK_SHARE":369098.46330000000030733644962310791015625,
"FLOAT_OS_SHARE":0,
"RESTRICTED_A_SHARE":0,
"RTD_A_SHARE_STATE":0,
"RTD_A_SHARE_STATEJUR":0,
"RTD_A_SHARE_OTHER_DOMES":0,
"RTD_A_SHARE_OTHER_DOMESJUR":0,
"SECURITY_NAME":"中国银河",
"COMP_ID":"604498",
"SHARE_CHANGE_REASON_STR":"转债转股",
"TYPE_CODE":"A",
"TYPE_CODE_INT":1001,
"EXCHANGE_STR":"SSE",
"EXCHANGE_INT":101}}}}

```

7.1.5 公司主营业务 A010010005

7.1.5.1 请求参数说明

参数	是否必填	参数说明	备注
function_id	是	功能号	功能号为 A010010005
market_code	是	市场代码	代码.市场, 例如: 688105.SH
start_date	是	公告期查询起始日期	填写 0 表示查询最新日期的数据
end_date	是	公告期查询结束日期	填写 0 表示查询最新日期的数据

7.1.5.2 返回值说明

字段名称	类型	字段说明	备注
MARKET_CODE	string	市场代码	
REPORT_PERIOD	string	报告期	
CURRENCY_CODE	string	货币代码	
SEGMENT_ITEM_CODE	int32	项目类别	1: 产品 2: 地区 3: 其他 4: 行业 5: 缴费方式
SEGMENT_ITEM	string	主营业务项目	
SEGMENT_SALES	double	主营业务收入(元)	
SEGMENT_PROFIT	double	主营业务利润(元)	
SEGMENT_COST	double	主营业务成本(元)	
IPUBLISHED_VALUE	int32	是否公布值	1: 是 : 否
PCT_SEGMENT_SALES	double	主营业务收入占比(%)	
PCT_SEGMENT_PROFIT	double	主营业务利润占比(%)	
PCT_SEGMENT_COST	double	主营业务成本占比(%)	
INC_SEGMENT_SALES	double	主营业务收入同比增长率(%)	
INC_SEGMENT_PROFIT	double	主营业务利润同比增长率(%)	
INC_SEGMENT_COST	double	主营业务成本同比增长率(%)	
GROSPROFIT_MARGIN	double	毛利率(%)	
INC_PROFIT_MARGIN	double	毛利率增长率(%)	
COMP_CODE	string	公司编码	
SECURITY_NAME	string	证券简称	
COMP_ID	string	公司 ID	
REPORT_TYPE	string	报告期名称	1: 一季度报表 2: 中期报表 3: 三季度报表 4: 年度报表
TYPE_CODE	string	证券类别	参考证券类别代码表
TYPE_CODE_INT	int32	证券类别代码	参考证券类别代码表
EXCHANGE_STR	string	交易市场	参考交易市场代码表
EXCHANGE_INT	int32	交易市场代码	参考交易市场代码表

示例数据:

```
{
  "code": "0",
  "msg": "",
  "body": {
    "data": [
      {
        "SEGMENT_ITEM_CODE": 1,
        "MARKET_CODE": "601881.SH",
        "REPORT_PERIOD": "20211231",
        "CURRENCY_CODE": "CNY",
```

```

"SEGMENT_ITEM": "证券经纪业务",
"SEGMENT_SALES": 12509319053.7,
"SEGMENT_PROFIT": 7641309097.49,
"SEGMENT_COST": 4868009956.21,
"IPUBLISHED_VALUE": 1,
"PCT_SEGMENT_SALES": 34.76,
"PCT_SEGMENT_PROFIT": 21.24,
"PCT_SEGMENT_COST": null,
"INC_SEGMENT_SALES": 16.4,
"INC_SEGMENT_PROFIT": 22.72,
"INC_SEGMENT_COST": 7.69,
"GROSPROFIT_MARGIN": 61.08,
"INC_PROFIT_MARGIN": 3.14,
"COMP_CODE": "604498",
"SECURITY_NAME": "中国银河",
"COMP_ID": "604498",
"TYPE_CODE": "A",
"REPORT_TYPE": "4",
"TYPE_CODE_INT": 1001,
"EXCHANGE_STR": "SSE",
"EXCHANGE_INT": 101}}}}

```

7.1.6 历史股票列表(仅京沪深 A 股) A010010006

7.1.6.1 请求参数说明

参数	是否必填	参数说明	备注
function_id	是	功能号	功能号为 A010010006
start_date	是	开始日期	格式为 yyyyymmdd 例如: 20220819
end_date	是	结束日期	格式为 yyyyymmdd 例如: 20220827

7.1.6.2 返回值说明

字段名称	类型	字段说明	备注
MARKET_CODE	string	市场代码	
SECURITY_NAME	string	证券简称	
CURRENCY_CODE	string	货币代码	
EXCHANGE	string	交易市场	参考交易市场代码表
EXCHANGE_INT	int32	交易市场代码	参考交易市场代码表

示例数据:

```
{
  "code": "0",
  "msg": "",
  "body": {
    "data": [
      {
        "MARKET_CODE": "000001.SZ",
        "SECURITY_NAME": "平安银行",
        "CURRENCY_CODE": "CNY",
        "EXCHANGE": "SZSE",
        "EXCHANGE_INT": 102
      }, {
        "MARKET_CODE": "000002.SZ",
        "SECURITY_NAME": "万科 A",
        "CURRENCY_CODE": "CNY",
        "EXCHANGE": "SZSE",
        "EXCHANGE_INT": 102
      }
    ]
  }
}
```

7.1.7 历史股票列表(含期货期权、指数) A010010007

7.1.7.1 请求参数说明

参数	是否必填	参数说明	备注
function_id	是	功能号	功能号为 A010010007
start_date	是	开始日期	格式为 yyyymmdd 例如: 20220819
end_date	是	结束日期	格式为 yyyymmdd 例如: 20220827

7.1.7.2 返回值说明

字段名称	类型	字段说明	备注
MARKET_CODE	string	市场代码	
SECURITY_NAME	string	证券简称	
CURRENCY_CODE	string	货币代码	
EXCHANGE	string	交易市场	参考交易市场代码表
EXCHANGE_INT	int32	交易市场代码	参考交易市场代码表

示例数据:

```
{
  "code": "0",
  "msg": "",
  "body": {
    "data": [
      {
        "MARKET_CODE": "AG2308.SHF",
        "VARIETYCATEGORY": 7,
        "SECURITY_NAME": "沪银 2308",
        "CURRENCY_CODE": "CNY",
        "EXCHANGE": "SHFE",
        "EXCHANGE_INT": 3
      }, {
        "MARKET_CODE": "AG2309.SHF",
        "VARIETYCATEGORY": 7,
        "SECURITY_NAME": "沪银 2309",

```

"CURRENCY_CODE": "CNY",
 "EXCHANGE": "SHFE", "EXCHANGE_INT": 3}]]}

7.2 发行与上市

7.2.1A 股首次公开发行 A010020001

7.2.1.1 请求参数说明

参数	是否必填	参数说明	备注
function_id	是	功能号	功能号为 A010020001
market_code	是	市场代码	代码.市场, 例如: 688105.SH

7.2.1.2 返回值说明

字段名称	类型	字段说明	备注
MARKET_CODE	string	市场代码	
CURRENCY_CODE	string	货币代码	
PRICE	double	发行价格(元)	网上申购价格
PRE_DILUTED_PE	double	发行市盈率(发行前股本)	
DILUTED_PE	double	发行市盈率(发行后股本)	
AMOUNT	double	公开及老股发行数量合计(万股)	
AMT_BY_PLACING	double	网上发行数量(万股)	
AMT_TO_JUR	double	网下发行数量(万股)	
COLLECTION	double	募集资金(万元)	含发行费用
CASH_RATIO	double	网上发行中签率(%)	
PURCHASE_CODE	string	网上申购代码	
SUBS_DATE	string	申购日	
JURIS_DATE	string	向一般法人配售上市日期	
INSTIS_DATE	string	向战略投资者配售部分上市日期	
EXPECT_LIST_DATE	string	预计上市日期	
FUND_VERIF_DATE	string	申购资金验资日	
RATIO_DATE	string	中签率公布日	
UNFROZE_DATE	string	网上中签结果公告日	
LIST_DATE	string	上市日	
PUB_OFFR_DATE	string	招股公告日	
ANN_DATE	string	发行公告日	
ANN_LST_DATE	string	上市公告日	

ROAD_SHOW_START_DATE	string	初步询价(预路演)起始日期	
ROAD_SHOW_END_DATE	string	初步询价(预路演)终止日期	
PLACING_DATE	string	网下配售发行公告日	
APPLY_START_DATE	string	网下申购起始日期	
APPLY_END_DATE	string	网下申购截止日期	
PRICE_ANN_DATE	string	网下定价公告日	
PLACING_RESULT_DATE	string	网下配售结果公告日	
FUND_END_DATE	string	网下申购资金到帐截止日	
CAP_VERIF_DATE	string	网下验资日	
REFUND_DATE	string	网下多余款项退还日	
EXPECTED_COLLECTION	double	预计募集资金(万元)	
LIST_FEE	double	发行费用(万元)	
CASH_AMT_UPLIMIT	double	申购上限(机构)	
CASH_MONEY_UPLIMIT	double	申购金额上限(机构)	
NAME_BY_PLACING	string	网上发行简称	
SHOW_PRICE_DOWNLIMIT	double	投标询价申购价格下限	
PAR	double	面值	
PURCHASE_UPLIMIT	double	网上申购上限(个人)(股)	
OP_UPLIMIT	double	网下申购上限(万股)	
OP_DOWNLIMIT	double	网下申购下限(万股)	
PURCHASE_MV_DT	string	网上市值申购登记日	
PUBOSDTOTISSCOLL	double	公开及原股东募集资金总额	
OSD_EXPOFF_AMT	double	新股发行数量	
OSD_EXPOFF_AMT_UP	double	原股东预计售股数量上限	
OSD_ACTOFF_AMT	double	原股东实际售股数量	
OSD_ACTOFF_PRICE	double	原股东实际售股金额	
OSD_UNDERWRITING_FEES	double	原股东应摊承销费用	
PUREFF_SUBS_RATIO	double	网上投资者有效认购倍数	
REPO_RATE	double	回拨比例	>0:网下往网上 <0:网上往网下 占本次发行数量合计的比例
ANN_DT	string	最新公告日期	
IS_FAILURE	double	是否发行失败	0:发行正常;1: 发行失败;2:发 行暂缓
OTC_CASH_PCT	double	网下申购配售比例	网下中签率
MIN_APPLY_UNIT	double	最低累进申购数量(万股)	
MV_CALCULATION_DATE	string	市值计算参考日	
MV_THRESHOLD	double	网下询价市值门槛	
PAYMENT_DATE	string	网上申购缴款日	
ISSUE_TYPE	string	发行方式	
VAL_PE	double	行业PE	

NEW_FACE_VALUE	double	最新每股面值	
MV_THRESHOLD_A	double	网下询价市值门槛(A类)(万元)	
UNDERWRITING_WAY	string	承销方式	
ONLINE_EFFECTIVE_NUM	int64	网上发行有效申购户数	
ONLINE_EFFECTIVE_QTY	int64	网上发行有效申购数量	
OVRSUBS_RATIO	double	超额认购倍数	
EFFECTIVE_QUOTE_OVRSUBS_RATIO	double	初步询价有效报价对应的超额认购倍数	
UNDER_SUBS_RATIO	double	网下超额认购倍数	
APPLICATION_DEADLINE	string	网下报备截止日	
APPLICATION_DEADLINE_TIME	string	网下报备截止时间	
STRATEGY_PLACING_QTY	int64	向战略投资者配售数量(万股)	
OFFLINE_RELEASES_NUM	int64	网下发行数量	
MV_THRESHOLD_STIB	double	网下询价市值门槛(科创主题与战略)(万元)	
COMMISSION_RATE	double	新股配售经纪佣金费率	
PLACING_RESULT_ANN_DATE	string	初步配售结果公告日	
BALANCE_SUBS_QTY	double	承销商余额认购数量(万股)	
PRELIMINARY_INQ_DOWN	double	初步询价报价结果下限	
PRELIMINARY_INQ_UP	double	初步询价报价结果上限	
EXCLUDE_HIGHEST_BID_LIMIT		剔除的最高报价下限(含)	
EXCLUDE_HIGHEST_BID_OFFER		剔除的最高报价的申购量	
PREL_INQ_ANN_DATE	string	初步询价公告日	
PREL_INQ_RESULT_ANN_DATE	string	初步询价结果公告日	
IS_ADDUP_SHOW_PRICE	double	是否进行累计投标询价	1:是 : 否
SECURITY_NAME	string	证券简称	
COMP_ID	string	公司编码	
TYPE_CODE	string	证券类别	参考证券类别代码表
TYPE_CODE_INT	int32	证券类别代码	参考证券类别代码表
EXCHANGE_STR	string	交易市场	参考交易市场代码表
EXCHANGE_INT	int32	交易市场代码	参考交易市场代码表

示例数据:

```
{
  "code": "0",
  "msg": "",
  "body": {
    "data": [
      {
        "MARKET_CODE": "601881.SH",
        "CURRENCY_CODE": "CNY",
        "PRICE": 6.81,

```

"PRE_DILUTED_PE":6.61,
"DILUTED_PE":7.02,
"AMOUNT":60000,
"AMT_BY_PLACING":54000,
"AMT_TO_JUR":6000,
"COLLECTION":408600,
"CASH_RATIO":0.2081083800000000094687635510126710869371891021728515625,
"PURCHASE_CODE":"780881",
"SUBS_DATE":"20170111",
"JURIS_DATE":"20170123",
"INSTIS_DATE":null,
"EXPECT_LIST_DATE":null,
"FUND_VERIF_DATE":null,
"RATIO_DATE":"20170112",
"UNFROZE_DATE":"20170113",
"LIST_DATE":"20170123",
"PUB_OFFR_DATE":"20161226",
"ANN_DATE":"20170110",
"ANN_LST_DATE":"20170120",
"ROAD_SHOW_START_DATE":"20170105",
"ROAD_SHOW_END_DATE":"20170106",
"PLACING_DATE":"20170110",
"APPLY_START_DATE":"20170111",
"APPLY_END_DATE":"20170111",
"PRICE_ANN_DATE":"20170110",
"PLACING_RESULT_DATE":"20170117",
"FUND_END_DATE":"20170113",
"CAP_VERIF_DATE":null,
"REFUND_DATE":null,
"EXPECTED_COLLECTION":408600,
"LIST_FEE":13178.6,
"CASH_AMT_UPLIMIT":180000,
"CASH_MONEY_UPLIMIT":1225800,
"NAME_BY_PLACING":"银河申购",
"SHOW_PRICE_DOWNLIMIT":null,
"PAR":1,
"PURCHASE_UPLIMIT":180000,
"OP_UPLIMIT":4200,
"OP_DOWNLIMIT":500,
"PURCHASE_MV_DT":"20170109",
"PUBOSDTOTISSCOLL":4086000000,
"OSD_EXPOFF_AMT":60000,
"OSD_EXPOFF_AMT_UP":0,
"OSD_ACTOFF_AMT":null,

```
"OSD_ACTOFF_PRICE":null,
"OSD_UNDERWRITING_FEES":null,
"PUREFF_SUBS_RATIO":null,
"REPO_RATE":60,
"ANN_DT":"20170120",
"IS_FAILURE":0,
"OTC_CASH_PCT":0.04670984000000000246899389821919612586498260498046875,
"MIN_APPLY_UNIT":10,
"MV_CALCULATION_DATE":"20170103",
"MV_THRESHOLD":5000,
"PAYMENT_DATE":"20170113",
"ISSUE_TYPE":"网下询价,网上定价",
"VAL_PE":12.37,
"NEW_FACE_VALUE":1,
"MV_THRESHOLD_A":null,
"UNDERWRITING_WAY":"余额包销",
"ONLINE_EFFECTIVE_NUM":13979359,
"ONLINE_EFFECTIVE_QTY":259480179000,
"OVRSUBS_RATIO":480.5189,
"EFFECTIVE_QUOTE_OVRSUBS_RATIO":306.8395,
"UNDER_SUBS_RATIO":2140.8767,
"APPLICATION_DEADLINE":null,
"APPLICATION_DEADLINE_TIME":null,
"STRATEGY_PLACING_QTY":null,
"OFFLINE_RELEASES_NUM":6000,
"MV_THRESHOLD_STIB":null,
"COMMISSION_RATE":null,
"PLACING_RESULT_ANN_DATE":"20170113",
"BALANCE_SUBS_QTY":108.3898,
"PRELIMINARY_INQ_DOWN":6.81,
"PRELIMINARY_INQ_UP":6.81,
"EXCLUDE_HIGHEST_BID_LIMIT":10.44,
"EXCLUDE_HIGHEST_BID_OFFER":4200,
"PREL_INQ_ANN_DATE":"20161226",
"PREL_INQ_RESULT_ANN_DATE":"20170110",
"IS_ADDUP_SHOW_PRICE":0,
"SECURITY_NAME":"中国银河",
"COMP_ID":"604498",
"TYPE_CODE":"A",
"TYPE_CODE_INT":1001,
"EXCHANGE_STR":"SSE",
"EXCHANGE_INT":101}}}}
```

7.2.2A 股增发数据 A010020002

7.2.2.1 请求参数说明

参数	是否必填	参数说明	备注
function_id	是	功能号	功能号为 A010020002
market_code	是	市场代码	代码.市场, 例如: 688105.SH

7.2.2.2 返回值说明

字段名称	类型	字段说明	备注
SEO_HOLDER_SUBS_MODE	string	大股东认购方式	
PRICING_MODE	int32	定向增发定价方式代码	1: 竞价 2: 定价
ISSUE_TYPE	int32	发行方式	发行方式类型 1:定向 2:公开
MARKET_CODE	string	市场代码	
SEO_PROGRESS	int32	方案进度	参考股票增发进度代码表
CURRENCY_CODE	string	货币代码	
SEO_PRICE	double	发行价格(元)	
SEO_AMOUNT	double	发行数量(万股)	
COLLECTION	double	募集资金(元)	含发行费用
RECORD_DATE	string	股权登记日	
PAY_START_DATE	string	缴款起始日期	
PAY_END_DATE	string	缴款截止日期	
SUBS_DATE	string	网上申购日	
OTC_DATE	string	网下发行日期	
LIST_DATE	string	向公众增发股份上市日期	
INST_LIST_DATE	string	向机构增发股份上市日期	定向增发部分流通日期
CHANGE_DATE	string	定向增发股份变动日期	股份实际派发日
ROAD_SHOW_DATE	string	网上路演日	
REFUND_DATE	string	网下申购定金退款日或补缴余款日	
UNFROZE_DATE	string	网上申购资金解冻日	
PREPLAN_DATE	string	预案公告日	
SMTG_ANNCNCE_DATE	string	股东大会公告日	
PASS_DATE	string	发审委/上市委通过公告日	
APPROVED_DATE	string	证监会通过公告日	
ANNCNCE_DATE	string	上网发行公告日	

RATIO_ANNCE_DATE	string	网上中签率公告日	
OFFERING_DATE	string	增发公告日	
LIST_ANN_DATE	string	上市公告日	
OFFERING_TARGET	string	发行对象	
PRICE_UPLIMIT	double	增发预案价上限	
PRICE_DOWNLIMIT	double	增发预案价下限	
SEO_CODE	string	增发代码	
SEO_NAME	string	增发简称	
SEO_PE	double	发行市盈率(摊薄)	
SEO_AMT_BY_PLACING	double	上网发行数量(万股)	
SEO_AMT_TO_JUR	double	网下发行数量(万股)	
SEO_HOLDER_SUBS_RATE	double	大股东认购比例(%)	
ANN_DT	string	最新公告日期	
ORGPRIE_MIN	double	原始预案价下限	
DISCNT_RATIO	double	折扣率	(增发日前一日的收盘价-增发价)/增发日前一日的收盘价*100
SEO_DATE	string	定增发行日期	
SUBS_INVITATION_DATE	string	认购邀请书日	
SEO_YEAR	string	增发年度	
OBJECTIVE_CODE	string	定向增发目的代码	
PRICING_DATE	string	定价基准日	
IS_NO_PUBLIC	int32	是否属于非公开发行	0: 否 1: 是
EXPENSE	double	发行费用(元)	
EX_DIVIDEND_DATE	string	除权日	
IS_EX_DIVIDEND	int32	是否除权	
SUBS_MODE	int32	定增认购方式代码	1: 现金 2: 资产 3: 债权 4: 现金, 资产 5: 现金, 债权 6: 资产, 债权 7: 现金, 资产, 债权
OLDSPLA_MOLE	double	老股东配售比例分子	
OLDSPLA_MODE	double	老股东配售代码	
EXP_COLLECTION	int64	预计募集资金(元)	
SEO_TYPE	string	增发类型	
OBJECTIVE	string	增发目的说明	
OFFERING_TARGET_DES	string	发行对象说明	
EVENT_ID	string	事件 ID	
PRICE_CONDITION	string	价格制定依据	
PRICE_DT_TYPE	string	定价基准日类型	
ORG_PRICE_PCT	double	预案价格相对基准价格比例	

		(%)	
ACT_PRICE_PCT	double	实施价格相对基准价格比例 (%)	
APPRVBYSASSC_DT	string	国资委批准公告日期	
INITIAL_PLAN	string	初始预案公告日	
LEAD_UNDERWRITER	string	主承销商	
CURRENCY_SUBS_NUM	double	货币认购数量(万股)	
NON_MONETARY_SUBS_NUM	double	非货币认购数量(万股)	
CURRENCY_SUBS_AMOUNT	double	货币认购金额(元)	
NOCURRENCY_SUBS_AMOUNT	double	非货币认购金额(元)	
IS_CONTAIN_HOLDER	int32	发行对象是否包含控股股东	
OLDSPLA_DENOMINATOR	double	老股东配售比例分母	
NUM_SHARES_PREAONLINE	double	网上向老股东优先配售数量 (万股)	
SHARE_RECORD_DATE	string	股份登记日 (定向)	
ADD_SUBS_STARTDATE	string	追加认购起始日期	
ADD_SUBS_ENDDATE	string	追加认购截止日期	
FUND_VERIFICATION_DATE	string	验资日期	
PRIVATE_PLACEMENT_SHARENUM	double	定向配售数量(万股)	
SECURITY_NAME	string	证券简称	
COMP_ID	string	公司编码	
TYPE_NAME	string	定向增发目的类型描述	
CLASSIFICATION	string	定向增发目的分类	
TYPE_CODE	string	证券类别	参考证券类别代码表
TYPE_CODE_INT	int32	证券类别代码	参考证券类别代码表
EXCHANGE_STR	string	交易市场	参考交易市场代码表
EXCHANGE_INT	int32	交易市场代码	参考交易市场代码表

示例数据:

```
{
  "code": "0",
  "msg": "",
  "body": {
    "data": [
      {
        "SEO_HOLDER_SUBS_MODE": "现金",
        "SUBS_MODE": 1,
        "PRICING_MODE": 1,
        "ISSUE_TYPE": 1,
        "MARKET_CODE": "000050.SZ",
        "SEO_PROGRESS": 3,
        "CURRENCY_CODE": "CNY",
        "SEO_PRICE": 13.66,
        "SEO_AMOUNT": 40962.46,
      }
    ]
  }
}
```

"COLLECTION":5595472400,
"RECORD_DATE":null,
"PAY_START_DATE":null,
"PAY_END_DATE":null,
"SUBS_DATE":null,
"OTC_DATE":20200824,
"LIST_DATE":null,
"INST_LIST_DATE":20210311,
"CHANGE_DATE":20200910,
"ROAD_SHOW_DATE":null,
"REFUND_DATE":null,
"UNFROZE_DATE":null,
"PREPLAN_DATE":20190829,
"SMTG_ANNCE_DATE":20191203,
"PASS_DATE":20200516,
"APPROVED_DATE":20200606,
"ANNCE_DATE":null,
"RATIO_ANNCE_DATE":null,
"OFFERING_DATE":20200908,
"LIST_ANN_DATE":20200908,
"OFFERING_TARGET":"大股东, 大股东关联方, 机构投资者",
"PRICE_UPLIMIT":null,
"PRICE_DOWNLIMIT":13.66,
"SEO_CODE":null,
"SEO_NAME":null,
"SEO_PE":null,
"SEO_AMT_BY_PLACING":null,
"SEO_AMT_TO_JUR":null,
"SEO_HOLDER_SUBS_RATE":17.3188,
"ANN_DT":20200908,
"ORGPRICE_MIN":13.66,
"DISCNT_RATIO":15.52,
"SEO_DATE":20200812,
"SUBS_INVITATION_DATE":20200805,
"SEO_YEAR":20201231,
"OBJECTIVE_CODE":243000001,
"PRICING_DATE":20200807,
"IS_NO_PUBLIC":1,
"EXPENSE":32489404,
"EX_DIVIDEND_DATE":null,
"IS_EX_DIVIDEND":0,
"OLDSPLA_MOLE":null,
"OLDSPLA_MODE":null,
"EXP_COLLECTION":7299999700,

"SEO_TYPE": "原有 A 股增发 A 股",
 "OBJECTIVE": "本次非公开发行 A 股股票拟募集资金总额不超过 730,000 万元...",
 "EVENT_ID": "190829A03",
 "PRICE_CONDITION": "本次非公开发行股票采取询价方式...",
 "PRICE_DT_TYPE": "发行期首日",
 "ORG_PRICE_PCT": 80,
 "ACT_PRICE_PCT": 80,
 "APPRVBYSASSC_DT": 20191129,
 "INITIAL_PLAN": 20190829,
 "LEAD_UNDERWRITER": null,
 "CURRENCY_SUBS_NUM": 40962.46,
 "NON_MONETARY_SUBS_NUM": null,
 "CURRENCY_SUBS_AMOUNT": 5595472400,
 "NOCURRENCY_SUBS_AMOUNT": null,
 "IS_CONTAIN_HOLDER": 0,
 "OLDSPLA_DENOMINATOR": null,
 "NUM_SHARES_PREAONLINE": null,
 "SHARE_RECORD_DATE": 20200824,
 "ADD_SUBS_STARTDATE": null,
 "ADD_SUBS_ENDDATE": null,
 "FUND_VERIFICATION_DATE": null,
 "PRIVATE_PLACEMENT_SHARENUM": 40962.46,
 "SECURITY_NAME": "深天马 A",
 "COMP_ID": "1000050",
 "TYPE_NAME": "项目融资",
 "CLASSIFICATION": "定向增发目的",
 "TYPE_CODE": "A",
 "TYPE_CODE_INT": 1001,
 "EXCHANGE_STR": "SZSE",
 "EXCHANGE_INT": 102}}}}

7.3 收益分配

7.3.1A 股股票分红 A010030001

7.3.1.1 请求参数说明

参数	是否必填	参数说明	备注
function_id	是	功能号	功能号为 A010030001
market_code	是	市场代码	代码.市场, 例如: 688105.SH
start_date	是	分红年度查询起始日期	填写 0 表示查询最新日期的数据

end_date	是	分红年度查询结束日期	填写 0 表示查询最新日期的数据
----------	---	------------	------------------

7.3.1.2 返回值说明

字段名称	类型	字段说明	备注
MARKET_CODE	string	市场代码	
DIV_PROGRESS	string	方案进度	参看股票分红进度代码表
DVD_PER_SHARE_STK	double	每股送转	
DVD_PER_SHARE_PRE_TAX_CASH	double	每股派息 (税前) (元)	
DVD_PER_SHARE_AFTER_TAX_CASH	double	每股派息 (税后) (元)	
DATE_EQY_RECORD	string	股权登记日	
DATE_EX	string	除权除息日	
DATE_DVD_PAYOUT	string	派息日	
LISTINGDATE_OF_DVD_SHR	string	红股上市日	
DIV_PRELANDATE	string	预案公告日	董事会预案公告日期
DIV_SMTGDATE	string	股东大会公告日	
DATE_DVD_ANN	string	分红实施公告日	
DIV_BASEDATE	string	基准日期	
DIV_BASESHARE	double	基准股本(万股)	
CURRENCY_CODE	string	货币代码	
ANN_DATE	string	最新公告日期	
IS_CHANGED	int32	方案是否变更	1: 有变更过 0: 未变更
REPORT_PERIOD	string	分红年度	
DIV_CHANGE	string	方案变更说明	
DIV_BONUSRATE	double	每股送股比例	
DIV_CONVERSED RATE	double	每股转增比例	
REMARK	string	备注	
DIV_PREANN_DATE	string	预案预披露公告日	股东提议的公告日期
DIV_TARGET	string	分红对象	
COMP_ID	string	公司编码	
SECURITY_NAME	string	证券简称	
TYPE_CODE	string	证券类别	参考证券类别代码表
TYPE_CODE_INT	int32	证券类别代码	参考证券类别代码表
EXCHANGE_STR	string	交易市场	参考交易市场代码表
EXCHANGE_INT	int32	交易市场代码	参考交易市场代码表

示例数据:

```
{
  "code": "0",
  "msg": "",
  "body": {
    "data": [
      {
        "MARKET_CODE": "601881.SH",
        "DIV_PROGRESS": "1",
        "DVD_PER_SHARE_STK": 0,

```

```

"DVD_PER_SHARE_PRE_TAX_CASH":0,
"DVD_PER_SHARE_AFTER_TAX_CASH":0,
"DATE_EQY_RECORD":null,
"DATE_EX":null,
"DATE_DVD_PAYOUT":null,
"LISTINGDATE_OF_DVD_SHR":null,
"DIV_PRELANDATE":"20220831",
"DIV_SMTGDATE":null,
"DATE_DVD_ANN":null,
"DIV_BASEDATE":null,
"DIV_BASESHARE":null,
"CURRENCY_CODE":"CNY",
"ANN_DATE":"20220831",
"IS_CHANGED":0,
"REPORT_PERIOD":"20220630",
"DIV_CHANGE":null,
"DIV_BONUSRATE":null,
"DIV_CONVERSEDRATE":null,
"REMARK":null,
"DIV_PREANN_DATE":null,
"DIV_TARGET":null,
"COMP_ID":"604498",
"SECURITY_NAME":"中国银河",
"TYPE_CODE":"A",
"TYPE_CODE_INT":1001,
"EXCHANGE_STR":"SSE",
"EXCHANGE_INT":101}}}}

```

7.3.2A 股股票配股 A010030002

7.3.2.1 请求参数说明

参数	是否必填	参数说明	备注
function_id	是	功能号	功能号为 A010030002
market_code	是	市场代码	代码.市场, 例如: 688105.SH
start_date	是	公告期查询起始日期	填写 0 表示查询最新日期的数据
end_date	是	公告期查询结束日期	填写 0 表示查询最新日期的数据

7.3.2.2 返回值说明

字段名称	类型	字段说明	备注
------	----	------	----

MARKET_CODE	string	市场代码	
PROGRESS	int32	方案进度	参看股票配股进度代码表
PRICE	double	配股价格(元)	
RATIO	double	配股比例	
AMT_PLAN	double	配股计划数量(万股)	
AMT_REAL	double	配股实际数量(万股)	
COLLECTION_FUND	double	募集资金(元)	
SHAREB_REG_DATE	string	股权登记日	
EX_DIVIDEND_DATE	string	除权日	
LISTED_DATE	string	配股上市日	
PAY_START_DATE	string	缴款起始日	
PAY_END_DATE	string	缴款终止日	
PREPLAN_DATE	string	预案公告日	
SMTG_ANN_DATE	string	股东大会公告日	
PASS_DATE	string	发审委通过公告日	
APPROVED_DATE	string	证监会核准公告日	
EXECUTE_DATE	string	配股实施公告日	
RESULT_DATE	string	配股结果公告日	
LIST_ANN_DATE	string	上市公告日	
GUARANTOR	string	基准年度	
GUARTYPE	double	基准股本(万股)	
RIGHTSISSUE_CODE	string	配售代码	
ANN_DATE	string	最新公告日期	
RIGHTSISSUE_YEAR	string	配股年度	
RIGHTSISSUE_DESC	string	配股说明	
RIGHTSISSUE_NAME	string	配股简称	
RATIO_DENOMINATOR	double	配股比例分母	
RATIO_MOLECULAR	double	配股比例分子	
SUBS_METHOD	string	认购方式	
EXPECTED_FUND_RAISING	double	预计募集资金(元)	
COMP_ID	string	公司编码	
SECURITY_NAME	string	证券简称	
TYPE_CODE	string	证券类别	参考证券类别代码表
TYPE_CODE_INT	int32	证券类别代码	参考证券类别代码表
EXCHANGE_STR	string	交易市场	参考交易市场代码表
EXCHANGE_INT	int32	交易市场代码	参考交易市场代码表

示例数据：

```
{
  "code": "0",
  "msg": "",
  "body": {
    "data": [
      {
        "MARKET_CODE": "000050.SZ",
        "PROGRESS": "3",
        "PRICE": 5.5,
        "RATIO": 0.2727,
        "AMT_PLAN": 2491.4336,
        "AMT_REAL": 2491.5,
        "COLLECTION_FUND": 137032500,
        "SHAREB_REG_DATE": "19980330",
        "EX_DIVIDEND_DATE": "19980331",
        "LISTED_DATE": "19980525",
        "PAY_START_DATE": "19980402",
        "PAY_END_DATE": "19980415",
        "PREPLAN_DATE": "19970826",
        "SMTG_ANN_DATE": "19970927",
        "PASS_DATE": "19980225",
        "APPROVED_DATE": null,
        "EXECUTE_DATE": "19980314",
        "RESULT_DATE": "19980331",
        "LIST_ANN_DATE": "19980516",
        "GUARANTOR": "19971231",
        "GUARTYPE": 9135.5,
        "RIGHTSISSUE_CODE": "8050",
        "ANN_DATE": "19980516",
        "RIGHTSISSUE_YEAR": "19971231",
        "RIGHTSISSUE_DESC": "1997 年度配股已实施：每 10 股配 2.7272 股，配股价 5.5 元",
        "RIGHTSISSUE_NAME": "天马 A1 配",
        "RATIO_DENOMINATOR": 10,
        "RATIO_MOLECULAR": 2.7272,
        "SUBS_METHOD": "货币方式",
        "EXPECTED_FUND_RAISING": null,
        "COMP_ID": "1000050",
        "SECURITY_NAME": "深天马 A",
        "TYPE_CODE": "A",
        "TYPE_CODE_INT": 1001,
        "EXCHANGE_STR": "SZSE",
        "EXCHANGE_INT": 102
      }
    ]
  }
}
```

7.3.3A 股股票除权除息 A010030003

7.3.3.1 请求参数说明

参数	是否必填	参数说明	备注
function_id	是	功能号	功能号为 A010030003
market_code	是	市场代码	代码.市场, 例如: 688105.SH
start_date	是	除权除息日查询起始日期	填写 0 表示查询最新日期的数据
end_date	是	除权除息日查询结束日期	填写 0 表示查询最新日期的数据

7.3.3.2 返回值说明

字段名称	类型	字段说明	备注
SEO_RATIO	double	增发比例	
REDUCED_RATIO	double	缩减比例	
MARKET_CODE	string	市场代码	
EX_RD_DATE	string	除权除息日	
EX_RD_TYPE	string	除权类型	
EX_RD_DESCRIPTION	string	除权说明	
DIV_PAYOUT_RATIO	double	派息比例	
BONUS_SHARE_RATIO	double	送股比例	
RIGHT_ISSUE_RATIO	double	配股比例	
RIGHT_ISSUE_PRICE	double	配股价格	
CONVER_INCR_RATIO	double	转增比例	
SEO_PRICE	double	增发价格	
COMP_ID	string	公司编码	
SECURITY_NAME	string	证券简称	
TYPE_CODE	string	证券类别	参考证券类别代码表
TYPE_CODE_INT	int32	证券类别代码	参考证券类别代码表
EXCHANGE_STR	string	交易市场	参考交易市场代码表
EXCHANGE_INT	int32	交易市场代码	参考交易市场代码表

示例数据:

```
{
  "code": "0",
  "msg": "",
  "body": {
    "data": [
      {
        "SEO_RATIO": 0,
        "REDUCED_RATIO": null,
        "MARKET_CODE": "601881.SH",
        "EX_RD_DATE": "20210715",
        "EX_RD_TYPE": "分红",
        "EX_RD_DESCRIPTION": "[20201231]每 10 股派 2.2 元(含税)",
        "DIV_PAYOUT_RATIO": 0.22,
        "BONUS_SHARE_RATIO": 0,

```

```

"RIGHT_ISSUE_RATIO":0,
"RIGHT_ISSUE_PRICE":0,
"CONVER_INCR_RATIO":0,
"SEO_PRICE":0,
"COMP_ID":"604498",
"SECURITY_NAME":"中国银河",
"TYPE_CODE":"A",
"TYPE_CODE_INT":1001,
"EXCHANGE_STR":"SSE",
"EXCHANGE_INT":101}}}}

```

7.3.4A 股复权因子表 A010030004

7.3.4.1 请求参数说明

参数	是否必填	参数说明	备注
function_id	是	功能号	功能号为 A010030004
market_code	是	市场代码	代码.市场, 例如: 688105.SH

7.3.4.2 返回值说明

字段名称	类型	字段说明	备注
INNER_CODE	string	市场代码	
SECURITY_CODE	string	仅代码	
EX_DATE	string	除权除息日	格式为 yyyyMMdd
EX_FACTOR	string	复权因子	当前累计复权因子/上一累计复权因子
CUM_FACTOR	string	累计复权因子	初始值为 1;当日复权因子=前一交易日收盘价/当日昨收盘价*前一交易日复权因子。

示例数据:

```

{"code": "0", "msg": "", "body": {"data": [{
  "INNER_CODE": "000333.SZ",
  "SECURITY_CODE": "000333",
  "EX_DATE": "20130918",
  "EX_FACTOR": 1, "CUM_FACTOR": 1}, {
  "INNER_CODE": "000333.SZ",
  "SECURITY_CODE": "000333",
  "EX_DATE": "20140430",
  "EX_FACTOR": 2.610556, "CUM_FACTOR": 2.610556}, {
  "INNER_CODE": "000333.SZ",

```

```
"SECURITY_CODE": "000333",
"EX_DATE": "20150430",
"EX_FACTOR": 1.027435, "CUM_FACTOR": 2.682176}, {
"INNER_CODE": "000333.SZ",
"SECURITY_CODE": "000333",
"EX_DATE": "20160506",
"EX_FACTOR": 1.556867, "CUM_FACTOR": 4.17579}, {
"INNER_CODE": "000333.SZ",
"SECURITY_CODE": "000333",
"EX_DATE": "20170510",
"EX_FACTOR": 1.029154, "CUM_FACTOR": 4.297533}, {
"INNER_CODE": "000333.SZ",
"SECURITY_CODE": "000333",
"EX_DATE": "20180504",
"EX_FACTOR": 1.023077, "CUM_FACTOR": 4.396707}, {
"INNER_CODE": "000333.SZ",
"SECURITY_CODE": "000333",
"EX_DATE": "20190530",
"EX_FACTOR": 1.026487, "CUM_FACTOR": 4.513164}, {
"INNER_CODE": "000333.SZ",
"SECURITY_CODE": "000333",
"EX_DATE": "20200602",
"EX_FACTOR": 1.027027, "CUM_FACTOR": 4.635141}, {
"INNER_CODE": "000333.SZ",
"SECURITY_CODE": "000333",
"EX_DATE": "20210602",
"EX_FACTOR": 1.019975, "CUM_FACTOR": 4.727726}, {
"INNER_CODE": "000333.SZ",
"SECURITY_CODE": "000333",
"EX_DATE": "20220602",
"EX_FACTOR": 1.031749, "CUM_FACTOR": 4.877827}, {
"INNER_CODE": "000333.SZ",
"SECURITY_CODE": "000333",
"EX_DATE": "20230601",
"EX_FACTOR": 1.050123, "CUM_FACTOR": 5.122317}}]}
```

7.4 股本与股东

7.4.1A 股十大股东名单 A010040001

7.4.1.1 请求参数说明

参数	是否必填	参数说明	备注
function_id	是	功能号	功能号为 A010040001
market_code	是	市场代码	代码.市场, 例如: 688105.SH
start_date	是	查询起始的截止日期	填写 0 表示查询最新日期的数据
end_date	是	查询结束的截止日期	填写 0 表示查询最新日期的数据

7.4.1.2 返回值说明

字段名称	类型	字段说明	备注
ANN_DATE	string	公告日期	
HOLDER_ENDDATE	string	到期日期	
HOLDER_TYPE	int32	股东类别	10:十大股东 20:流通股前十大股东
QTY_NUM	int32	持股量序号	
HOLDER_NAME	string	股东名称	
H_COMPCODE	string	股东编码	
HOLDER_HOLDER_CATEGORY	string	股东性质	1: 个人 2: 公司
HOLDER_QUANTITY	int64	持股数 (股)	
HOLDER_PCT	double	持股比例 (%)	
HOLDER_SHARECATEGORYNAME	string	股份类型	
FLOAT_QTY	int64	流通股数量	
COMPTYPE_CODE	string	股东公司类型	参考公司类型代码表
TYPE_CODE	string	证券类别	参考证券类别代码表
TYPE_CODE_INT	int32	证券类别代码	参考证券类别代码表
EXCHANGE_STR	string	交易市场	参考交易市场代码表
EXCHANGE_INT	int32	交易市场代码	参考交易市场代码表

示例数据:

```
{
  "code": "0",
  "msg": "",
  "body": {
    "data": [
      {
        "ANN_DATE": "20221028",
        "HOLDER_ENDDATE": "20220930",
        "HOLDER_TYPE": 20,
        "QTY_NUM": 1,
        "HOLDER_NAME": "中国银河金融控股有限责任公司",
        "H_COMPCODE": "15727F",

```

```

"HOLDER_HOLDER_CATEGORY": "2",
"HOLDER_QUANTITY": 5160610864,
"HOLDER_PCT": 51.16,
"HOLDER_SHARECATEGORYNAME": "A FloatHolder",
"FLOAT_QTY": 5160610864,
"TYPE_CODE": "A",
"COMPTYPE_CODE": null,
"TYPE_CODE_INT": 1001,
"EXCHANGE_STR": "SSE",
"EXCHANGE_INT": 101}}}}

```

7.4.2 限售股解禁明细 A010040002

7.4.2.1 请求参数说明

参数	是否必填	参数说明	备注
function_id	是	功能号	功能号为 A010040002
market_code	是	市场代码	代码.市场, 例如: 688105.SH
start_date	是	查询解禁上市的起始日期	填写 0 表示查询最新日期的数据
end_date	是	查询解禁上市的结束日期	填写 0 表示查询最新日期的数据

7.4.2.2 返回值说明

字段名称	类型	字段说明	备注
MARKET_CODE	string	市场代码	
ANN_DT	string	公告日期	
LIST_DATE	string	上市日期	
SHARE_LST	int64	新增可上市股份数量	
SHARE_RATIO	double	解禁股占总股本比例 (%)	
SHARE_LST_TYPE_NAME	string	限售股份类型名称	
HOLDER_NAME	string	限售股东名称	
HOLDER_CODE	string	限售股东代码	
SHARE_LST_IS_ANN	int32	是否是实际值	1:是 : 否
COMPTYPE_CODE	string	股东公司类型	参考公司类型代码表
TYPE_CODE	string	证券类别	参考证券类别代码表
TYPE_CODE_INT	int32	证券类别代码	参考证券类别代码表
EXCHANGE_STR	string	交易市场	参考交易市场代码表
EXCHANGE_INT	int32	交易市场代码	参考交易市场代码表

示例数据:

```
{
  "code": "0",
  "msg": "",
  "body": {
    "data": [
      {
        "MARKET_CODE": "688150.SH",
        "ANN_DT": "20220419",
        "LIST_DATE": "20250917",
        "SHARE_LST": 17680000,
        "SHARE_RATIO": 4.3932,
        "SHARE_LST_TYPE_NAME": "首发原股东限售股份",
        "HOLDER_NAME": "西安麒麟投资有限公司",
        "HOLDER_CODE": "MHSZ9VcKoj",
        "SHARE_LST_IS_ANN": 0,
        "TYPE_CODE": "A",
        "TYPE_CODE_INT": 1001,
        "EXCHANGE_STR": "SSE",
        "EXCHANGE_INT": 101
      }
    ]
  }
}
```

7.4.3 解禁数据 A010040003

7.4.3.1 请求参数说明

参数	是否必填	参数说明	备注
function_id	是	功能号	功能号为 A010040003
market_code	是	市场代码	代码.市场, 例如: 688105.SH

7.4.3.2 返回值说明

字段名称	类型	字段说明	备注
MARKET_CODE	string	市场代码	
EXCHANGE	string	交易市场	参考交易市场代码表
EXCHANGE_INT	int32	交易市场代码	参考交易市场代码表
SECURITY_NAME	string	证券简称	
TYPE_CODE	string	证券类别	参考证券类别代码表
TYPE_CODE_INT	int32	证券类别代码	参考证券类别代码表
LIST_DATE	string	解禁日期	
SHARE_RATIO	double	解禁股占总股本比 (%)	
SHARE_LST_TYPE_NAME	string	解禁股份类型名称	
SHARE_LST	int64	解禁数量 (股)	
SHARE_LST_IS_ANN	int32	上市数量是否公布值	0: 否, 为预测值 1: 是, 为实际公布值
CLOSE_PRICE	double	前日收盘价 (元)	
SHARE_LST_MARKET_VALUE	double	解禁市值 (元)	SHARE_LST * CLOSE_PRICE

示例数据:

```
{
  "code": "0",
  "msg": "",
  "body": {
    "data": [
      {
        "RN": 1,
        "MARKET_CODE": "688159.SH",
        "LIST_DATE": 20230130,
        "SHARE_RATIO": 0.2945,
        "SHARE_LST_TYPE_NAME": "首发原股东限售股份",
        "SHARE_LST": 270000,
        "SECURITY_NAME": "有方科技",
        "TYPE_CODE": "A",
        "SHARE_LST_IS_ANN": 1,
        "CLOSE_PRICE": 16.47,
        "SHARE_LST_MARKET_VALUE": 4446900,
        "EXCHANGE": "SSE",
        "EXCHANGE_INT": 101,
        "TYPE_CODE_INT": 1001
      }
    ]
  }
}
```

7.4.4 股权持股冻结/质押情况 A010040004

7.4.4.1 请求参数说明

参数	是否必填	参数说明	备注
function_id	是	功能号	功能号为 A010040004
market_code	是	市场代码	代码.市场, 例如: 002507.SZ
start_date	是	公告期查询起始日期	填写 0 表示查询最新日期的数据
end_date	是	公告期查询结束日期	填写 0 表示查询最新日期的数据

7.4.4.2 返回值说明

字段名称	类型	字段说明	备注
MARKET_CODE	string	市场代码	
COMP_ID	string	公司代码	
ANN_DATE	string	公告日期	
HOLDER_ID	string	股东 ID	
HOLDER_NAME	string	股东名称	
HOLDER_TYPE_CODE	int32	股东类型代码	2:公司 3:个人
TOTAL_HOLDING_SHR"	double	持股总数 (万股)	
FRO_SHARES	double	本次冻结/质押股数	
FRO_SHR_TO_TOTAL_HOLDING_RATIO	double	本次冻结/质押占所持股比例	

FRO_SHR_TO_TOTAL_RATIO	double	本次冻结/质押占总股本比例	
TOTAL_PLEDGE_SHR	double	累计质押股数	仅质押时有效
TOTAL_PLEDGE_HOLD_SHR_RATIO	double	累计质押占持股比例 (%)	
TOTAL_PLEDGE_SHR_RATIO	double	累计质押占总股本比例 (%)	
IS_EQUITY_PLEDGE_REPO	int32	是否股权质押回购	1:是 0:否
BEGIN_DATE	string	冻结/质押起始日	
END_DATE	string	解冻/解押日期	
IS_DISFROZEN	int32	是否质押或解冻	1:是 0:否
FROZEN_INSTITUTION	string	执行冻结机构/质权方	
DISFROZEN_TIME	string	解压或解冻日期	
SHR_CATEGORY_CODE	int32	股份性质类别代码	1:法人股 2:个人股 3:国有股 4:国有股,法人股 5:流通股 6:流通股,限售流通股 7:外资股 8:限售流通股 9:优先股
FREEZE_TYPE	int32	冻结/质押类型	1:质押 2:司法 3:质押式回购

示例数据:

```
{
  "code": "0",
  "msg": "",
  "body": {
    "data": [
      {
        "RN": 1,
        "MARKET_CODE": "600818.SH",
        "COMP_ID": "1600818",
        "ANN_DATE": "20230225",
        "HOLDER_ID": "502043377",
        "HOLDER_NAME": "上海中路(集团)有限公司",
        "HOLDER_TYPE_CODE": 2,
        "TOTAL_HOLDING_SHR": 8391.903,
        "FRO_SHARES": 365.8828,
        "FRO_SHR_TO_TOTAL_HOLDING_RATIO": 26.11,
        "FRO_SHR_TO_TOTAL_RATIO": 1.14,
        "TOTAL_PLEDGE_SHR": null,
        "TOTAL_PLEDGE_HOLD_SHR_RATIO": null,
        "TOTAL_PLEDGE_SHR_RATIO": null,
        "IS_EQUITY_PLEDGE_REPO": null,
        "BEGIN_DATE": "20230223",
        "END_DATE": "20260222",
        "IS_DISFROZEN": 0,
        "FROZEN_INSTITUTION": "上海市浦东新区人民法院",
        "DISFROZEN_TIME": null,
        "SHR_CATEGORY_CODE": 5,
        "FREEZE_TYPE": 2
      }
    ]
  }
}
```

7.5 财务数据

7.5.1A 股一般企业资产负债表 A010050001

7.5.1.1 请求参数说明

参数	是否必填	参数说明	备注
function_id	是	功能号	功能号为 A010050001
market_code	是	市场代码	代码.市场, 例如: 688105.SH
start_date	是	报告期查询起始日期	填写 0 表示查询最新日期的数据
end_date	是	报告期查询结束日期	填写 0 表示查询最新日期的数据

7.5.1.2 返回值说明

字段名称	类型	字段说明	备注
ACC_PAYABLE	double	应付票据及应付账款	
ACC_RECEIVABLE	double	应收票据及应收账款	
ACC_RECEIVABLES	double	应收款项	
ACCRUED_EXP	double	预提费用	
ACCT_PAYABLE	double	应付账款	
ACCT_RECEIVABLE	double	应收账款	
ACT_TRADING_SEC	double	代理买卖证券款	
ACT_UW_SEC	double	代理承销证券款	
ACTUAL_ANN_DATE	string	实际公告日期	
ADV_PREM	double	预收保费	
ADV_RECEIPT	double	预收款项	
AGENCY_ASSETS	double	代理业务资产	
AGENCY_BUSINESS_LIAB	double	代理业务负债	
ANN_DATE	string	公告日期	
ANTICIPATION_LIAB	double	预计负债	
ASSET_DEP_FUNDS_OTH_FIN_INST	double	存放同业和其它金融机构款项	
BONDS_PAYABLE	double	应付债券	
CAP_RESV	double	资本公积金	
CAP_STOCK	double	股本	金额(元), 公布值
CASH_CENTRAL_BANK_DEPOSITS	double	现金及存放中央银行款项	
CED_INSUR_CONT_RESERVES_RCV	double	应收分保合同准备金	
CLAIMS_PAYABLE	double	应付赔付款	

CLIENTS_FUND_DEPOSIT	double	客户资金存款	
CLIENTS_RESERVES	double	客户备付金	
CNVD_DIFF_FOREIGN_CURR_STAT	double	外币报表折算差额	
COMP_ID	string	公司 ID	
COMP_TYPE_CODE	int32	公司类型代码	1: 非金融类 2: 银行 3: 保险 4: 证券
CONST_IN_PROC	double	在建工程	
CONST_IN_PROC_TOTAL	double	在建工程(合计)(元)	
CONSUMP_BIO_ASSETS	double	消耗性生物资产	
CONT_ASSETS	double	合同资产	单位 (元)
CONT_LIABILITIES	double	合同负债	单位 (元)
CURRENCY_CAP	double	货币资金	
CURRENCY_CODE	string	货币代码	
DEBT_INV	double	债权投资(元)	
DEFERRED_INC_NONCUR_LIAB	double	递延收益-非流动负债	
DEFERRED_INCOME	double	递延收益	
DEFERRED_TAX_ASSETS	double	递延所得税资产	
DEFERRED_TAX_LIAB	double	递延所得税负债	
DEP_RECEIVED_IB_DEP	double	吸收存款及同业存放	
DEPOSIT_CAP_RECOG	double	存出资本保证金	
DEPOSIT_TAKING	double	吸收存款	
DEPOSITS_RECEIVED	double	存入保证金	
DER_FIN_ASSETS	double	衍生金融资产	
DERI_FIN_LIAB	double	衍生金融负债	
DEVELOP_EXP	double	开发支出	
DISPOSAL_FIX_ASSETS	double	固定资产清理	
DIV_PAYABLE	double	应付股利	
DIV_RECEIVABLE	double	应收股利	
EMPL_PAY_PAYABLE	double	应付职工薪酬	
ENGIN_MAT	double	工程物资	
FIN_ASSETS_AVA_FOR_SALE	double	可供出售金融资产	
FIN_ASSETS_COST_SHARING	double	以摊余成本计量的金融资产	
FIN_ASSETS_FAIR_VALUE	double	以公允价值计量且其变动计入其他综合收益的金融资产	
FIXED_ASSETS	double	固定资产	
FIXED_ASSETS_TOTAL	double	固定资产(合计)(元)	
FIXED_TERM_DEPOSITS	double	定期存款	
GOODWILL	double	商誉	
GUA_DEPOSITS_PAID	double	存出保证金	

GUA_PLEDGE_LOANS	double	保户质押贷款	
HOLD_ASSETS_FOR_SALE	double	持有待售的资产	
HOLD_TO_MTY_INV	double	持有至到期投资	
INC_PLEDGE_LOAN	double	其中:质押借款	
INCL_TRADING_SEAT_FEES	double	其中:交易席位费	
IND_ACCT_ASSETS	double	独立账户资产	
IND_ACCT_LIAB	double	独立账户负债	
INSURED_DEPOSIT_INV	double	保户储金及投资款	
INSURED_DIV_PAYABLE	double	应付保单红利	
INT_RECEIVABLE	double	应收利息	
INTANGIBLE_ASSETS	double	无形资产	
INTEREST_PAYABLE	double	应付利息	
INV	double	存货	
INV_REALESTATE	double	投资性房地产	
LEASE_LIABILITY	double	租赁负债	
LEND_FUNDS	double	融出资金	
LENDING_FUNDS	double	拆出资金	
LESS_TREASURY_STK	double	减:库存股	
LIA_HFS	double	持有待售的负债	
LIAB_DEP_FUNDS_OTH_FIN_INST	double	同业和其它金融机构 存放款项	
LIFE_INSUR_RESV	double	寿险责任准备金	
LOAN_CENTRAL_BANK	double	向中央银行借款	
LOANS_AND_ADVANCES	double	发放贷款及垫款	
LOANS_FROM_OTH_BANKS	double	拆入资金	
LT_DEFERRED_EXP	double	长期待摊费用	
LT_EMP_COMP_PAY	double	长期应付职工薪酬	
LT_EQUITY_INV	double	长期股权投资	
LT_HEALTH_INSUR_RESV	double	长期健康险责任准备 金	
LT_LOAN	double	长期借款	
LT_PAYABLE	double	长期应付款	
LT_PAYABLE_TOTAL	double	长 期 应 付 款 (合 计)(元)	
LT_RECEIVABLES	double	长期应收款	
MARKET_CODE	string	市场代码	
MINORITY_EQUITY	double	少数股东权益	
NOM_RISKS_PREP	double	一般风险准备	
NONCUR_ASSETS_DUE_WITHIN_1Y	double	一年内到期的非流动 资产	
NONCUR_LIAB_DUE_WITHIN_1Y	double	一年内到期的非流动 负债	
NOTES_PAYABLE	double	应付票据	

NOTES_RECEIVABLE	double	应收票据	
OIL_AND_GAS_ASSETS	double	油气资产	
OTH_COMP_INCOME	double	其他综合收益	
OTH_EQUITY_TOOLS	double	其他权益工具	
OTH_EQUITY_TOOLS_PRE_SHR	double	其他权益工具: 优先股	
OTH_NONCUR_ASSETS	double	其他非流动资产	
OTHER_ASSETS	double	其他资产	
OTHER_CUR_ASSETS	double	其他流动资产	
OTHER_CUR_LIAB	double	其他流动负债	
OTHER_DEBT_INV	double	其他债权投资(元)	
OTHER_EQUITY_INV	double	其他权益工具投资(元)	
OTHER_LIAB	double	其他负债	
OTHER_NONCUR_FIN_ASSETS	double	其他非流动金融资产(元)	
OTHER_NONCUR_LIAB	double	其他非流动负债	
OTHER_PAYABLE	double	其他应付款	
OTHER_PAYABLE_TOTAL	double	其他应付款(合计)(元)	
OTHER_RCV_TOTAL	double	其他应收款(合计)(元)	
OTHER_RECEIVABLE	double	其他应收款	
OTHER_SUSTAIN_BOND	double	其他权益工具: 永续债(元)	
OUT_LOSS_RESV	double	未决赔款准备金	
PAYABLE	double	应付款项	
PAYABLE_FOR_REINSURER	double	应付分保账款	
PRECIOUS_METAL	double	贵金属	
PREPAYMENT	double	预付款项	
PROD_BIO_ASSETS	double	生产性生物资产	
RCV_CED_CLAIM_RESV	double	应收分保未决赔款准备金	
RCV_CED_LIFE_INSUR_RESV	double	应收分保寿险责任准备金	
RCV_CED_LT_HEALTH_INSUR_RESV	double	应收分保长期健康险责任准备金	
RCV_CED_UNEARNED_PREM_RESV	double	应收分保未到期责任准备金	
RCV_FINANCING	double	应收款项融资	
RCV_INV	double	应收款项类投资	
RECEIVABLE_PREM	double	应收保费	
RED_MON_CAP_FOR_SALE	double	买入返售金融资产	

REINSURANCE_ACC_RCV	double	应收分保账款	
REPORT_TYPE	string	报告期名称	
REPORTING_PERIOD	string	报告期	
RSRV_FUND_INSUR_CONT	double	保险合同准备金	
SECURITY_NAME	string	证券简称	
SELL_REPO_FIN_ASSETS	double	卖出回购金融资产款	
SERVICE_CHARGE_COMM_PAYABLE	double	应付手续费及佣金	
SETTLE_FUNDS	double	结算备付金	
SPE_ASSETS_BAL_DIFF	double	资产差额(特殊报表科目)	
SPE_CUR_ASSETS_DIFF	double	流动资产差额(特殊报表科目)	
SPE_CUR_LIAB_DIFF	double	流动负债差额(特殊报表科目)	
SPE_LIAB_BAL_DIFF	double	负债差额(特殊报表科目)	
SPE_LIAB_EQUITY_BAL_DIFF	double	负债及股东权益差额(特殊报表项目)	
SPE_NONCUR_ASSETS_DIFF	double	非流动资产差额(特殊报表科目)	
SPE_NONCUR_LIAB_DIFF	double	非流动负债差额(特殊报表科目)	
SPE_SHARE_EQUITY_BAL_DIFF	double	股东权益差额(特殊报表科目)	
SPECIAL_PAYABLE	double	专项应付款	
SPECIAL_RESV	double	专项储备	
ST_BONDS_PAYABLE	double	应付短期债券	
ST_BORROWING	double	短期借款	
ST_FIN_PAYABLE	double	应付短期融资款	
STATEMENT_TYPE	string	报表类型	参看报表类型代码表
SUBR_RCV	double	应收代位追偿款	
SURPLUS_RESV	double	盈余公积金	
TAX_PAYABLE	double	应交税费	
TOT_ASSETS_BAL_DIFF	double	资产差额(合计平衡项目)	
TOT_CUR_ASSETS_DIFF	double	流动资产差额(合计平衡项目)	
TOT_CUR_LIAB_DIFF	double	流动负债差额(合计平衡项目)	
TOT_LIAB_BAL_DIFF	double	负债差额(合计平衡项目)	
TOT_LIAB_EQUITY_BAL_DIFF	double	负债及股东权益差额	

		(合计平衡项目)	
TOT_NONCUR_ASSETS	double	非流动资产合计	
TOT_NONCUR_ASSETS_DIFF	double	非流动资产差额(合计平衡项目)	
TOT_NONCUR_LIAB_DIFF	double	非流动负债差额(合计平衡项目)	
TOT_SHARE	double	期末总股本	单位(股)
TOT_SHARE_EQUITY_BAL_DIFF	double	股东权益差额(合计平衡项目)	
TOT_SHARE_EQUITY_EXCL_MIN_INT	double	股东权益合计(不含少数股东权益)	
TOT_SHARE_EQUITY_INCL_MIN_INT	double	股东权益合计(含少数股东权益)	
TOTAL_ASSETS	double	资产总计	
TOTAL_CUR_ASSETS	double	流动资产合计	
TOTAL_CUR_LIAB	double	流动负债合计	
TOTAL_LIAB	double	负债合计	
TOTAL_LIAB_SHARE_EQUITY	double	负债及股东权益总计	
TOTAL_NONCUR_LIAB	double	非流动负债合计	
TRADING_FIN_LIAB	double	交易性金融负债	
TRADING_FINASSETS	double	交易性金融资产	
UNAMORTIZED_EXP	double	待摊费用	
UNCONFIRMED_INV_LOSS	double	未确认的投资损失	
UNDISTRIBUTED_PRO	double	未分配利润	
UNEARNED_PREM_RESV	double	未到期责任准备金	
USE_RIGHT_ASSETS	double	使用权资产	

示例数据:

```
{
  "code": "0",
  "msg": "",
  "body": {
    "data": [
      {
        "STATEMENT_TYPE": "1",
        "MARKET_CODE": "601881.SH",
        "REPORTING_PERIOD": "20220930",
        "SPECIAL_PAYABLE": null,
        "ANTICIPATION_LIAB": null,
        "DEFERRED_TAX_LIAB": 1076069242.66,
        "DEFERRED_INC_NONCUR_LIAB": null,
        "OTHER_NONCUR_LIAB": null,
        "TOTAL_NONCUR_LIAB": null,
        "LIAB_DEP_FUNDS_OTH_FIN_INST": null,
        "DERI_FIN_LIAB": 3554056646.32,
        "DEPOSIT_TAKING": null,
        "AGENCY_BUSINESS_LIAB": null,
        "OTHER_LIAB": 21910756931.89,
        "ADV_PREM": null,
        "DEPOSITS_RECEIVED": null,
      }
    ]
  }
}
```

"INSURED_DEPOSIT_INV":null,
"UNEARNED_PREM_RESV":null,
"OUT_LOSS_RESV":null,
"LIFE_INSUR_RESV":null,
"LT_HEALTH_INSUR_RESV":null,
"IND_ACCT_LIAB":null,
"INC_PLEDGE_LOAN":null,
"CLAIMS_PAYABLE":null,
"INSURED_DIV_PAYABLE":null,
"TOTAL_LIAB":519151139192.26,
"CAP_STOCK":10137265160,
"CAP_RESV":25051524854.2,
"SPECIAL_RESV":null,
"SURPLUS_RESV":7732341274.17,
"UNDISTRIBUTED_PRO":29320116933.57,
"LESS_TREASURY_STK":null,
"NOM_RISKS_PREF":13628042625.07,
"CNVD_DIFF_FOREIGN_CURR_STAT":null,
"UNCONFIRMED_INV_LOSS":null,
"MINORITY_EQUITY":22880995.43,
"TOT_SHARE_EQUITY_EXCL_MIN_INT":103150147604.28,
"TOT_SHARE_EQUITY_INCL_MIN_INT":103173028599.71,
"TOTAL_LIAB_SHARE_EQUITY":622324167791.97,
"COMP_TYPE_CODE":"4",
"ACTUAL_ANN_DATE":"20221028",
"SPE_CUR_ASSETS_DIFF":null,
"TOT_CUR_ASSETS_DIFF":null,
"SPE_NONCUR_ASSETS_DIFF":null,
"TOT_NONCUR_ASSETS_DIFF":null,
"SPE_ASSETS_BAL_DIFF":null,
"TOT_ASSETS_BAL_DIFF":null,
"SPE_CUR_LIAB_DIFF":null,
"TOT_CUR_LIAB_DIFF":null,
"SPE_NONCUR_LIAB_DIFF":null,
"TOT_NONCUR_LIAB_DIFF":null,
"SPE_LIAB_BAL_DIFF":null,
"TOT_LIAB_BAL_DIFF":null,
"SPE_SHARE_EQUITY_BAL_DIFF":null,
"TOT_SHARE_EQUITY_BAL_DIFF":null,
"SPE_LIAB_EQUITY_BAL_DIFF":null,
"TOT_LIAB_EQUITY_BAL_DIFF":null,
"LT_EMP_COMP_PAY":null,
"OTH_COMP_INCOME":1345146717.53,
"OTH_EQUITY_TOOLS":15935710039.74,

"OTH_EQUITY_TOOLS_PRE_SHR":null,
"LEND_FUNDS":84670647259.9,
"ACC_RECEIVABLES":13376602476.48,
"ST_FIN_PAYABLE":29264411179.37,
"PAYABLE":14491265492.37,
"COMP_ID":"604498",
"TOT_SHARE":10137265160,
"HOLD_ASSETS_FOR_SALE":null,
"LIA_HFS":null,
"FIN_ASSETS_COST_SHARING":null,
"FIN_ASSETS_FAIR_VALUE":null,
"CONT_ASSETS":null,
"CONT_LIABILITIES":null,
"ACC_RECEIVABLE":null,
"ACC_PAYABLE":null,
"OTHER_RCV_TOTAL":null,
"FIXED_ASSETS_TOTAL":null,
"CONST_IN_PROC_TOTAL":null,
"OTHER_PAYABLE_TOTAL":null,
"LT_PAYABLE_TOTAL":null,
"DEBT_INV":3050074183.73,
"OTHER_DEBT_INV":125823656852.21,
"OTHER_EQUITY_INV":40848917708.54,
"OTHER_NONCUR_FIN_ASSETS":null,
"OTHER_SUSTAIN_BOND":14885850943.44,
"RCV_FINANCING":null,
"USE_RIGHT_ASSETS":1449801214.25,
"LEASE_LIABILITY":1457988408.28,
"ANN_DATE":"20221028",
"CURRENCY_CODE":"CNY",
"CURRENCY_CAP":101532175765.6,
"TRADING_FINASSETS":167286554528.62,
"NOTES_RECEIVABLE":null,
"ACCT_RECEIVABLE":null,
"OTHER_RECEIVABLE":null,
"PREPAYMENT":null,
"DIV_RECEIVABLE":null,
"INT_RECEIVABLE":null,
"INV":null,
"CONSUMP_BIO_ASSETS":null,
"UNAMORTIZED_EXP":null,
"NONCUR_ASSETS_DUE_WITHIN_1Y":null,
"SETTLE_FUNDS":35879849454.88,
"LENDING_FUNDS":null,

"RECEIVABLE_PREM":null,
"REINSURANCE_ACC_RCV":null,
"CED_INSUR_CONT_RESERVES_RCV":null,
"RED_MON_CAP_FOR_SALE":18365585150.98,
"OTHER_CUR_ASSETS":null,
"TOTAL_CUR_ASSETS":null,
"FIN_ASSETS_AVA_FOR_SALE":null,
"HOLD_TO_MTY_INV":null,
"LT_EQUITY_INV":57878146,
"INV_REALESTATE":7493400,
"FIXED_TERM_DEPOSITS":null,
"OTHER_ASSETS":8493248592.62,
"LT_RECEIVABLES":null,
"FIXED_ASSETS":521234387.09,
"CONST_IN_PROC":null,
"ENGIN_MAT":null,
"DISPOSAL_FIX_ASSETS":null,
"PROD_BIO_ASSETS":null,
"OIL_AND_GAS_ASSETS":null,
"INTANGIBLE_ASSETS":731638255.62,
"DEVELOP_EXP":null,
"GOODWILL":1031342566.35,
"LT_DEFERRED_EXP":null,
"DEFERRED_TAX_ASSETS":164377843.7,
"LOANS_AND_ADVANCES":null,
"OTH_NONCUR_ASSETS":null,
"TOT_NONCUR_ASSETS":null,
"CASH_CENTRAL_BANK_DEPOSITS":null,
"ASSET_DEP_FUNDS_OTH_FIN_INST":null,
"PRECIOUS_METAL":null,
"DER_FIN_ASSETS":4535770779.27,
"AGENCY_ASSETS":null,
"SUBR_RCV":null,
"RCV_CED_UNEARNED_PREM_RESV":null,
"RCV_CED_CLAIM_RESV":null,
"RCV_CED_LIFE_INSUR_RESV":null,
"RCV_CED_LT_HEALTH_INSUR_RESV":null,
"GUA_DEPOSITS_PAID":14497319226.13,
"GUA_PLEDGE_LOANS":null,
"DEPOSIT_CAP_RECOG":null,
"IND_ACCT_ASSETS":null,
"CLIENTS_FUND_DEPOSIT":83922384134.98,
"CLIENTS_RESERVES":24670060192.83,
"INCL_TRADING_SEAT_FEES":null,

```

"RCV_INV":null,
"TOTAL_ASSETS":622324167791.97,
"ST_BORROWING":8949047480.9,
"LOAN_CENTRAL_BANK":null,
"DEP_RECEIVED_IB_DEP":null,
"LOANS_FROM_OTH_BANKS":9695819116.93,
"TRADING_FIN_LIAB":36273408898.47,
"NOTES_PAYABLE":null,
"ACCT_PAYABLE":null,
"ADV_RECEIPT":null,
"SELL_REPO_FIN_ASSETS":173471333887.48,
"SERVICE_CHARGE_COMM_PAYABLE":null,
"EMPL_PAY_PAYABLE":4713492512.8,
"TAX_PAYABLE":411837272.77,
"INTEREST_PAYABLE":null,
"DIV_PAYABLE":null,
"OTHER_PAYABLE":null,
"ACCRUED_EXP":null,
"DEFERRED_INCOME":null,
"ST_BONDS_PAYABLE":null,
"PAYABLE_FOR_REINSURER":null,
"RSRV_FUND_INSUR_CONT":null,
"ACT_TRADING_SEC":112845867062.56,
"ACT_UW_SEC":92418718.96,
"NONCUR_LIAB_DUE_WITHIN_1Y":null,
"OTHER_CUR_LIAB":null,
"TOTAL_CUR_LIAB":null,
"LT_LOAN":null,
"BONDS_PAYABLE":100943366340.5,
"LT_PAYABLE":null,
"SECURITY_NAME":"中国银河",
"REPORT_TYPE":"3"}]}}

```

7.5.2A 股一般企业利润表 A010050002

7.5.2.1 请求参数说明

参数	是否必填	参数说明	备注
function_id	是	功能号	功能号为 A010050002
market_code	是	市场代码	代码.市场, 例如: 688105.SH
start_date	是	报告期查询起始日期	填写 0 表示查询最新日期的数据
end_date	是	报告期查询结束日期	填写 0 表示查询最新日期的数据

7.5.2.2 返回值说明

字段名称	类型	字段说明	备注
ACTUAL_ANN_DATE	string	实际公告日期	
ADJ_PREV_YEAR_LOSS_GAIN	double	调整以前年度损益	
AMORT_COST_FIN_ASSETS_EAR	double	以摊余成本计量的金融资产终止确认收益	
ANN_DATE	string	公告日期	
BASIC_EPS	double	基本每股收益	
BEG_UNDISTRIBUTED_PRO	double	年初未分配利润	
CAPITALIZED_COM_STOCK_DIV	double	转作股本的普通股股利	
COMMENTS	string	备注	
COMMON_STOCK_DIV_PAYABLE	double	应付普通股股利	
COMP_ID	string	公司 ID	
COMP_TYPE_CODE	string	公司类型代码	1: 非金融类 2: 银行 3: 保险 4: 证券
CONTINUED_NET_OPERA_PRO	double	持续经营净利润	
CREDIT_IMPAIR_LOSS	double	信用减值损失	
CURRENCY_CODE	string	货币代码	
DILUTED_EPS	double	稀释每股收益	
DISTRIBUTIVE_PRO	double	可分配利润	
DISTRIBUTIVE_PRO_SHAREHOLDER	double	可供股东分配的利润	
DIV_EXP_INSUR	double	保户红利支出	
EBIT	double	息税前利润	正向法
EBITDA	double	息税折旧摊销前利润	
EMPLOYEE_WELFARE	double	职工奖金福利	
END_NET_OPERA_PRO	double	终止经营净利润	
EXT_INSUR_CONT_RSRV	double	提取保险责任准备金	
EXT_UNEARNED_PREM_RES	double	提取未到期责任准备金	
FIN_EXP_INT_EXP	double	财务费用:利息费用	
FIN_EXP_INT_INC	double	财务费用:利息收入	
GAIN_DISPOSAL_ASSETS	double	资产处置收益	
HANDLING_CHRG_COMM_FEE	double	手续费及佣金收入	
INCL_INC_INV_JV_ENTP	double	其中:对联营企业和合营企业的投资收益	
INCL_LESS_LOSS_DISP_NCUR_ASSET	double	其中:减:非流动资产处置净损失	
INCL_REINSUR_PREM_INC	double	其中:分保费收入	
INCOME_TAX	double	所得税	

INSUR_EXP	double	保险业务支出	
INSUR_PREM	double	已赚保费	
INTEREST_INC	double	利息收入	
IS_CALCULATION	double	是否计算报表	
LESS_ADMIN_EXP	double	减:管理费用	
LESS_AMORT_COMPEN_EXP	double	减:摊回赔付支出	
LESS_AMORT_INSUR_CONT_RSRV	double	减:摊回保险责任准备金	
LESS_AMORT_REINSUR_EXP	double	减:摊回分保费用	
LESS_ASSETS_IMPAIR_LOSS	double	减:资产减值损失	
LESS_BUS_TAX_SURCHARGE	double	减:营业税金及附加	
LESS_FIN_EXP	double	减:财务费用	
LESS_HANDLING_CHRG_COMM_FEE	double	减:手续费及佣金支出	
LESS_INTEREST_EXP	double	减:利息支出	
LESS_NON_OPERA_EXP	double	减:营业外支出	
LESS_OPERA_COST	double	减:营业成本	
LESS_REINSUR_PREM	double	减:分出保费	
LESS_SELLING_EXP	double	减:销售费用	
MARKET_CODE	string	市场代码	
MIN_INT_INC	double	少数股东损益	
NET_EXPOSURE_HEDGING_GAIN	double	净敞口套期收益	
NET_HANDLING_CHRG_COMM_FEE	double	手续费及佣金净收入	
NET_INC_EC_ASSET_MGMT_BUS	double	受托客户资产管理业务净收入	
NET_INC_SEC_BROK_BUS	double	代理买卖证券业务净收入	
NET_INC_SEC_UW_BUS	double	证券承销业务净收入	
NET_INTEREST_INC	double	利息净收入	
NET_PRO_AFTER_DED_NR_GL	double	扣除非经常性损益后净利润(扣除少数股东损益)	
NET_PRO_AFTER_DED_NR_GL_COR	double	扣除非经常性损益后的净利润(财务重要指标(更正前))	
NET_PRO_EXCL_MIN_INT_INC	double	净利润(不含少数股东损益)	
NET_PRO_INCL_MIN_INT_INC	double	净利润(含少数股东损益)	
NET_PRO_UNDER_INT_ACC_STA	double	国际会计准则净利润	
OPERA_EXP	double	营业支出	
OPERA_PROFIT	double	营业利润	
OPERA_REV	double	营业收入	

OTH_ASSETS_IMPAIR_LOSS	double	其他资产减值损失	
OTH_BUS_COST	double	其他业务成本	
OTH_BUS_INC	double	其他业务收入	
OTH_COMPRE_INC	double	其他综合收益	
OTH_INCOME	double	其他收益	
OTH_NET_OPERA_INC	double	其他经营净收益	
PLUS_NET_FX_INC	double	加:汇兑净收益	
PLUS_NET_GAIN_CHG_FV	double	加:公允价值变动净收益	
PLUS_NET_INV_INC	double	加:投资净收益	
PLUS_NON_OPERA_REV	double	加:营业外收入	
PLUS_OTH_NET_BUS_INC	double	加:其他业务净收益	
PREFERRED_SHARE_DIV_PAYABLE	double	应付优先股股利	
PREM_BUS_INC	double	保费业务收入	
RD_EXP	double	研发费用	
REINSURANCE_EXP	double	分保费用	
REPORT_TYPE	string	报告期名称	
REPORTING_PERIOD	string	报告期	
SECURITY_NAME	string	证券简称	
SPE_BAL_NET_PRO_MARG	double	净利润差额(特殊报表科目)	
SPE_BAL_OPERA_PRO_MARG	double	营业利润差额(特殊报表科目)	
SPE_BAL_TOT_OPERA_COST_DIF	double	营业总成本差额(特殊报表科目)	
SPE_BAL_TOT_OPERA_INC_DIF	double	营业总收入差额(特殊报表科目)	
SPE_BAL_TOT_PRO_MARG	double	利润总额差额(特殊报表科目)	
SPE_TOT_OPERA_COST_DIF_STATE	string	营业总成本差额说明(特殊报表科目)	
SPE_TOT_OPERA_INC_DIF_STATE	string	营业总收入差额说明(特殊报表科目)	
STATEMENT_TYPE	int32	报表类型	参看报表类型代码表
SURR_VALUE	double	退保金	
TOT_BAL_NET_PRO_MARG	double	净利润差额(合计平衡项目)	
TOT_BAL_OPERA_PRO_MARG	double	营业利润差额(合计平衡项目)	
TOT_BAL_TOT_PRO_MARG	double	利润总额差额(合计平衡项目)	
TOT_COMPEN_EXP	double	赔付总支出	

TOT_COMPRE_INC	double	综合收益总额	
TOT_COMPRE_INC_MIN_SHARE	double	综合收益总额(少数股东)	
TOT_COMPRE_INC_PARENT_COMP	double	综合收益总额(母公司)	
TOT_OPERA_COST	double	营业总成本	
TOT_OPERA_COST2	double	营业总成本 2	
TOT_OPERA_REV	double	营业总收入	
TOTAL_PROFIT	double	利润总额	
TRANSFER_HOUSING_REVO_FUNDS	double	住房周转金转入	
TRANSFER_OTHERS	double	其他转入	
TRANSFER_SURPLUS_RESERVE	double	盈余公积转入	
UNCONFIRMED_INV_LOSS	double	未确认投资损失	
WITHDRAW_ANY_SURPLUS_RESV	double	提取任意盈余公积金	
WITHDRAW_ENT_DEVELOP_FUND	double	提取企业发展基金	
WITHDRAW_LEG_PUB_WEL_FUND	double	提取法定公益金	
WITHDRAW_LEG_SURPLUS	double	提取法定盈余公积	
WITHDRAW_RESV_FUND	double	提取储备基金	

示例数据:

```
{
  "code": "0",
  "msg": "",
  "body": {
    "data": [
      {
        "STATEMENT_TYPE": "6",
        "MARKET_CODE": "601881.SH",
        "GAIN_DISPOSAL_ASSETS": 1066177.03,
        "CONTINUED_NET_OPERA_PRO": 5534446302.09,
        "END_NET_OPERA_PRO": null,
        "CREDIT_IMPAIR_LOSS": 73744826.46,
        "NET_EXPOSURE_HEDGING_GAIN": null,
        "RD_EXP": null,
        "FIN_EXP_INT_EXP": null,
        "FIN_EXP_INT_INC": null,
        "IS_CALCULATION": 0,
        "OTH_ASSETS_IMPAIR_LOSS": null,
        "TOT_OPERA_COST2": null,
        "AMORT_COST_FIN_ASSETS_EAR": null,
        "SPE_BAL_TOT_OPERA_INC_DIF": null,
        "SPE_TOT_OPERA_INC_DIF_STATE": null,
        "SPE_BAL_TOT_OPERA_COST_DIF": null,
        "SPE_TOT_OPERA_COST_DIF_STATE": null,
        "ANN_DATE": "20221028",
        "REPORTING_PERIOD": "20220930",
        "CURRENCY_CODE": "CNY",
        "TOT_OPERA_REV": 11312471641.09,
        "OPERA_REV": 11312471641.09,
        "INTEREST_INC": 9094924487.58,
      }
    ]
  }
}
```

"NET_INTEREST_INC":3124825044.26,
"INSUR_PREM":null,
"HANDLING_CHRG_COMM_FEE":null,
"NET_HANDLING_CHRG_COMM_FEE":4661286912.98,
"OTH_NET_OPERA_INC":3526359683.85,
"PLUS_OTH_NET_BUS_INC":2623917.57,
"PREM_BUS_INC":null,
"LESS_REINSUR_PREM":null,
"EXT_UNEARNED_PREM_RES":null,
"INCL_REINSUR_PREM_INC":null,
"NET_INC_SEC_BROK_BUS":4149269776.01,
"NET_INC_SEC_UW_BUS":463432656.63,
"NET_INC_EC_ASSET_MGMT_BUS":null,
"OTH_BUS_INC":2623917.57,
"PLUS_NET_GAIN_CHG_FV":349476068.32,
"PLUS_NET_INV_INC":3139374050.96,
"INCL_INC_INV_JV_ENTP":null,
"PLUS_NET_FX_INC":-9979480.93,
"TOT_OPERA_COST":5328693459.97,
"LESS_OPERA_COST":null,
"LESS_INTEREST_EXP":5970099443.32,
"LESS_HANDLING_CHRG_COMM_FEE":null,
"LESS_BUS_TAX_SURCHARGE":80256125.88,
"LESS_SELLING_EXP":null,
"LESS_ADMIN_EXP":5174692507.63,
"LESS_FIN_EXP":null,
"LESS_ASSETS_IMPAIR_LOSS":null,
"SURRE_VALUE":null,
"TOT_COMPEN_EXP":null,
"EXT_INSUR_CONT_RSRV":null,
"DIV_EXP_INSUR":null,
"REINSURANCE_EXP":null,
"OPERA_EXP":5328693459.97,
"LESS_AMORT_COMPEN_EXP":null,
"LESS_AMORT_INSUR_CONT_RSRV":null,
"LESS_AMORT_REINSUR_EXP":null,
"OTH_BUS_COST":null,
"OPERA_PROFIT":5983778181.12,
"PLUS_NON_OPERA_REV":11560923.76,
"LESS_NON_OPERA_EXP":8258132.46,
"INCL_LESS_LOSS_DISP_NCUR_ASSET":null,
"TOTAL_PROFIT":5987080972.42,
"INCOME_TAX":452634670.33,
"UNCONFIRMED_INV_LOSS":null,

"NET_PRO_INCL_MIN_INT_INC":5534446302.09,
"NET_PRO_EXCL_MIN_INT_INC":5534446302.09,
"MIN_INT_INC":null,
"OTH_COMPRE_INC":394981021.55,
"TOT_COMPRE_INC":5929427323.64,
"TOT_COMPRE_INC_PARENT_COMP":5929427323.64,
"TOT_COMPRE_INC_MIN_SHARE":null,
"EBIT":null,
"EBITDA":null,
"NET_PRO_AFTER_DED_NR_GL":null,
"NET_PRO_UNDER_INT_ACC_STA":null,
"COMP_TYPE_CODE":"4",
"BASIC_EPS":null,
"DILUTED_EPS":null,
"ACTUAL_ANN_DATE":"20221028",
"INSUR_EXP":null,
"SPE_BAL_OPERA_PRO_MARG":null,
"TOT_BAL_OPERA_PRO_MARG":null,
"SPE_BAL_TOT_PRO_MARG":null,
"TOT_BAL_TOT_PRO_MARG":null,
"SPE_BAL_NET_PRO_MARG":null,
"TOT_BAL_NET_PRO_MARG":null,
"BEG_UNDISTRIBUTED_PRO":null,
"ADJ_PREV_YEAR_LOSS_GAIN":null,
"TRANSFER_SURPLUS_RESERVE":null,
"TRANSFER_HOUSING_REVO_FUNDS":null,
"TRANSFER_OTHERS":null,
"DISTRIBUTIVE_PRO":null,
"WITHDRAW_LEG_SURPLUS":null,
"WITHDRAW_LEG_PUB_WEL_FUND":null,
"EMPLOYEE_WELFARE":null,
"WITHDRAW_ENT_DEVELOP_FUND":null,
"WITHDRAW_RESV_FUND":null,
"DISTRIBUTIVE_PRO_SHAREHOLDER":null,
"PREFERRED_SHARE_DIV_PAYABLE":null,
"WITHDRAW_ANY_SURPLUS_RESV":null,
"COMMON_STOCK_DIV_PAYABLE":null,
"CAPITALIZED_COM_STOCK_DIV":null,
"COMP_ID":"604498",
"NET_PRO_AFTER_DED_NR_GL_COR":null,
"OTH_INCOME":43798950.9,
"COMMENTS":null,
"SECURITY_NAME":"中国银河",
"REPORT_TYPE":"3"}]}}

7.5.3A 股一般企业现金流表 A010050003

7.5.3.1 请求参数说明

参数	是否必填	参数说明	备注
function_id	是	功能号	功能号为 A010050003
market_code	是	市场代码	代码.市场, 例如: 688105.SH
start_date	是	报告期查询起始日期	填写 0 表示查询最新日期的数据
end_date	是	报告期查询结束日期	填写 0 表示查询最新日期的数据

7.5.3.2 返回值说明

字段名称	类型	字段说明	备注
ABSORB_CASH_RECIP_INV	double	吸收投资收到的现金	
ACTUAL_ANN_DATE	string	实际公告日期	
AMORT_INTAN_ASSETS	double	无形资产摊销	
AMORT_LT_DEFERRED_EXP	double	长期待摊费用摊销	
ANN_DATE	string	公告日期	
BEG_BAL_CASH_CASH_EQU	double	期初现金及现金等价物余额	
CASH_END_BAL	double	现金的期末余额	
CASH_FOR_CHARGE	double	支付手续费的现金	
CASH_PAID_INSUR_POLICY	double	支付保单红利的现金	
CASH_PAID_INV	double	投资支付的现金	
CASH_PAID_PUR_CONST_FIOLTA	double	购建固定资产、无形资产和其他长期资产支付的现金	
CASH_PAY_CLAIMS_OIC	double	支付原保险合同赔付款项的现金	
CASH_PAY_DIST_DIV_PRO_INT	double	分配股利、利润或偿付利息支付的现金	
CASH_PAY_EMPLOYEE	double	支付给职工以及为职工支付的现金	
CASH_PAY_FOR_DEBT	double	偿还债务支付的现金	
CASH_PAY_GOODS_SERVICES	double	购买商品、接受劳务支付的现金	
CASH_RECE_BORROW	double	取得借款收到的现金	
CASH_RECE_ISSUE_BONDS	double	发行债券收到的现金	
CASH_RECIP_INV_INCOME	double	取得投资收益收到的现金	
CASH_RECIP_PREM_OIC	double	收到原保险合同保费	

		取得的现金	
CASH_REC_P_RECV_INV	double	收回投资收到的现金	
CASH_REC_P_SG_AND_RS	double	销售商品、提供劳务收到的现金	
COMP_ID	string	公司 ID	
COMP_TYPE_CODE	string	公司类型代码	1: 非金融类 2: 银行 3: 保险 4: 证券
CONV_CORP_BONDS_DUE_WITHIN_1Y	double	一年内到期的可转换公司债券	
CONV_DEBT_INTO_CAP	double	债务转为资本	
CREDIT_IMPAIR_LOSS	double	信用减值损失	
CURRENCY_CODE	string	货币代码	
DECR_DEFE_INC_TAX_ASSETS	double	递延所得税资产减少	
DECR_DEFERRED_EXPENSE	double	待摊费用减少	
DECR_INVENTORY	double	存货的减少	
DECR_OPERA_RECEIVABLE	double	经营性应收项目的减少	
DEPRE_FA_OGA_PBA	double	固定资产折旧、油气资产折耗、生产性生物资产折旧	
EFF_FX_FLUC_CASH	double	汇率变动对现金的影响	
END_BAL_CASH_CASH_EQU	double	期末现金及现金等价物余额	
FINANCIAL_EXP	double	财务费用	
FIXED_ASSETS_FIN_LEASE	double	融资租入固定资产	
FREE_CASH_FLOW	double	企业自由现金流量	
INCL_CASH_REC_P_SAIMS	double	其中:子公司吸收少数股东投资收到的现金	
INCL_DIV_PRO_PAID_SMS	double	其中:子公司支付给少数股东的股利、利润	
INCR_ACCRUED_EXP	double	预提费用增加	
INCR_DEFE_INC_TAX_LIAB	double	递延所得税负债增加	
INCR_OPERA_PAYABLE	double	经营性应付项目的增加	
IND_NET_CASH_FLOWS_OPERA_ACT	double	间接法-经营活动产生的现金流量净额	
IND_NET_INCR_CASH_AND_EQU	double	间接法-现金及现金等价物净增加额	
INV_LOSS	double	投资损失	

IS_CALCULATION	int32	是否计算报表	
LESS_OPEN_BAL_CASH	double	减:现金的期初余额	
LESS_OPEN_BAL_CASH_EQU	double	减:现金等价物的期初余额	
LOSS_DISP_FIOLTA	double	处置固定、无形资产和其他长期资产的损失	
LOSS_FAIRVALUE_CHG	double	公允价值变动损失	
LOSS_FIXED_ASSETS	double	固定资产报废损失	
MARKET_CODE	string	市场代码	
NET_CASH_FLOWS_FIN_ACT	double	筹资活动产生的现金流量净额	
NET_CASH_FLOWS_INV_ACT	double	投资活动产生的现金流量净额	
NET_CASH_FLOWS_OPERA_ACT	double	经营活动产生的现金流量净额	
NET_CASH_PAID_SOBU	double	取得子公司及其他营业单位支付的现金净额	
NET_CASH_REC_SEC	double	代理买卖证券收到的现金净额	
NET_CASH_REC_P_DISP_FIOLTA	double	处置固定资产、无形资产和其他长期资产收回的现金净额	
NET_CASH_REC_P_DISP_SOBU	double	处置子公司及其他营业单位收到的现金净额	
NET_CASH_REC_P_REINSU_BUS	double	收到再保业务现金净额	
NET_INCR_BORR_FUND	double	拆入资金净增加额	
NET_INCR_BORR_OFI	double	向其他金融机构拆入资金净增加额	
NET_INCR_CASH_AND_CASH_EQU	double	现金及现金等价物净增加额	
NET_INCR_CUS_LOAN_ADV	double	客户贷款及垫款净增加额	
NET_INCR_DEP_CB_IB	double	存放央行和同业款项净增加额	
NET_INCR_DEP_CUS_AND_IB	double	客户存款和同业存放款项净增加额	
NET_INCR_DISMANTLE_CAP	double	拆出资金净增加额	
NET_INCR_DISP_FAAS	double	处置可供出售金融资产净增加额	

NET_INCR_DISP_TFA	double	处置交易性金融资产 净增加额	
NET_INCR_INSURED_SAVE	double	保户储金净增加额	
NET_INCR_INT_AND_CHARGE	double	收取利息和手续费净 增加额	
NET_INCR_LOANS_CENTRAL_BANK	double	向中央银行借款净增 加额	
NET_INCR_PLEDGE_LOAN	double	质押贷款净增加额	
NET_INCR_REPU_BUS_FUND	double	回购业务资金净增加 额	
NET_PROFIT	double	净利润	
OTH_CASH_PAY_INV_ACT	double	支付其他与投资活动 有关的现金	
OTH_CASH_PAY_OPERA_ACT	double	支付其他与经营活动 有关的现金	
OTH_CASH_REC_P_INV_ACT	double	收到其他与投资活动 有关的现金	
OTHER_ASSETS_IMPAIR_LOSS	double	其他资产减值损失	
OTHER_CASH_PAY_FIN_ACT	double	支付其他与筹资活动 有关的现金	
OTHER_CASH_REC_P_FIN_ACT	double	收到其他与筹资活动 有关的现金	
OTHER_CASH_REC_P_OPER_ACT	double	收到其他与经营活动 有关的现金	
OTHERS	double	其他（废弃）	
PAY_ALL_TAX	double	支付的各项税费	
PLUS_ASSETS_DEPRE_PREP	double	加：资产减值准备	
PLUS_END_BAL_CASH_EQU	double	加：现金等价物的期 末余额	
REC_P_TAX_REFUND	double	收到的税费返还	
REPORT_TYPE	string	报告期名称	
REPORTING_PERIOD	string	报告期	
SECURITY_NAME	string	证券简称	
SPE_BAL_CASH_INFLOW_FIN_ACT	double	筹资活动现金流入差 额	
SPE_BAL_CASH_INFLOW_INV_ACT	double	投资活动现金流入差 额	
SPE_BAL_CASH_INFLOW_OPERA_ACT	double	经营活动现金流入差 额	
SPE_BAL_CASH_OUTFLOW_FIN	double	筹资活动现金流出差 额	
SPE_BAL_CASH_OUTFLOW_INV	double	投资活动现金流出差 额	

SPE_BAL_CASH_OUTFLOW_OPERA	double	经营活动现金流出差额	
SPE_BAL_NETCASH_INC_DIFF_IND	double	间接法-现金净增加额差额	
SPE_BAL_NETCASH_INCR_DIFF	double	现金净增加额差额	
SPE_BAL_NETCASH_OPERA_IND	double	间接法-经营活动现金流量净额差额	
STATEMENT_TYPE	string	报表类型	参看报表类型代码表
TOT_BAL_CASH_INFLOW_FIN_ACT	double	筹资活动现金流入差额	
TOT_BAL_CASH_INFLOW_INV_ACT	double	投资活动现金流入差额	
TOT_BAL_CASH_INFLOW_OPERA_ACT	double	经营活动现金流入差额	
TOT_BAL_CASH_OUTFLOW_FIN	double	筹资活动现金流出差额	
TOT_BAL_CASH_OUTFLOW_INV	double	投资活动现金流出差额	
TOT_BAL_CASH_OUTFLOW_OPERA	double	经营活动现金流出差额	
TOT_BAL_NETCASH_FLOW_FIN	double	筹资活动产生的现金流量净额差额	
TOT_BAL_NETCASH_FLOW_INV	double	投资活动产生的现金流量净额差额	
TOT_BAL_NETCASH_FLOW_OPERA	double	经营活动产生的现金流量净额差额	
TOT_BAL_NETCASH_INC_DIFF_IND	double	间接法-现金净增加额差额	
TOT_BAL_NETCASH_INCR_DIFF	double	现金净增加额差额	
TOT_BAL_NETCASH_OPERA_IND	double	间接法-经营活动现金流量净额差额	
TOT_CASH_INFLOW_FIN_ACT	double	筹资活动现金流入小计	
TOT_CASH_INFLOW_INV_ACT	double	投资活动现金流入小计	
TOT_CASH_INFLOW_OPERA_ACT	double	经营活动现金流入小计	
TOT_CASH_OUTFLOW_FIN_ACT	double	筹资活动现金流出小计	
TOT_CASH_OUTFLOW_INV_ACT	double	投资活动现金流出小计	
TOT_CASH_OUTFLOW_OPERA_ACT	double	经营活动现金流出小计	

		计	
UNCONFIRMED_INV_LOSS	double	未确认投资损失	
USE_RIGHT_ASSET_DEP	double	使用权资产折旧	

示例数据:

```
{
  "code": "0",
  "msg": "",
  "body": {
    "data": [
      {
        "STATEMENT_TYPE": "2",
        "MARKET_CODE": "601881.SH",
        "CREDIT_IMPAIR_LOSS": null,
        "USE_RIGHT_ASSET_DEP": null,
        "ANN_DATE": "20221028",
        "REPORTING_PERIOD": "20220930",
        "CURRENCY_CODE": "CNY",
        "CASH_REC_P_SG_AND_RS": null,
        "REC_P_TAX_REFUND": null,
        "NET_INCR_DEP_CUS_AND_IB": null,
        "NET_INCR_LOANS_CENTRAL_BANK": null,
        "NET_INCR_BORR_OFI": null,
        "CASH_REC_P_PREM_OIC": null,
        "NET_INCR_INSURED_SAVE": null,
        "NET_CASH_REC_P_REINSU_BUS": null,
        "NET_INCR_DISP_TFA": null,
        "NET_INCR_INT_AND_CHARGE": 6000467745.01,
        "NET_INCR_DISP_FAAS": null,
        "NET_INCR_BORR_FUND": null,
        "NET_INCR_REPU_BUS_FUND": 4295138233.46,
        "OTHER_CASH_REC_P_OPER_ACT": 5585808937.36,
        "TOT_CASH_INFLOW_OPER_ACT": 6265329741.35,
        "CASH_PAY_GOODS_SERVICES": null,
        "CASH_PAY_EMPLOYEE": 2160155204.77,
        "PAY_ALL_TAX": 511803857.9,
        "NET_INCR_CUS_LOAN_ADV": null,
        "NET_INCR_DEP_CB_IB": null,
        "CASH_PAY_CLAIMS_OIC": null,
        "CASH_FOR_CHARGE": 1633998688.41,
        "CASH_PAID_INSUR_POLICY": null,
        "OTH_CASH_PAY_OPERA_ACT": 3879146517.93,
        "TOT_CASH_OUTFLOW_OPERA_ACT": 38229274925.76,
        "NET_CASH_FLOWS_OPERA_ACT": -31963945184.41,
        "CASH_REC_P_RECOV_INV": null,
        "CASH_REC_P_INV_INCOME": 2394058459.85,
        "NET_CASH_REC_P_DISP_FIOLTA": 523785.45,
        "NET_CASH_REC_P_DISP_SOBU": null,
        "OTH_CASH_REC_P_INV_ACT": 25461034119.98,
        "TOT_CASH_INFLOW_INV_ACT": 27848136279.43,
      }
    ]
  }
}
```

"CASH_PAID_PUR_CONST_FIOLTA":200453770.88,
"CASH_PAID_INV":18980000,
"NET_CASH_PAID_SOBU":null,
"OTH_CASH_PAY_INV_ACT":26208164308.11,
"NET_INCR_PLEDGE_LOAN":null,
"TOT_CASH_OUTFLOW_INV_ACT":27418557650.39,
"NET_CASH_FLOWS_INV_ACT":429578629.04,
"ABSORB_CASH_RECP_INV":null,
"INCL_CASH_RECP_SAIMS":null,
"CASH_RECE_BORROW":562189932.2,
"CASH_RECE_ISSUE_BONDS":null,
"OTHER_CASH_RECP_FIN_ACT":null,
"TOT_CASH_INFLOW_FIN_ACT":33337119932.2,
"CASH_PAY_FOR_DEBT":19080764918.92,
"CASH_PAY_DIST_DIV_PRO_INT":4580851536.52,
"INCL_DIV_PRO_PAID_SMS":null,
"OTHER_CASH_PAY_FIN_ACT":158409225.07,
"TOT_CASH_OUTFLOW_FIN_ACT":24034187288.75,
"NET_CASH_FLOWS_FIN_ACT":9302932643.45,
"EFF_FX_FLUC_CASH":414126108.24,
"NET_INCR_CASH_AND_CASH_EQU":-21817307803.68,
"BEG_BAL_CASH_CASH_EQU":136398080715.3,
"END_BAL_CASH_CASH_EQU":114580772911.62,
"NET_PROFIT":null,
"UNCONFIRMED_INV_LOSS":null,
"PLUS_ASSETS_DEPRE_PREP":null,
"DEPRE_FA_OGA_PBA":null,
"AMORT_INTAN_ASSETS":null,
"AMORT_LT_DEFERRED_EXP":null,
"DECR_DEFERRED_EXPENSE":null,
"INCR_ACCRUED_EXP":null,
"LOSS_DISP_FIOLTA":null,
"LOSS_FIXED_ASSETS":null,
"LOSS_FAIRVALUE_CHG":null,
"FINANCIAL_EXP":null,
"INV_LOSS":null,
"DECR_DEFE_INC_TAX_ASSETS":null,
"INCR_DEFE_INC_TAX_LIAB":null,
"DECR_INVENTORY":null,
"DECR_OPERA_RECEIVABLE":null,
"INCR_OPERA_PAYABLE":null,
"OTHERS":null,
"IND_NET_CASH_FLOWS_OPERA_ACT":null,
"CONV_DEBT_INTO_CAP":null,

```
"CONV_CORP_BONDS_DUE_WITHIN_1Y":null,
"FIXED_ASSETS_FIN_LEASE":null,
"CASH_END_BAL":null,
"LESS_OPEN_BAL_CASH":null,
"PLUS_END_BAL_CASH_EQU":null,
"LESS_OPEN_BAL_CASH_EQU":null,
"IND_NET_INCR_CASH_AND_EQU":null,
"FREE_CASH_FLOW":null,
"COMP_TYPE_CODE":"4",
"ACTUAL_ANN_DATE":"20221028",
"SPE_BAL_CASH_INFLOW_OPERA_ACT":-1857312536.05,
"TOT_BAL_CASH_INFLOW_OPERA_ACT":null,
"SPE_BAL_CASH_OUTFLOW_OPERA":31171853838.35,
"TOT_BAL_CASH_OUTFLOW_OPERA":null,
"TOT_BAL_NETCASH_FLOW_OPERA":null,
"SPE_BAL_CASH_INFLOW_INV_ACT":-7480085.85,
"TOT_BAL_CASH_INFLOW_INV_ACT":null,
"SPE_BAL_CASH_OUTFLOW_INV":990959571.4,
"TOT_BAL_CASH_OUTFLOW_INV":null,
"TOT_BAL_NETCASH_FLOW_INV":null,
"SPE_BAL_CASH_INFLOW_FIN_ACT":32774930000,
"TOT_BAL_CASH_INFLOW_FIN_ACT":null,
"SPE_BAL_CASH_OUTFLOW_FIN":214161608.24,
"TOT_BAL_CASH_OUTFLOW_FIN":null,
"TOT_BAL_NETCASH_FLOW_FIN":null,
"SPE_BAL_NETCASH_INCR_DIFF":null,
"TOT_BAL_NETCASH_INCR_DIFF":null,
"SPE_BAL_NETCASH_OPERA_IND":null,
"TOT_BAL_NETCASH_OPERA_IND":null,
"SPE_BAL_NETCASH_INC_DIFF_IND":null,
"TOT_BAL_NETCASH_INC_DIFF_IND":null,
"COMP_ID":"604498",
"NET_INCR_DISMANTLE_CAP":-1127683181.6,
"IS_CALCULATION":1,
"NET_CASH_REC_SEC":null,
"OTHER_ASSETS_IMPAIR_LOSS":null,
"SECURITY_NAME":"中国银河",
"REPORT_TYPE":"3"}]]}
```

7.5.4A 股财务指标 A010050004

7.5.4.1 请求参数说明

参数	是否必填	参数说明	备注
function_id	是	功能号	功能号为 A010050004
market_code	是	市场代码	代码.市场, 例如: 688105.SH
start_date	是	报告期查询起始日期	填写 0 表示查询最新日期的数据
end_date	是	报告期查询结束日期	填写 0 表示查询最新日期的数据

7.5.4.2 返回值说明

字段名称	类型	字段说明	备注
ADMIN_EXPENSE_TO_GR	double	管理费用/营业总收入	%
ANN_DATE	string	公告日期	
AR_TURN	double	应收账款周转率	次
AR_TURNDAYS	double	应收账款周转天数	天
ASSETS_TO_EQUITY	double	权益乘数	
ASSETS_TURN	double	总资产周转率	次
BPS	double	每股净资产	元
CA_TO_ASSETS	double	流动资产/总资产	%
CA_TURN	double	流动资产周转率	次
CAPITALIZED_TO_DA	double	资本支出/折旧和摊销	%
CASH_RATIO	double	保守速动比率	%
CASH_TO_LIQDEBT	double	货币资金 / 流动负债	%
CASH_TO_LIQDEBT_WITH_INT EREST	double	货币资金 / 带息流动 负债	%
CFPS	double	每股现金流量净额	元
COGS_TO_SALES	double	销售成本率	%
COMP_CODE	string	公司 ID	
CURRENCY_CODE	string	货币代码	默认为人民币
CURRENT_DEBT_TO_DEBT	double	流动负债/负债合计	%
CURRENT_RATIO	double	流动比率	%
DEBT_TO_ASSETS	double	资产负债率	%
DEBT_TO_EQUITY	double	产权比率	%
DEDUCTED_PROFIT	double	扣除非经常性损益后的 净利润(扣除少数股 东损益)	元
DEDUCTED_PROFIT_TO_PROFI T	double	扣除非经常损益后的 净利润/净利润	%

DUPONT_ASSETS_TO_EQUITY	double	权益乘数(用于杜邦分析)	
DUPONT_ROA	double	总资产净利率(杜邦分析)	%
EBIT	double	息税前利润	元, 反向法
EBIT_TO_GR	double	息税前利润/营业总收入	%
EBIT_TO_INTEREST	double	已获利息倍数(EBIT/利息费用)	%
EBITDA	double	息税折旧摊销前利润	元
EBITDA_TO_DEBT	double	息税折旧摊销前利润/负债合计	%
EBITPS	double	每股息税前利润	元
EPS_BASIC	double	基本每股收益	元
EPS_DILUTED	double	稀释每股收益	元
EPS_DILUTED2	double	期末摊薄每股收益	元
EQUITY_TO_INTEREST_DEBT	double	归属于母公司的股东权益/带息债务	%
EQUITY_TO_TOTALCAP	double	归属于母公司的股东权益/全部投入资本	%
EQUITY_TOD_EBT	double	归属于母公司的股东权益/负债合计	%
EXINTEREST_DEBT_CURRENT	double	无息流动负债	元
EXINTEREST_DEBT_NONCURRENT	double	无息非流动负债	元
EXPENSE_TO_SALES	double	销售期间费用率	%,
EXTRA_ORDINARY	double	非经常性损益	元
FA_TURN	double	固定资产周转率	次
FCFE	double	股权自由现金流量(FCFE)	元
FCFEPS	double	每股股东自由现金流量	元
FCFF	double	企业自由现金流量(FCFF)	元
FCFFPS	double	每股企业自由现金流量	元
FINA_EXPENSE_TO_GR	double	财务费用/营业总收入	%
GC_TO_GR	double	营业总成本/营业总收入	%
GROSS_MARGIN	double	毛利	元
GROSS_PROFIT_MARGIN	double	销售毛利率	%
GRPS	double	每股营业总收入	元
IMPAIR_TO_GR_TTM	double	资产减值损失/营业总	%

		收入	
INTDEBT_TO_TOTALCAP	double	带息债务/全部投入资本	%
INTEREST_DEBT	double	带息债务	元
INV_TURN	double	存货周转率	次
INV_TURNDAYS	double	存货周转天数	天
INVEST_CAPITAL	double	全部投入资本	元
INVEST_INCOME	double	价值变动净收益	元
INVEST_INCOME_TO_EBT	double	价值变动净收益/利润总额	%
LONG_DEBT_TO_DEBT	double	非流动负债/负债合计	%
LONGDEBT_TO_WORKING_CAPITAL	double	长期债务与营运资金比率	%
MARKET_CODE	string	代码	
NCA_TO_ASSETS	double	非流动资产/总资产	%
NET_DEBT	double	净债务	元
NET_PROFIT_MARGIN	double	销售净利率	%
NETWORKING_CAPITAL	double	营运流动资本	元
NONOP_PROFIT	double	非营业利润	元
NONOPERATE_PROFIT_TO_EBT	double	营业外收支净额/利润总额	%
NOP_TO_EBT	double	非营业利润 / 利润总额	%
OCF_TO_DEBT	double	经营活动产生的现金流量净额/负债合计	%
OCF_TO_INTEREST_DEBT	double	经营活动产生的现金流量净额/带息债务	%
OCF_TO_NETDEBT	double	经营活动产生的现金流量净额/净债务	%
OCF_TO_OPERATE_INCOME	double	经营活动产生的现金流量净额/经营活动净收益	%
OCF_TO_OR	double	经营活动产生的现金流量净额/营业收入	%
OCF_TO_PROFIT	double	经营活动产生的现金流量净额 / 营业利润	%
OCF_TO_SHORT_DEBT	double	经营活动产生的现金流量净额/流动负债	%
OCFPS	double	每股经营活动产生的现金流量净额	元
OP_TO_DEBT	double	营业利润 / 负债合计	%
OP_TO_EBT	double	营业利润 / 利润总额	%
OP_TO_GR	double	营业利润/营业总收入	%

OP_TO_LIQDEBT	double	营业利润 / 流动负债	%
OPERATE_INCOME	double	经营活动净收益	元
OPERATE_INCOME_TO_EBT	double	经营活动净收益/利润总额	%
ORPS	double	每股营业收入	元
PREFIN_EXPENSE_OPPROFIT	double	扣除财务费用前营业利润	元
PROFIT_TO_GR	double	净利润/营业总收入	%
PROFIT_TOOP	double	利润总额 / 营业收入	%
Q_ADMIN_EXPENSE_TO_GR	double	单季度, 管理费用 / 营业总收入	%
Q_CGR_GR	double	单季度, 营业总收入环比增长率(%)	%
Q_CGR_NETPROFIT	double	单季度, 归属母公司股东的净利润环比增长率(%)	%
Q_CGR_OP	double	单季度, 营业利润环比增长率(%)	%
Q_CGR_PROFIT	double	单季度, 净利润环比增长率(%)	%
Q_CGR_SALES	double	单季度, 营业收入环比增长率(%)	%
Q_DEDUCTED_PROFIT	double	单季度, 扣除非经常损益后的净利润	元
Q_DEDUCTED_PROFIT_TO_PROFIT	double	单季度, 扣除非经常损益后的净利润 / 净利润	%
Q_EPS	double	单季度, 每股收益	元
Q_EXPENSE_TO_SALES	double	单季度, 销售期间费用率	%, (销售费用+管理费用+财务费用+研发费用)/营业收入*100
Q_FINA_EXPENSE_TO_GR	double	单季度, 财务费用 / 营业总收入	%
Q_GC_TO_GR	double	单季度, 营业总成本 / 营业总收入	%
Q_GROSS_PROFIT_MARGIN	double	单季度, 销售毛利率	%
Q_IMPAIR_TO_GR_TTM	double	单季度, 资产减值损失 / 营业总收入	%
Q_INVEST_INCOME	double	单季度, 价值变动净收益	元
Q_INVEST_INCOME_TO_EBT	double	单季度, 价值变动净收益 / 利润总额	%
Q_NET_PROFIT_MARGIN	double	单季度, 销售净利率	%

Q_OCF_TO_OR	double	单季度, 经营活动产生的现金流量净额 / 经营活动净收益	%
Q_OCF_TO_SALES	double	单季度, 经营活动产生的现金流量净额 / 营业收入	%
Q_OP_TO_GR	double	单季度, 营业利润 / 营业总收入	%
Q_OPERATE_INCOME	double	单季度, 经营活动净收益	元
Q_OPERATE_INCOME_TO_EBT	double	单季度, 经营活动净收益 / 利润总额	%
Q_PROFIT_TO_GR	double	单季度, 净利润 / 营业总收入	%
Q_ROA	double	单季度, 总资产净利润	%
Q_ROE	double	单季度, 净资产收益率	%
Q_ROE_DEDUCTED	double	单季度, 净资产收益率 (扣除非经常损益)	%
Q_SALE_EXPENSE_TO_GR	double	单季度, 销售费用 / 营业总收入	%
Q_SALES_CASH_IN_TO_OR	double	单季度, 销售商品提供劳务收到的现金 / 营业收入	%
Q_YOY_EQUITY	double	净资产(同比增长率)	%
Q_YOY_GR	double	单季度, 营业总收入同比增长率(%)	%
Q_YOY_NETPROFIT	double	单季度, 归属母公司股东的净利润同比增长率(%)	%
Q_YOY_OP	double	单季度, 营业利润同比增长率(%)	%
Q_YOY_PROFIT	double	单季度, 净利润同比增长率(%)	%
Q_YOY_SALES	double	单季度, 营业收入同比增长率(%)	%
QUICK_RATIO	double	速动比率	%
RD_EXPENSE	double	研发费用	元
REPORT_PERIOD	string	报告期	
REPORT_TYPE	string	报告期名称	
RETAINED_EARNINGS	double	留存收益	元
RETAINED_PS	double	每股留存收益	元
ROA	double	总资产净利率	%
ROA_YEARLY	double	年化总资产净利率	%

ROA2	double	总资产报酬率	%
ROA2_YEARLY	double	年化总资产报酬率	%
ROE	double	净资产收益率	%
ROE_AVG	double	平均净资产收益率(增发条件)	%
ROE_DEDUCTED	double	净资产收益率(扣除非经常损益)	%
ROE_YEARLY	double	年化净资产收益率	%
ROIC	double	投入资本回报率	%
ROIC_YEARLY	double	年化投入资本回报率	%
SALE_EXPENSE_TO_GR	double	销售费用/营业总收入	%
SALES_CASH_IN_TO_OR	double	销售商品提供劳务收到的现金/营业收入	%
SECURITY_NAME	string	证券简称	
STM_BS	double	固定资产合计	元
STM_IS	double	折旧与摊销	元, 现金流量表的固定资产折旧、油气资产折耗、生产性生物资产折旧+无形资产摊销+长期待摊费用摊销+使用权资产折旧
STMNOTE_FINEXP	double	利息费用	元
SURPLUS_CAPITAL_PS	double	每股资本公积	元
SURPLUS_RESERVE_PS	double	每股盈余公积	元
TANG_ASSET_TO_INT_DEBT	double	有形资产/带息债务	%
TANGIBLE_ASSET	double	有形资产	元
TANGIBLE_ASSET_TO_DEBT	double	有形资产/负债合计	%
TANGIBLE_ASSET_TO_NETDEBT	double	有形资产/净债务	%
TANGIBLE_ASSETS_TO_ASSETS	double	有形资产/总资产	%
TAX_TO_EBT	double	所得税/利润总额	%
TOT_FATURN	double	固定资产合计周转率	次
TURNDAYS	double	营业周期	天
UNDISTRIBUTED_PS	double	每股未分配利润	元
WAA_ROE	double	加权平均净资产收益率	%
WORKING_CAPITAL	double	营运资金	元
YOY_ASSETS	double	相对年初增长率-资产总计(%)	%
YOY_BPS	double	相对年初增长率-每股净资产(%)	%
YOY_EBT	double	同比增长率-利润总额	%

		(%)	
YOY_EPS_BASIC	double	同比增长率-基本每股收益 (%)	%
YOY_EPS_DILUTED	double	同比增长率-稀释每股收益 (%)	%
YOY_EQUITY	double	相对年初增长率-归属母公司的股东权益 (%)	%
YOY_NETPROFIT	double	同比增长率-归属母公司股东的净利润 (%)	%
YOY_NETPROFIT_DEDUCTED	double	同比增长率-归属母公司股东的净利润-扣除非经常损益 (%)	%
YOY_OCF	double	同比增长率-经营活动产生的现金流量净额 (%)	%
YOY_OCFPS	double	同比增长率-每股经营活动产生的现金流量净额 (%)	%
YOY_OP	double	同比增长率-营业利润 (%)	%
YOY_OR	double	营业收入同比增长率 (%)	%
YOY_ROE	double	同比增长率-净资产收益率 (摊薄) (%)	%
YOY_TR	double	营业总收入同比增长率 (%)	%
TYPE_CODE	string	证券类别	参考证券类别代码表
TYPE_CODE_INT	int32	证券类别代码	参考证券类别代码表
EXCHANGE_STR	string	交易市场	参考交易市场代码表
EXCHANGE_INT	int32	交易市场代码	参考交易市场代码表

示例数据:

```
{
  "code": "0",
  "msg": "",
  "body": {
    "data": [
      {
        "Q_CGR_NETPROFIT": -23.7012,
        "MARKET_CODE": "601881.SH",
        "REPORT_PERIOD": "20220930",
        "Q_YOY_EQUITY": 7.2878,
        "RD_EXPENSE": null,
        "WAA_ROE": 6.23,
        "COMP_CODE": "604498",
        "ANN_DATE": "20221028",
        "CURRENCY_CODE": "CNY",
        "EXTRA_ORDINARY": 6530760.03,
        "DEDUCTED_PROFIT": 6433882999.96,
        "GROSS_MARGIN": null,
      }
    ]
  }
}
```

"OPERATE_INCOME":-10238282643.69,
"INVEST_INCOME":4861512070.79,
"STMNOTE_FINEXP":null,
"STM_IS":null,
"EBIT":null,
"EBITDA":null,
"FCFF":null,
"FCFE":null,
"EXINTEREST_DEBT_CURRENT":null,
"EXINTEREST_DEBT_NONCURRENT":null,
"INTEREST_DEBT":null,
"NET_DEBT":null,
"TANGIBLE_ASSET":null,
"WORKING_CAPITAL":null,
"NETWORKING_CAPITAL":null,
"INVEST_CAPITAL":null,
"RETAINED_EARNINGS":37052458207.74,
"EPS_BASIC":0.59,
"EPS_DILUTED":0.56,
"EPS_DILUTED2":0.6353,
"BPS":8.6033,
"OCFPS":2.3565,
"GRPS":2.6909,
"ORPS":2.6909,
"SURPLUS_CAPITAL_PS":2.4712,
"SURPLUS_RESERVE_PS":0.7628,
"UNDISTRIBUTED_PS":2.8923,
"RETAINED_PS":3.6551,
"CFPS":-0.7827,
"EBITPS":null,
"FCFFPS":null,
"FCFEPS":null,
"NET_PROFIT_MARGIN":23.6123,
"GROSS_PROFIT_MARGIN":null,
"COGS_TO_SALES":null,
"EXPENSE_TO_SALES":null,
"PROFIT_TO_GR":23.6123,
"SALE_EXPENSE_TO_GR":null,
"ADMIN_EXPENSE_TO_GR":26.4165,
"FINA_EXPENSE_TO_GR":null,
"IMPAIR_TO_GR_TTM":null,
"GC_TO_GR":null,
"OP_TO_GR":26.4575,
"EBIT_TO_GR":null,

"ROE":6.3733,
"ROE_DEDUCTED":6.3668,
"ROA2":null,
"ROA":1.0894,
"ROIC":null,
"ROE_YEARLY":8.4977,
"ROA2_YEARLY":null,
"ROE_AVG":null,
"OPERATE_INCOME_TO_EBT":-141.7019,
"INVEST_INCOME_TO_EBT":67.2853,
"NONOPERATE_PROFIT_TO_EBT":0.1119,
"TAX_TO_EBT":10.8534,
"DEDUCTED_PROFIT_TO_PROFIT":99.8986,
"SALES_CASH_IN_TO_OR":null,
"OCF_TO_OR":0.8757,
"OCF_TO_OPERATE_INCOME":null,
"CAPITALIZED_TO_DA":null,
"DEBT_TO_ASSETS":83.4213,
"ASSETS_TO_EQUITY":6.0318,
"DUPONT_ASSETS_TO_EQUITY":5.8507,
"CA_TO_ASSETS":null,
"NCA_TO_ASSETS":null,
"TANGIBLE_ASSETS_TO_ASSETS":null,
"INTDEBT_TO_TOTALCAP":null,
"EQUITY_TO_TOTALCAP":null,
"CURRENT_DEBT_TO_DEBT":null,
"LONG_DEBT_TO_DEBT":null,
"CURRENT_RATIO":null,
"QUICK_RATIO":null,
"CASH_RATIO":null,
"OCF_TO_SHORT_DEBT":null,
"DEBT_TO_EQUITY":5.0318,
"EQUITY_TO_D_EBT":0.1987,
"EQUITY_TO_INTEREST_DEBT":null,
"TANGIBLE_ASSET_TO_DEBT":null,
"TANG_ASSET_TO_INT_DEBT":null,
"TANGIBLE_ASSET_TO_NETDEBT":null,
"OCF_TO_DEBT":0.046,
"OCF_TO_INTEREST_DEBT":null,
"OCF_TO_NETDEBT":null,
"EBIT_TO_INTEREST":null,
"LONGDEBT_TO_WORKING_CAPITAL":null,
"EBITDA_TO_DEBT":null,
"TURNDAYS":null,

"INV_TURNDAYS":null,
"AR_TURNDAYS":null,
"INV_TURN":null,
"AR_TURN":null,
"CA_TURN":null,
"FA_TURN":null,
"ASSETS_TURN":0.0461,
"ROA_YEARLY":1.4525,
"DUPONT_ROA":1.0893,
"STM_BS":528727787.09,
"PREFIN_EXPENSE_OP PROFIT":null,
"NONOP_PROFIT":8082393.69,
"OP_TO_EBT":99.8881,
"NOP_TO_EBT":0.1119,
"OCF_TO_PROFIT":330.9967,
"CASH_TO_LIQDEBT":null,
"CASH_TO_LIQDEBT_WITH_INTEREST":null,
"OP_TO_LIQDEBT":null,
"OP_TO_DEBT":0.0139,
"ROIC_YEARLY":null,
"TOT_FATURN":55.737,
"PROFIT_TOOP":26.4871,
"Q_OPERATE_INCOME":-3495536792.91,
"Q_INVEST_INCOME":1443182419.9,
"Q_DEDUCTED_PROFIT":2110141532.4,
"Q_EPS":0.2086,
"Q_NET_PROFIT_MARGIN":23.3148,
"Q_GROSS_PROFIT_MARGIN":null,
"Q_EXPENSE_TO_SALES":null,
"Q_PROFIT_TO_GR":23.3148,
"Q_SALE_EXPENSE_TO_GR":null,
"Q_ADMIN_EXPENSE_TO_GR":26.6082,
"Q_FINA_EXPENSE_TO_GR":null,
"Q_IMPAIR_TO_GR_TTM":null,
"Q_GC_TO_GR":null,
"Q_OP_TO_GR":24.5021,
"Q_ROE":2.0738,
"Q_ROE_DEDUCTED":2.0698,
"Q_ROA":0.3416,
"Q_OPERATE_INCOME_TO_EBT":-156.9778,
"Q_INVEST_INCOME_TO_EBT":64.8105,
"Q_DEDUCTED_PROFIT_TO_PROFIT":99.8085,
"Q_SALES_CASH_IN_TO_OR":null,
"Q_OCF_TO_SALES":-352.4979,

```

"Q_OCF_TO_OR":null,
"YOY_EPS_BASIC":-14.49,
"YOY_EPS_DILUTED":-18.84,
"YOY_OCFPS":-47.0711,
"YOY_OP":-22.6753,
"YOY_EBT":-21.9256,
"YOY_NETPROFIT":-12.97,
"YOY_NETPROFIT_DEDUCTED":-13.58,
"YOY_OCF":-47.07,
"YOY_ROE":-18.8863,
"YOY_BPS":3.7392,
"YOY_ASSETS":11.1025,
"YOY_EQUITY":4.24,
"YOY_TR":4.5026,
"YOY_OR":4.5026,
"Q_YOY_GR":-17.2885,
"Q_CGR_GR":-16.605,
"Q_YOY_SALES":-17.2885,
"Q_CGR_SALES":-16.605,
"Q_YOY_OP":-43.1532,
"Q_CGR_OP":-32.4805,
"Q_YOY_PROFIT":-33.5891,
"Q_CGR_PROFIT":-23.721,
"Q_YOY_NETPROFIT":-32.7922,
"SECURITY_NAME":"中国银河",
"REPORT_TYPE":"3",
"TYPE_CODE":"A",
"TYPE_COUNT_INT":1001,
"EXCHANGE_STR":"SSE",
"EXCHANGE_INT":101}}}}

```

7.5.5A 股业绩预告 A010050005

7.5.5.1 请求参数说明

参数	是否必填	参数说明	备注
function_id	是	功能号	功能号为 A010050005
market_code	是	市场代码	代码.市场, 例如: 688105.SH
start_date	是	报告期查询起始日期	填写 0 表示查询最新日期的数据
end_date	是	报告期查询结束日期	填写 0 表示查询最新日期的数据

7.5.5.2 返回值说明

字段名称	类型	字段说明	备注
P_TYPECODE	string	业绩预告类型代码	1: 不确定 2: 略减 3: 略增 4: 扭亏 5: 其他 6: 首亏 7: 续亏 8: 续盈 9: 预减 10: 预增 11: 持平
REPORTING_PERIOD	string	报告期	分为年度、半年度、季度
ANN_DATE	string	公告日期	公告发布当天的日期
P_SIGN_CHANGE	string	是否变脸	1: 是; 0: 否
P_CHANGE_MAX	double	预告净利润变动幅度上限 (%)	对于净利润金额同比变动幅度预计的最高值
P_CHANGE_MIN	double	预告净利润变动幅度下限 (%)	对于净利润金额同比变动幅度预计的最低值
NET_PROFIT_MAX	int32	预告净利润上限 (万元)	对于净利润金额预计的最高值
NET_PROFIT_MIN	int32	预告净利润下限 (万元)	对于净利润金额预计的最低值
FIRST_ANN_DATE	string	首次公告日	首次披露本报告期业绩预告内容的公告日期
P_NUMBER	int32	公布次数	同一报告期的业绩预告公告的披露次数
P_REASON	string	业绩变动原因	
P_SUMMARY	string	业绩预告摘要	
P_NET_PARENT_FIRM	double	上年同期归母净利润	业绩预告中直接公布的上年同期归母净利润
MARKET_CODE	string	市场代码	
SECURITY_NAME	string	证券简称	
REPORT_TYPE	string	报告期名称	1: 一季度报表 2: 中期报表 3: 三季度报表

4: 年度报表

示例数据:

```
{
  "code": "0",
  "msg": "",
  "body": {
    "data": [
      {
        "P_TYPECODE": "10",
        "REPORTING_PERIOD": "20220630",
        "ANN_DATE": "20220722",
        "P_SIGN_CHANGE": "0",
        "P_CHANGE_MAX": 60.77,
        "P_CHANGE_MIN": 53.12,
        "NET_PROFIT_MAX": 63000,
        "NET_PROFIT_MIN": 60000,
        "FIRST_ANN_DATE": "20220722",
        "P_NUMBER": 1,
        "P_REASON": "2022 年上半年度，公司积极应对新冠疫情不利影响，常规业务各细分领域业务有序开展，在保持优势产品与服务销售增加的同时，进一步实现成长性业务领域产品与服务的销售转化。公司常规业务整体保持稳定增长。进入 2022 年以来，新冠病毒的全球扩散与持续突变为境内外疫情防控带来不断挑战，国务院应对新冠疫情联防联控机制综合组印发了《新冠病毒抗原检测应用方案》，同时全国多地动态开展常态化新冠核酸检测。2022 年上半年度，我国境内对新冠检测相关生物试剂原料与终端检测试剂需求相对去年同期大幅增加，公司自主研发生产的新冠病毒抗原检测试剂盒及相关生物试剂原料的销售促进报告期内营业收入与净利润的增长。2022 年半年度，预计公司非经常性损益金额较上年同期增幅较大，主要系公司针对暂时闲置资金进行现金管理的资金收益较上年同期大幅增加所致。",
        "P_SUMMARY": "预计:净利润 60000-63000",
        "P_NET_PARENT_FIRM": 39185.8,
        "MARKET_CODE": "688105.SH",
        "SECURITY_NAME": "诺唯赞",
        "REPORT_TYPE": "2"
      }
    ]
  }
}
```

7.5.6A 股财务衍生指标 A010050006

7.5.6.1 请求参数说明

参数	是否必填	参数说明	备注
function_id	是	功能号	功能号为 A010050006
market_code	是	市场代码	代码.市场，例如: 688105.SH
start_date	是	报告期截止日期的查询起始日期	填写 0 表示查询最新报告期的数据
end_date	是	报告期截止日期的查询结束日期	填写 0 表示查询最新报告期的数据

7.5.6.2 返回值说明

字段名称	类型	字段说明	备注
TANGIBLE_ASSET_TO_DEBT	double	有形资产 / 负债合计	有形资产与总负债的比率
OCF_TO_DEBT	double	经营活动产生的现金流量净额 / 负债合计	经营活动产生的现金流净额与总负债的比率
TANGIBLE_ASSET_TO_NETDEBT1	double	有形资产 / 净债务	有形资产与净债务的比率
TANGIBLE_ASSET_TO_NET_DEBT2	double	经营活动产生的现金流量净额 / 净债务	经营活动产生的现金流净额与净债务的比率
OCF_TO_INTEREST_DEBT	double	经营活动产生的现金流量净额 / 带息债务	经营活动产生的现金流净额与带息债务的比率
EBIT_TO_INTEREST	double	已获利息倍数(EBIT / 利息费用)	息税前利润与利息费用的占比
LONG_DEBT_TO_WORKINGCAPITAL	double	长期债务与营运资金比率	一年以上的债务与营运资金的比率
TURN_DAYS	int32	营业周期(天)	
EBITDA_TO_DEBT	double	息税折旧摊销前利润 / 负债合计	息税折旧摊销前利润与总负债的比率
ARTURN_DAYS	int32	应收账款周转天数(天)	
INVTURN_DAYS	int32	存货周转天数(天)	
AR_TURN	int32	应收账款周转率(次)	
INV_TURN	int32	存货周转率(次)	
FA_TURN	int32	固定资产周转率(次)	
CA_TURN	int32	流动资产周转率(次)	
OCF_TO_PROFIT	double	经营活动产生的现金流量净额 / 营业利润(含)(%)	
ASSETS_TURN	int32	总资产周转率(次)	
CA_SH_TO_LIQDEBT_WITH_INTEREST	double	货币资金 / 带息流动负债	货币资金与带息流动负债的比率
CA_SH_TO_LIQDEBT	double	货币资金 / 流动负债	货币资金与流动负债的比率
OP_TO_DEBT	double	营业利润 / 负债合计	营业利润与总负债的比率
OP_TO_LIQDEBT	double	营业利润 / 流动负债	营业利润与流动负债的比率
NET_PROFIT5	double	归属母公司的净利润 / 净利润(%)	

PROFIT_TO_OP	double	利润总额 / 营业总收入 (%)	
TOTAL_PROFIT_EBIT	double	利润总额 / EBIT (%)	
NET_TOTAL_PROFIT	double	净利润 / 利润总额 (%)	
YOY_EPS_DILUTED	double	稀释每股收益同比增长率 (%)	
YOY_EPS_BASIC	double	基本每股收益同比增长率 (%)	
YOY_TR	double	营业总收入同比增长率 (%)	
YOY_OCF_PS	double	每股经营活动产生的现金流量净额同比增长率 (%)	
YOY_OP	double	营业利润同比增长率 (含) (%)	
YOY_OR	double	营业收入同比增长率 (%)	
YOY_NET_PROFIT	double	归属母公司股东的净利润同比增长率 (%)	
YOY_EBT	double	利润总额同比增长率 (%)	
YOY_OCF	double	经营活动产生的现金流量净额同比增长率 (%)	
YOY_NET_PROFIT_DEDUCTED	double	归属母公司股东的净利润-扣除非经常损益同比增长率 (%)	
YOY_BPS	double	每股净资产同比增长率 (%)	
YOY_ROE	double	净资产收益率 (平均) 同比增长率 (%)	
YOY_EQUITY	double	归属母公司的股东权益同比增长率 (%)	
YOY_ASSETS	double	资产总计同比增长率 (%)	
END_DATE	string	截止日期	报告期截止的日期
START_DATE	string	起始日期	报告期开始的日期
FISCAL_YEAR	string	会计年度	截止日期所处年份
INTERVAL_LENGTH	int32	区间长度 (月)	报告期包含的月份数量
STATEMENT_TYPE	string	报表类型	参看报表类型代码表
TOTAL_SHARE	int32	期末总股本 (股)	股
CURRENCY_CODE	string	货币代码	
OPERATE_INCOME	double	经营活动净收益 (元)	
OP_PROFIT1	double	营业利润 (含价值变动损益) (元)	
STM_IS	double	折旧与摊销 (元)	
INVEST_INCOME	double	价值变动净收益 (元)	
EBITDA	double	息税折旧摊销前利润 (元)	
EBIT	double	息税前利润 (元)	
FCFE	double	股权自由现金流量 (FCFE) (元)	
FCFF	double	企业自由现金流量 (FCFF) (元)	
EXINTEREST_DEBT_NO	double	无息非流动负债 (元)	

NCURRENT			
EXINTEREST_DEBT_CURR RENT	double	无息流动负债(元)	
NET_DEBT	double	净债务(元)	
INTEREST_DEBT	double	带息债务(元)	
WORKING_CAPITAL	double	营运资金(元)	
TANGIBLE_ASSET	double	有形资产(元)	
INVEST_CAPITAL	double	全部投入资本(元)	
NETWORKING_CAPITAL	double	营运流动资本(元)	
EPS_BASIC	double	基本每股收益(元)	
RETAINED_EARNINGS	double	留存收益(元)	
EPS_DILUTED2	double	每股收益(期末摊薄)(元)	
EPS_DILUTED	double	稀释每股收益(元)	
BPS	double	每股净资产(元)	
EPS_DILUTED3	double	每股收益(扣除 / 期末股本摊薄)(元)	
GR_PS	double	每股营业总收入(元)	
OCF_PS	double	每股经营活动产生的现金流量净额(元)	
SURPLUS_CAPITAL_PS	double	每股资本公积(元)	
OR_PS	double	每股营业收入(元)	
CFPS	double	每股现金流量净额(元)	
RETAINED_PS	double	每股留存收益(元)	
FCFF_PS	double	每股企业自由现金流量(元)	
EBIT_PS	double	每股息税前利润(元)	
NET_PROFIT_MARGIN	double	销售净利率(%)	
FCFE_PS	double	每股股东自由现金流量(元)	
COGS_TO_SALES	double	销售成本率(%)	
GROSS_PROFIT_MARGIN	double	销售毛利率(%)	
SALE_EXPENSE_TO_GR	double	销售费用 / 营业总收入(%)	
PROFIT_TO_GR	double	净利润 / 营业总收入(%)	
FINA_EXPENSE_TO_GR	double	财务费用 / 营业总收入(%)	
ADMIN_EXPENSE_TO_G R	double	行政(管理)费用 / 营业总收入(%)	
OP_TO_GR	double	营业利润(含价值变动损益) / 营业总收入(%)	
GC_TO_GR	double	营业总成本 / 营业总收入(%)	
ROE	double	净资产收益率(平均)(%)	
EBIT_TO_GR	double	息税前利润 / 营业总收入(%)	
ROA2	double	总资产报酬率(平均)(%)	
ROE_DEDUCTED	double	净资产收益率(扣除平均)(%)	
ROIC	double	投入资本回报率(平均)(%)	

ROA	double	总资产净利润(平均)(%)	
ROA2_YEARLY	double	年化总资产报酬率(%)	
ROE_YEARLY	double	年化净资产收益率(%)	
ROIC_YEARLY	double	年化投入资本回报率	
ROA_YEARLY	double	年化总资产净利率(%)	
INVEST_INCOME_TO_EBT	double	价值变动净收益 / 除税前利润(%)	
OPERATE_INCOME_TO_EBT	double	经营活动净收益 / 除税前利润(%)	
TAX_TO_EBT	double	所得税 / 利润总额(%)	
NONOPERATE_PROFIT_TO_EBT	double	营业外收支净额 / 除税前利润(%)	
NONNET_OP_TO_TAX_PROFIT	double	非持续经营净利润/除税后利润(%)	
CONTINUED_NET_PROFIT	double	持续经营净利润 / 除税后利润(%)	
OCF_TO_OR1	double	经营活动产生的现金流量净额 / 经营活动净收益	经营活动产生的现金流净额与经营活动净收益的比率
OCF_TO_OR	double	经营活动产生的现金流量净额 / 营业收入(%)	
DEBT_TO_ASSETS	double	资产负债率(%)	
CAPITALIZED_TO_DA	double	资本支出 / 折旧和摊销	资本支出与折旧摊销的比率
DUPONT_ASSETS_TO_EQUITY	double	权益乘数(用于杜邦分析)	平均总资产与平均归属母公司权益的比率
ASSETS_TO_EQUITY	double	权益乘数	股东权益比例的倒数
NCA_TO_ASSETS	double	非流动资产 / 总资产(%)	
CA_TO_ASSETS	double	流动资产 / 总资产(%)	
TANGIBLE_ASSETS_TO_ASSETS	double	有形资产 / 总资产(%)	
INTDEBT_TO_TOTALCAP	double	带息债务 / 全部投入资本(%)	
CURRENT_DEBT_TO_DEBT	double	流动负债 / 负债合计(%)	
EQUITY_TO_TOTALCAPITAL	double	归属于母公司的股东权益 / 全部投入资本(%)	
CURRENT1	double	流动比率	流动资产与流动负债的比率
LONG_DEBT_TO_DEBT	double	非流动负债 / 负债合计(%)	
CA_SH_RATIO	double	保守速动比率	现金、短期投资、应收账款和票据的合计值与流动负债的比率

QUICK	double	速动比率	扣除存货的流动资产与流动负债的比率
DEBT_TO_EQUITY	double	产权比率	总负债与总权益的比率
OCF_TO_SHORT_DEBT	double	经营活动产生的现金流量净额 / 流动负债	经营活动产生的现金流净额与流动负债的比率
EQUITY_TO_INTEREST_DEBT	double	归属于母公司的股东权益 / 带息债务	归属于母公司的股东权益与带息债务的比率
EQUITY_TO_DEBT	double	归属于母公司的股东权益 / 负债合计	归属于母公司的股东权益与总负债的比率
TANG_ASSET_TO_INTDEBT	double	有形资产 / 带息债务	有形资产与带息债务的比率
MARKET_CODE	string	市场代码	
SECURITY_NAME	string	证券简称	
REPORT_TYPE	string	报告期名称	年报、一季报、中报、三季报

示例数据：

```
{
  "code": "0",
  "msg": "",
  "body": {
    "data": [
      {
        "MARKET_CODE": "688105.SH",
        "TANGIBLE_ASSET_TO_DEBT": 2.1261,
        "OCF_TO_DEBT": 0.0257,
        "TANGIBLE_ASSET_TO_NETDEBT1": -6.7032,
        "TANGIBLE_ASSET_TO_NET_DEBT2": -0.0812,
        "OCF_TO_INTEREST_DEBT": 0.0368,
        "EBIT_TO_INTEREST": null,
        "LONG_DEBT_TO_WORKINGCAPITAL": 0.0691,
        "TURN_DAYS": 623.1148,
        "EBITDA_TO_DEBT": -0.0102,
        "ARTURN_DAYS": 177.9008,
        "INVTURN_DAYS": 445.214,
        "AR_TURN": 1.0118,
        "INV_TURN": 0.4043,
        "FA_TURN": 1.0826,
        "CA_TURN": 0.1283,
        "OCF_TO_PROFIT": -18.948,
        "ASSETS_TURN": 0.0966,
        "CA_SH_TO_LIQDEBT_WITH_INTEREST": 1.4537,
        "CA_SH_TO_LIQDEBT": 1.1406,
        "OP_TO_DEBT": -0.1359,
        "OP_TO_LIQDEBT": -0.1525,
        "NET_PROFIT5": 98.9078,
        "PROFIT_TO_OP": -18.8187,
      }
    ]
  }
}
```

"TOTAL_PROFIT_EBIT":100,
"NET_TOTAL_PROFIT":76.658,
"YOY_EPS_DILUTED":-113.0719,
"YOY_EPS_BASIC":-113.0719,
"YOY_TR":-64.7596,
"YOY_OCF_PS":-65.8529,
"YOY_OP":-139.1102,
"YOY_OR":-64.7596,
"YOY_NET_PROFIT":-113.2087,
"YOY_EBT":-115.2099,
"YOY_OCF":-65.8487,
"YOY_NET_PROFIT_DEDUCTED":-117.7262,
"YOY_BPS":-12.8632,
"YOY_ROE":-15.9097,
"YOY_EQUITY":-12.8628,
"YOY_ASSETS":-4.151,
"COMP_CODE":"3v06MdXX0",
"END_DATE":"20230630",
"START_DATE":"20230101",
"FISCAL_YEAR":"2023",
"INTERVAL_LENGTH":6,
"STATEMENT_TYPE":"合并报表",
"REPORT_TYPE":"中报",
"TOTAL_SHARE":400010000,
"CURRENCY_CODE":"CNY",
"OPERATE_INCOME":-226865561.87,
"OP_PROFIT1":-236288062.67,
"STM_IS":89263726.48,
"INVEST_INCOME":0,
"EBITDA":-17690544.43,
"EBIT":-106954270.91,
"FCFE":1370614974.43,
"FCFF":497794305.53,
"EXINTEREST_DEBT_NONCURRENT":189686410.06,
"EXINTEREST_DEBT_CURRENT":333651374.27,
"NET_DEBT":-551554756.12,
"INTEREST_DEBT":1215621605.05,
"WORKING_CAPITAL":2746799617.92,
"TANGIBLE_ASSET":3697180000.71,
"INVEST_CAPITAL":5231696603.36,
"NETWORKING_CAPITAL":2195244861.8,
"EPS_BASIC":-0.2,
"RETAINED_EARNINGS":998480725.03,
"EPS_DILUTED2":-0.2027,

"EPS_DILUTED":-0.2,
"BPS":10.0399,
"EPS_DILUTED3":-0.2676,
"GR_PS":1.4208,
"OCF_PS":0.1119,
"SURPLUS_CAPITAL_PS":6.0522,
"OR_PS":1.4208,
"CFPS":1.0564,
"RETAINED_PS":2.4961,
"FCFF_PS":1.2445,
"EBIT_PS":-0.2674,
"NET_PROFIT_MARGIN":-14.426,
"FCFE_PS":3.4265,
"COGS_TO_SALES":27.8437,
"GROSS_PROFIT_MARGIN":72.1563,
"SALE_EXPENSE_TO_GR":39.9551,
"PROFIT_TO_GR":-14.426,
"FINA_EXPENSE_TO_GR":-1.6579,
"ADMIN_EXPENSE_TO_GR":20.4209,
"OP_TO_GR":-41.5751,
"GC_TO_GR":139.9172,
"ROE":-1.8804,
"EBIT_TO_GR":-18.8187,
"ROA2":-1.8188,
"ROE_DEDUCTED":-2.4818,
"ROIC":-1.5946,
"ROA":-1.3942,
"ROA2_YEARLY":-3.6376,
"ROE_YEARLY":-3.7608,
"ROIC_YEARLY":-3.1892,
"ROA_YEARLY":-2.7884,
"INVEST_INCOME_TO_EBT":0,
"OPERATE_INCOME_TO_EBT":212.1145,
"TAX_TO_EBT":23.342,
"NONOPERATE_PROFIT_TO_EBT":-87.2272,
"NONNET_OP_TO_TAX_PROFIT":null,
"CONTINUED_NET_PROFIT":131.9808,
"OCF_TO_OR1":-19.735,
"OCF_TO_OR":7.8776,
"DEBT_TO_ASSETS":30.2116,
"CAPITALIZED_TO_DA":1.9462,
"DUPONT_ASSETS_TO_EQUITY":1.3636,
"ASSETS_TO_EQUITY":1.4329,
"NCA_TO_ASSETS":25.3628,

"CA_TO_ASSETS":74.6372,
 "TANGIBLE_ASSETS_TO_ASSETS":64.2325,
 "INTDEBT_TO_TOTALCAP":23.2357,
 "CURRENT_DEBT_TO_DEBT":89.092,
 "EQUITY_TO_TOTALCAPITAL":76.7643,
 "CURRENT1":2.773,
 "LONG_DEBT_TO_DEBT":10.908,
 "CA_SH_RATIO":2.3579,
 "QUICK":2.5027,
 "DEBT_TO_EQUITY":0.433,
 "OCF_TO_SHORT_DEBT":0.0289,
 "EQUITY_TO_INTEREST_DEBT":3.3037,
 "EQUITY_TO_DEBT":2.3095,
 "TANG_ASSET_TO_INTDEBT":3.0414,
 "SECURITY_NAME":"诺唯赞",
 "TYPE_CODE":"A",
 "TYPE_COUNT_INT":1001,
 "EXCHANGE_STR":"SSE",
 "EXCHANGE_INT":101}}}}

7.6 交易数据

7.6.1 交易日历(仅京沪深 A 股) A010060001

7.6.1.1 请求参数说明

参数	是否必填	参数说明	备注
function_id	是	功能号	功能号为 A010060001
start_date	是	开始日期	格式为 yyyyymmdd 例如: 20220819
end_date	是	结束日期	格式为 yyyyymmdd 例如: 20220827
market	否	证券市场	默认全市场, 多市场之间用英文逗号分隔

说明: 开始时间和结束时间不能超过 30 天。

7.6.1.2 返回值说明

字段名称	类型	字段说明	备注
EXCHANGE	string	交易市场	参考交易市场代码表
EXCHANGE_INT	int32	交易市场代码	参考交易市场代码表
TRADE_DAYS	string	交易日期	
LAST_TRADE_DAYS	string	上一个交易日期	

NEXT_TRADE_DAYS	string	下一个交易日期	
-----------------	--------	---------	--

示例数据:

```
{
  "code": "0",
  "msg": "",
  "body": {
    "data": [
      {
        "NEXT_TRADE_DAYS": "20220822",
        "EXCHANGE_INT": 101,
        "LAST_TRADE_DAYS": "20220818",
        "TRADE_DAYS": "20220819",
        "EXCHANGE": "SSE"
      }, {
        "NEXT_TRADE_DAYS": "20220823",
        "EXCHANGE_INT": 101,
        "LAST_TRADE_DAYS": "20220819",
        "TRADE_DAYS": "20220822",
        "EXCHANGE": "SSE"
      }, {
        "NEXT_TRADE_DAYS": "20220824",
        "EXCHANGE_INT": 101,
        "LAST_TRADE_DAYS": "20220822",
        "TRADE_DAYS": "20220823",
        "EXCHANGE": "SSE"
      }, {
        "NEXT_TRADE_DAYS": "20220825",
        "EXCHANGE_INT": 101,
        "LAST_TRADE_DAYS": "20220823",
        "TRADE_DAYS": "20220824",
        "EXCHANGE": "SSE"
      }, {
        "NEXT_TRADE_DAYS": "20220826",
        "EXCHANGE_INT": 101,
        "LAST_TRADE_DAYS": "20220824",
        "TRADE_DAYS": "20220825",
        "EXCHANGE": "SSE"
      }, {
        "NEXT_TRADE_DAYS": "20220829",
        "EXCHANGE_INT": 101,
        "LAST_TRADE_DAYS": "20220825",
        "TRADE_DAYS": "20220826",
        "EXCHANGE": "SSE"
      }
    ]
  }
}
```

7.6.2 交易日历(含期货期权) A010061001

7.6.2.1 请求参数说明

参数	是否必填	参数说明	备注
function_id	是	功能号	功能号为 A010061001
start_date	是	开始日期	格式为 yyyyymmdd 例如: 20220819
end_date	是	结束日期	格式为 yyyyymmdd 例如: 20220827

market	否	证券市场	默认全市场，多市场之间用英文逗号分隔
--------	---	------	--------------------

说明：开始时间和结束时间不能超过 30 天。

7.6.2.2 返回值说明

字段名称	类型	字段说明	备注
EXCHANGE	string	交易市场	参考交易市场代码表
EXCHANGE_INT	int32	交易市场代码	参考交易市场代码表
TRADE_DAYS	string	交易日期	
LAST_TRADE_DAYS	string	上一个交易日期	
NEXT_TRADE_DAYS	string	下一个交易日期	

示例数据：

```
{
  "code": "0",
  "msg": "",
  "body": {
    "data": [
      {
        "NEXT_TRADE_DAYS": "20230601",
        "EXCHANGE_INT": 4,
        "LAST_TRADE_DAYS": "20230530",
        "TRADE_DAYS": "20230531",
        "EXCHANGE": "CFFEX"
      }, {
        "NEXT_TRADE_DAYS": "20230602",
        "EXCHANGE_INT": 4,
        "LAST_TRADE_DAYS": "20230531",
        "TRADE_DAYS": "20230601",
        "EXCHANGE": "CFFEX"
      }, {
        "NEXT_TRADE_DAYS": "20220601",
        "EXCHANGE_INT": 6,
        "LAST_TRADE_DAYS": "20220530",
        "TRADE_DAYS": "20220531",
        "EXCHANGE": "CZCE"
      }, {
        "NEXT_TRADE_DAYS": "20220602",
        "EXCHANGE_INT": 6,
        "LAST_TRADE_DAYS": "20220531",
        "TRADE_DAYS": "20220601",
        "EXCHANGE": "CZCE"
      }, {
        "NEXT_TRADE_DAYS": "20220606",
        "EXCHANGE_INT": 6,
        "LAST_TRADE_DAYS": "20220601",
        "TRADE_DAYS": "20220602",
        "EXCHANGE": "CZCE"
      }
    ]
  }
}
```

7.7 交易异动统计

7.7.1 大宗交易数据 A010070001

7.7.1.1 请求参数说明

参数	是否必填	参数说明	备注
function_id	是	功能号	功能号为 A010070001
market_code	是	市场代码	代码.市场，例如 002507.SZ
start_date	是	查询交易日的起始日期	填写 0 表示查询最新日期的数据
end_date	是	查询交易日的结束日期	填写 0 表示查询最新日期的数据

7.7.1.2 返回值说明

字段名称	类型	字段说明	备注
MARKET_CODE	string	市场代码	
EXCHANGE	string	交易市场	参考交易市场代码表
EXCHANGE_INT	int32	交易市场代码	参考交易市场代码表
SECURITY_NAME	string	证券简称	
TYPE_CODE	string	证券类别	参考证券类别代码表
TYPE_CODE_INT	int32	证券类别代码	参考证券类别代码表
TRADE_DATE	string	交易日期	
B_SHARE_PRICE	double	成交价（元）	
B_SHARE_VOLUME	double	成交量（万股）	
B_FREQUENCY	int32	笔数	
BLOCK_AVG_VOLUME	double	每笔成交数量（万股份）	
B_SHARE_AMOUNT	double	成交金额（万元）	
B_BUYER_NAME	string	买方营业部名称	
B_SELLER_NAME	string	卖方营业部名称	

示例数据：

```
{
  "code": "0",
  "msg": "",
  "body": {
    "data": [
      {
        "RN": 1,
        "MARKET_CODE": "000001.SZ",
        "TRADE_DATE": "20230301",
        "B_SHARE_PRICE": 12.7,
        "B_SHARE_VOLUME": 18,
        "B_FREQUENCY": 1,
        "B_SHARE_AMOUNT": 228.6,
        "B_BUYER_NAME": "华泰证券股份有限公司南京止马营证券营业部",

```

```

"B_SELLER_NAME": "华泰证券股份有限公司南京止马营证券营业部",
"BLOCK_AVG_VOLUME": 18,
"SECURITY_NAME": "平安银行",
"TYPE_CODE": "A",
"EXCHANGE": "SZSE",
"EXCHANGE_INT": 102,
"TYPE_CODE_INT": 1001}}}}

```

7.7.2 交易异动营业部买卖信息 A010070002

7.7.2.1 请求参数说明

参数	是否必填	参数说明	备注
function_id	是	功能号	功能号为 A010070002
market_code	是	市场代码	代码.市场, 例如 000037.SZ
start_date	是	查询交易日的起始日期	填写 0 表示查询最新日期的数据
end_date	是	查询交易日的结束日期	填写 0 表示查询最新日期的数据

7.7.2.2 返回值说明

字段名称	类型	字段说明	备注
MARKET_CODE	string	市场代码	
EXCHANGE	string	交易市场	参考交易市场代码表
EXCHANGE_INT	int32	交易市场代码	参考交易市场代码表
SECURITY_NAME	string	证券简称	
TYPE_CODE	string	证券类别	参考证券类别代码表
TYPE_CODE_INT	int32	证券类别代码	参考证券类别代码表
BEG_TRADE_DATE	string	交易起始日期	
END_TRADE_DATE	string	交易截止日期	
CHANGE_RANGE	double	涨跌幅	
TRADER_NAME	string	营业部名称	
BUY_AMOUNT	double	买入金额（元）	
SELL_AMOUNT	double	卖出金额（元）	
FLOW_MARK	int32	买卖标识	1 买入 2 卖出
TOTAL_AMOUNT	double	实际交易金额（元）	
TOTAL_VOLUME	double	实际交易量（万股）	

示例数据:

```

{"code": "0", "msg": "", "body": {"data": [{"RN": 1,

```

```

"MARKET_CODE": "000037.SZ",
"BEG_TRADE_DATE": null,
"END_TRADE_DATE": "20221110",
"CHANGE_RANGE": -10,
"TRADER_NAME": "东方财富证券股份有限公司拉萨东环路第二证券营业部",
"BUY_AMOUNT": 4319856,
"SELL_AMOUNT": 6909597,
"FLOW_MARK": 2,
"TOTAL_AMOUNT": 369300000,
"TOTAL_VOLUME": 4382,
"SECURITY_NAME": "深南电 A",
"TYPE_CODE": "A",
"EXCHANGE": "SZSE",
"EXCHANGE_INT": 102,
"TYPE_CODE_INT": 1001}, {
"RN": 2,
"MARKET_CODE": "000037.SZ",
"BEG_TRADE_DATE": null,
"END_TRADE_DATE": "20221110",
"CHANGE_RANGE": -10,
"TRADER_NAME": "机构专用",
"BUY_AMOUNT": 4005783,
"SELL_AMOUNT": 264227,
"FLOW_MARK": 1,
"TOTAL_AMOUNT": 369300000,
"TOTAL_VOLUME": 4382,
"SECURITY_NAME": "深南电 A",
"TYPE_CODE": "A",
"EXCHANGE": "SZSE",
"EXCHANGE_INT": 102,
"TYPE_CODE_INT": 1001}}}]

```

7.8 融资融券数据

7.8.1 融资融券交易明细 A010080001

7.8.1.1 请求参数说明

参数	是否必填	参数说明	备注
function_id	是	功能号	功能号为 A010080001
market_code	是	市场代码	代码.市场, 例如 002545.SZ

start_date	是	查询交易日的起始日期	填写 0 表示查询最新日期的数据
end_date	是	查询交易日的结束日期	填写 0 表示查询最新日期的数据

7.8.1.2 返回值说明

字段名称	类型	字段说明	备注
MARKET_CODE	string	市场代码	
EXCHANGE	string	交易市场	参考交易市场代码表
EXCHANGE_INT	int32	交易市场代码	参考交易市场代码表
SECURITY_NAME	string	证券简称	
TYPE_CODE	string	证券类别	参考证券类别代码表
TYPE_CODE_INT	int32	证券类别代码	参考证券类别代码表
TRADE_DATE	string	交易日期	
BORROW_MONEY_BALANCE	double	融资余额(元)	
PURCH_WITH_BORROW_MONEY	double	融资买入额(元)	
REPAYMENT_OF_BORROW_MONEY	double	融资偿还额(元)	
SEC_LENDING_BALANCE	double	融券余额(元)	
SALES_OF_BORROWED_SEC	int64	融券卖出量(股,份,手)	
REPAYMENT_OF_BORROW_SEC	int64	融券偿还量(股,份,手)	
SEC_LENDING_BALANCE_VOL	int64	融券余量(股,份,手)	
MARGIN_TRADE_BALANCE	double	融资融券余额(元)	

示例数据:

```
{
  "code": "0",
  "msg": "",
  "body": {
    "data": [
      {
        "RN": 1,
        "MARKET_CODE": "601881.SH",
        "TRADE_DATE": "20230302",
        "BORROW_MONEY_BALANCE": 998481730,
        "PURCH_WITH_BORROW_MONEY": 22567404,
        "REPAYMENT_OF_BORROW_MONEY": 25200688,
        "SEC_LENDING_BALANCE": 4075457,
        "SALES_OF_BORROWED_SEC": 0,
        "REPAYMENT_OF_BORROW_SEC": 0,
        "SEC_LENDING_BALANCE_VOL": 415863,
        "MARGIN_TRADE_BALANCE": 1002557180,
        "SECURITY_NAME": "中国银河",
        "TYPE_CODE": "A",
        "EXCHANGE": "SSE",
        "EXCHANGE_INT": 101,
        "TYPE_CODE_INT": 1001
      }
    ]
  }
}
```

7.8.2 融资融券成交汇总 A010080002

7.8.2.1 请求参数说明

参数	是否必填	参数说明	备注
function_id	是	功能号	功能号为 A010080002
start_date	是	开始日期	格式为 yyyyymmdd 例如: 20220819
end_date	是	结束日期	格式为 yyyyymmdd 例如: 20220827

7.8.2.2 返回值说明

字段名称	类型	字段说明	备注
TRADE_DATE	string	交易日期	
EXCHANGE	string	交易市场	参考交易市场代码表
EXCHANGE_INT	int32	交易市场代码	参考交易市场代码表
SUM_BORROW_MONEY_BALANCE	double	融资余额(元)	
SUM_PURCH_WITH_BORROW_MONEY	double	融资买入额(元)	
SUM_REPAYMENT_OF_BORROW_MONEY	double	融资偿还额(元)	
SUM_SEC_LENDING_BALANCE	double	融券余额(元)	
SUM_SALES_OF_BORROWED_SEC	int64	融券卖出量(股,份,手)	
SUM_MARGIN_TRADE_BALANCE	double	融资融券余额(元)	

示例数据:

```
{
  "code": "0",
  "msg": "",
  "body": {
    "data": [
      {
        "RN": 1,
        "TRADE_DATE": "20220826",
        "EXCHANGE": "SSE",
        "EXCHANGE_INT": 101,
        "SUM_BORROW_MONEY_BALANCE": 813729710000,
        "SUM_PURCH_WITH_BORROW_MONEY": 27455991800,
        "SUM_REPAYMENT_OF_BORROW_MONEY": 30692096000,
        "SUM_SEC_LENDING_BALANCE": 69313798000,
        "SUM_SALES_OF_BORROWED_SEC": 483549760,
        "SUM_MARGIN_TRADE_BALANCE": 883043530000
      }
    ]
  }
}
```

7.9 基金基本信息

7.9.1 基金最新指标 A010090001

7.9.1.1 请求参数说明

参数	是否必填	参数说明	备注
function_id	是	功能号	功能号为 A010090001
market_code	是	市场代码	代码.市场, 例如 150231.SZ

7.9.1.2 返回值说明

字段名称	类型	字段说明	备注
MARKET_CODE	string	市场代码	
EXCHANGE	string	交易市场	参考交易市场代码表
EXCHANGE_INT	int32	交易市场代码	参考交易市场代码表
FOUND_NAME	string	基金简称	
FUND_SHAREANCE"	double	基金份额(万份)	
FLOW_SHARE	double	流通份额(万份)	

示例数据:

```
{
  "code": "0",
  "msg": "",
  "body": {
    "data": [
      {
        "MARKET_CODE": "150231.SZ",
        "FOUND_NAME": "电子 A",
        "EXCHANGE": "SZSE",
        "EXCHANGE_INT": 102,
        "FUND_SHARE": 1805.0822,
        "FLOW_SHARE": 1805.0822
      }
    ]
  }
}
```

7.10 指数信息

7.10.1 A 股指数成分股 A010200001

7.10.1.1 请求参数说明

参数	是否必填	参数说明	备注
function_id	是	功能号	功能号为 A010200001
index_code	是	市场代码	代码.市场, 例如 000902.SH

7.10.1.2 返回值说明

字段名称	类型	字段说明	备注
INDEX_CODE	string	指数代码	
EXCHANGE_STR	string	交易市场	参考交易市场代码表
EXCHANGE_INT	int32	交易市场代码	参考交易市场代码表
INDEXNAME	string	指数简称	
CON_CODE	string	成分股市场代码	例如 688701.SH
RN	int32	序号	

示例数据:

```
{
  "code": "0",
  "msg": "",
  "body": {
    "data": [
      {
        "RN": 1,
        "INDEX_CODE": "000902.SH",
        "CON_INDATE": 20221230,
        "CON_CODE": "872925.BJ",
        "INDEXNAME": "中证流通(SH)",
        "EXCHANGE_STR": "SSE",
        "EXCHANGE_INT": 101
      },
      {
        "RN": 2,
        "INDEX_CODE": "000902.SH",
        "CON_INDATE": 20221230,
        "CON_CODE": "871981.BJ",
        "INDEXNAME": "中证流通(SH)",
        "EXCHANGE_STR": "SSE",
        "EXCHANGE_INT": 101
      }
    ]
  }
}
```

7.11 资讯数据信息

7.11.1 查询资讯数据更新日期 queryLastFuncDataTime

7.11.1.1 请求参数说明

参数	是否必填	参数说明	备注
function_id	是	功能号	功能号为 queryLastFuncDataTime
queryfuncid	是	查询更新日期的资讯数据功能号	本文中定义的任一资讯数据查询功能号, 例如 A010020002 表示查询 A 股增发数据的最新数据更新时间

7.11.1.2 返回值说明

字段名称	类型	字段说明	备注
LAST_UPD_TIME	string	当前指定资讯数据的最新数据更新时间	时间字符格式 YYYY-MM-DD hh:mm:ss

示例数据：

```
{"code": "0", "msg": "", "body": {"data": [{"LAST_UPD_TIME": "2023-05-30 00:08:58"}]}}
```

7.12 公共参数表

7.12.1 交易市场代码表

交易市场	交易市场代码	备注
SSE	101	上交所
SZSE	102	深交所
BSE	2	北交所
SZN	253	深港通中深股通
SHN	254	沪港通中沪股通
SHFE	3	上海商品交易所
CFFEX	4	中金期货交易所
DCE	5	大连商品交易所
CZCE	6	郑州商品交易所
INE	7	上海能源交易所

7.12.2 证券类别代码表

证券类别	证券类别代码	备注
A	1001	A 股
B	1002	B 股

7.12.3 公司类型代码表

公司类别名称	公司类别代码
证券公司	1
证券公司(经纪类)	2
信托公司	3
保险公司	4
保险公司(寿险)	5
保险公司(财险)	6
保险经纪公司	7
保险代理公司	8
期货公司	9
期货经纪公司	10
商业银行	11
金融租赁公司	12

财务公司	13
投资咨询公司	14
其他金融机构	15
交易所	16
其他公司	17
基金管理公司	18
投资银行	19
金融资产管理公司	20
担保公司	21
金融控股集团	22
社会团体	23
机关事业单位	24
企业集团	25
会计师事务所	26
外资金融机构	27
律师事务所	28
政策性银行	29
社保机构	30
香港创业板上市公司	31
香港主板上市公司	32
信用社	33
停业公司	34
基金	35
持股会	36
国有资产管理机构	37
QFII 机构	38
保险资产管理公司	39
国资委直属企业	40
基金会	41
保险公司(车险)	42
保险公司(健康险)	43
保险公司(农业险)	44
保险公司(养老险)	45
农村商业银行	46
企业年金	47
证券公司集合理财	48
中央部委	49
保险公估公司	50
保险公司产品	51
信托公司产品	52
阳光私募	53
证券公司专项产品	54
养老金产品	55

保险债权投资计划	56
证券公司定向产品	57
保险资产管理产品	58
基金子公司产品	59
互联网理财产品	60
券商收益凭证	61
期货资产管理产品	62
养老保障管理产品	63
券商资产管理公司	64

7.12.4 股票增发进度代码表

增发进度描述	进度代码
董事会预案	1
股东大会通过	2
实施	3
未通过	4
证监会批准	5
达成转让意向	6
签署转让协议	7
国资委批准	8
商务部批准	9
过户	10
延期实施	11
停止实施	12
分红方案待定	13
传闻	14
证监会受理	15
传闻被否认	16
股东提议	17
保监会批复	18
董事会预案预披露	19
发审委通过	20
发审委未通过	21
股东大会未通过	22
银监会批准	23
证监会恢复审核	24

7.12.5 股票分红进度代码表

分红进度描述	进度代码
--------	------

董事会预案	1
股东大会通过	2
实施	3
未通过	4
停止实施	12
股东提议	17
董事会预案预披露	19

分红实施进程：股东提议—董事会预案—股东大会—实施

7.12.6 股票配股进度代码表

配股进度描述	进度代码
董事会预案	1
股东大会通过	2
实施	3
未通过	4
证监会核准	5
达成转让意向	6
签署转让协议	7
国资委批准	8
商务部批准	9
过户	10
延期实施	11
停止实施	12
分红方案待定	13
传闻	14
证监会受理	15
传闻被否认	16
股东提议	17
保监会批复	18
董事会预案预披露	19
发审委通过	20
发审委未通过	21
股东大会未通过	22
银监会批准	23
证监会恢复审核	24
预发行	25
提交注册	26

7.12.7 报表类型代码表

报表类型	报表类型代码	备注
1	合并报表	涵盖母公司的财务报表数据，为最新报表
2	合并报表(单季度)	合并报表(单季度)=合并报表(本期)-合并报表(上一季)
3	合并报表(单季度调整)	合并报表(单季度调整)=合并报表(本期调整)-合并报表(上一季调整)
4	合并报表(调整)	本年度公布上年同期的财务报表数据，报告期为上年度
5	合并报表(更正前)	即出更正公告后，把合并报表的记录修改为合并报表(更正前)；复制原来的记录，更正后报表类型改为合并报表
6	母公司报表	该公司母公司的财务报表数据
7	母公司报表(单季度)	母公司报表(单季度)=母公司报表(本期)-母公司报表(上一季)
8	母公司报表(单季度调整)	母公司报表(单季度调整)=母公司报表(本期调整)-母公司报表(上一季调整)
9	母公司报表(调整)	该公司母公司的本年度公布上年同期的财务报表数据
10	母公司报表(更正前)	之前上市公司已披露财务报表数据，但是由于某些特定原因导致出错，未调整之前的原始财务报表数据。
19	合并报表(调整借壳前)	借壳前的合并报表(调整)
21	合并报表(单季度借壳前)	借壳前的合并报表(单季度)
22	合并报表(单季度调整借壳前)	借壳前的合并报表(单季度调整)
23	母公司报表(借壳前)	借壳前的母公司报表
24	母公司报表(调整借壳前)	借壳前的母公司报表(调整)
25	母公司报表(单季度借壳前)	借壳前的母公司报表(单季度)
26	母公司报表(单季度调整借壳前)	借壳前的母公司报表(单季度调整)
27	合并报表(第一次更正)	有多次更正时，合并报表的第一次更正
28	合并报表(第二次更正)	有多次更正时，合并报表的第二次更正
29	合并调整(第一次更正)	有多次更正时，合并调整的第一次更正
33	母公司调整(第一次更正)	有多次更正时，母公司调整的第一次更正
34	母公司报表(第二次更正)	有多次更正时，母公司报表的第二次更正
35	母公司报表(第一次更正)	有多次更正时，母公司报表的第一次更正
50	合并调整(更正前)	即出更正公告后，把合并报表(调整)的记录修改为合并调整(更正前)；复制原来的记录，更正后报表类型改为合并报表(调整)
60	母公司调整(更正前)	该公司母公司的本年度公布上年同期的财务报表数据，但是由于某些特定原因导致出错，未调整之前的原始财务报表数据。